arXiv:2409.08036v2 [cs.LG] 16 Apr 2026

Heterogeneous Sheaf Neural Networks

Luke Braithwaite[1*] **Alessio Borgi**[1,3†*] **Gabriele Onorato**[2,3*] **Kristjan Tarantelli**[3*]
Iulia Duta[1] **Francesco Restuccia**[2] **Fabrizio Silvestri**[3] **Pietro Liò**[1]
[1]Department of Computer Science and Technology, University of Cambridge
[2]Department of Electrical and Computer Engineering, Northeastern University
[3]Department of Computer, Control and Management Engineering, Sapienza University of Rome

Abstract

Heterogeneous graphs, whose nodes and edges may belong to different types and feature spaces, arise in a wide variety of real-world domains such as biology, chemistry and computer networks. Existing methods typically address this heterogeneity by modifying the model architecture itself, which often results in specialized and parameter-intensive designs. To address this issue, we propose HetSheaf, a framework that models heterogeneous relational data through *cellular sheaves*, which provide a principled topological framework for encoding type-specific local feature spaces and their interactions directly in the data representation. We also introduce a family of heterogeneous sheaf predictors that learn restriction maps conditioned on node and edge types. To enable graph-level predictions, we further propose SheafPool, a graph pooling mechanism that aggregates node representations in stalk space while remaining invariant to local changes of basis, ultimately enabling stalk-space graph-level representations for the first time. HetSheaf achieves strong predictive performance on standard heterogeneous graph benchmarks, over numerous tasks such as node/graph classification, link prediction and recommendation, while reducing by up to `10x` the number of parameters with respect to state-of-the-art baselines.

1 Introduction

Graph Neural Networks (GNNs) learn relational data by combining node features with graph structure through iterative message passing [6, 14, 27–29, 58, 65]. Their success has led to widespread adoption across a variety of domains such as biology, chemistry and computer networks, where entities and their interactions are naturally represented as graphs. While most standard GNNs are designed for *homogeneous* graphs, where all nodes and edges are assumed to belong to the same type and to be represented in a shared feature space, many real-world relational systems – for example, protein prediction [24], recommendation systems [18, 76], social network analysis [45, 56], and traffic prediction [31] – are best represented as *heterogeneous graphs* [15, 39, 75], with nodes and edges having different types and carrying features with different semantic meanings or dimensionalities. In these cases, GNNs struggle to process heterogeneous data mostly due to *oversmoothing* [48]. The latter refers to the case where neighboring nodes have increasingly similar representations. As the GNN becomes deeper, oversmoothing causes loss of type-specific features and information, thus significantly reducing the model's performance [4, 7, 10].

The key issue is that existing *Heterogeneous GNNs* (HGNNs) represent the data as a simple graph and attempt to account for the heterogeneity only in the model architecture [40, 59, 68, 79]. Recent literature [2–4, 17] has demonstrated that attaching a *cellular sheaf* [12,

*Equal contribution. †Corresponding author: `ab3352@cam.ac.uk`

Preprint.

60] to a standard graph helps mitigate oversmoothing and increases the expressivity by generalizing the diffusion process through a learned Sheaf Laplacian [36], which prevents feature homogenization and captures complex, asymmetric edge relations. This makes sheaves particularly well-suited for heterogeneous settings: restriction maps bridge distinct feature spaces, allowing entities to interact while preserving type-specific semantics. Motivated by this perspective, we introduce HetSheaf, a framework that learns a cellular sheaf encoding the relevant type-specific information, by representing heterogeneous relational data through a structure (fig. 1b) that explicitly preserves its local feature spaces and interactions. At the same time, moving from graphs to sheaves also makes global aggregation and readout more delicate. Because node representations now live in local vector spaces rather than in a single common coordinate system, standard pooling strategies cannot be adopted naively. A meaningful sheaf readout must account for the geometry of these stalk spaces. To address this challenge we introduce SheafPool – a novel aggregation mechanism that explicitly handles for these distinct geometries and ensures consistent global aggregation.

Contributions. Our main contributions can be summarised as follows:

1. We propose HetSheaf, a general framework for heterogeneous sheaf neural networks that shifts the modelling of heterogeneity from architecture design to the underlying data representation through cellular sheaves.
2. We demonstrate that Sheaf Graph Neural Networks provide an effective and parameter-efficient approach for heterogeneous learning, achieving *competitive and often state-of-the-art performance across several benchmark datasets and tasks.*
3. We develop a *family of heterogeneous sheaf predictors* that incorporate node and edge type information directly into the prediction of restriction maps, yielding sheaf structures tailored to heterogeneous relational data.
4. We propose SheafPool, a universal *stalk-space readout* for graph classification that is both permutation-invariant and gauge-invariant, making graph-level prediction with sheaf neural networks well defined and practically effective for the first time.
5. We evaluate HetSheaf and SheafPool across a diverse suite of heterogeneous node classification, link prediction, and graph classification benchmarks. We achieve strong predictive performance and outperform both standard heterogeneous GNNs and type-agnostic sheaf models, while using up to 10x fewer parameters than existing heterogeneous baselines.

2 Background and Related Work

Homogeneous and Heterogeneous Graph Neural Networks. A wide range of GNNs have been proposed for *homogeneous* graphs [11, 13, 34, 42, 46, 67]. Although these models can be directly applied to heterogeneous graphs by ignoring node and edge types, they are not explicitly designed to capture the type-specific structure of heterogeneous relational data. While in homogeneous graphs all nodes/edges encode the same type of data, in *Heterogeneous GNNs*, instead, nodes and edges can represent different types of information in heterogeneous graphs. Several HGNNs have been proposed: a specific typology of them, use *meta-paths* [62, 63], i.e., pre-defined node or edge type patterns that aim to capture important semantic information. HAN [68] employs a hierarchical attention mechanism to capture node-level importance among nodes and semantic-level importance across meta-paths. MAGNN [25] combines several meta-path encoders to aggregate all information along the meta-path. Both HAN and MAGNN require meta-paths to be generated manually in advance. Graph Transformer Networks (GTNs) [77] automate this process by learning meta-paths with graph transformation layers. However, Meta-paths are computationally expensive, making them infeasible for graphs with many distinct edge types. Instead, the graph may be treated as a relational graph. RSHN [83] constructs a coarsened line graph to generate edge features, and then applies a Message Passing Neural Network (MPNN) [26] to propagate node and edge features. R-GCN [59] splits the heterogeneous graph into the subgraph induced by each edge type, then applies a standard GCN to each subgraph. HGT [79] is a graph transformer that can process large heterogeneous graphs using subgraph sampling. HetGNN [79] uses random walks to sample strongly correlated heterogeneous neighbours and

then applies an RNN for representation learning. SlotGAT [81] uses a slot-based message passing approach with an attention mechanism for aggregation. SimpleHGNN [48], applies a series of additional techniques to lift GATs to heterogeneous domains and Space4HGNN [80] defines a unified design space of modular components for heterogeneous GNNs and allows exhaustive evaluation of many combinations of techniques. More recently, HAGNN [82] meta-path-based intra-type aggregation with meta-path-free inter-type aggregation through a fused meta-path graph, while HGNN-AR^2 [47] adaptively reconstructs relations to address same-category connection deficiencies in meta-path methods. Instead of introducing a new architecture for heterogeneous graphs, we propose shifting from representing heterogeneous relational data as graphs to cellular sheaves, which preserve all type-specific information and enable powerful processing with sheaf-specific tools.

Sheaf Neural Networks. Standard GNNs are known to suffer from limitations such as oversmoothing [7], oversquashing [1] and degradation of performances in heterophilic settings, i.e., when adjacent nodes belong to different classes or carry contrasting attributes. Cellular sheaf theory [12, 60] equips graphs with local feature spaces, or *stalks*, together with linear restriction maps along edges, thereby providing a principled geometric framework for modelling and alleviating these pathologies. Sheaf Neural Networks were originally introduced using an hand-crafted sheaf with a single dimensionality in [35], and further improved by allowing to learn the sheaf using a learnable parametric function in Bodnar et al. [4]. Since then, sheaf-based graph learning has been extended in several directions: attention mechanisms over sheaves [2], connection Laplacians learned during preprocessing [3], sheaf-based positional encodings [37], nonlinear diffusion processes [78], joint diffusion biases [8], polynomial spectral filtering [5], and directional extensions [23]. Beyond standard graphs, the sheaf framework has also been extended to hypergraphs [17, 51], recommendation systems [55], federated learning [52], cooperative sheaf dynamics [57], and more general topological constructions via copresheaves [32]. Existing sheaf neural networks have focused almost exclusively on node-level tasks, leaving graph-level prediction largely unexplored. Indeed, graph-level readout in sheaf networks is more subtle than standard aggregation, since a valid pooling operator must not only be permutation-invariant over nodes, but also invariant to node-wise stalk reparameterisations. Existing graph pooling methods [30, 41, 43, 46] satisfy the former requirement but not the latter, and therefore cannot be directly applied in a principled way to sheaf representations.

Cellular Sheaves on Graphs. Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a finite undirected graph endowed with an arbitrary orientation on its edges. A *cellular sheaf* $\mathcal{F}$ on $\mathcal{G}$ assigns a finite-dimensional vector space, or *stalk*, $\mathcal{F}(u)$ to each node $u \in \mathcal{V}$ and $\mathcal{F}(e)$ to each edge $e \in \mathcal{E}$, together with linear *restriction maps* $\mathcal{F}_{u \trianglelefteq e} : \mathcal{F}(u) \to \mathcal{F}(e)$ for every incident node-edge pair $u \trianglelefteq e$. By stacking the node and edge stalks, we obtain the spaces of 0- and 1-cochains, $C^0(\mathcal{G}, \mathcal{F}) \coloneqq \bigoplus_{u \in \mathcal{V}} \mathcal{F}(u), \quad C^1(\mathcal{G}, \mathcal{F}) \coloneqq \bigoplus_{e \in \mathcal{E}} \mathcal{F}(e)$. Given an oriented edge $e = (u \to v)$, these spaces induce the *coboundary map* $\delta : C^0(\mathcal{G}, \mathcal{F}) \to C^1(\mathcal{G}, \mathcal{F})$, defined edgewise by $\delta(\mathbf{x})_e \coloneqq \mathcal{F}_{v \trianglelefteq e}\mathbf{x}_v - \mathcal{F}_{u \trianglelefteq e}\mathbf{x}_u$. The coboundary measures disagreement between neighbouring node assignments after they are mapped into the shared edge space. The corresponding *sheaf Laplacian* is then defined as $L_{\mathcal{F}} \coloneqq \delta^\top \delta$, and acts node-wise as $L_{\mathcal{F}}(\mathbf{x})_u = \sum_{u, v \trianglelefteq e} \mathcal{F}^\top_{u \trianglelefteq e}\big(\mathcal{F}_{u \trianglelefteq e}\mathbf{x}_u - \mathcal{F}_{v \trianglelefteq e}\mathbf{x}_v\big)$. Its normalised version is then given by $\Delta_{\mathcal{F}} \coloneqq D^{-1/2} L_{\mathcal{F}} D^{-1/2}$, where D is the block diagonal of $L_{\mathcal{F}}$. Unless otherwise stated, we assume all stalks have the same dimension d, i.e. $\mathcal{F}(u) = \mathbb{R}^d$ and $\mathcal{F}(e) = \mathbb{R}^d$, so that $L_{\mathcal{F}} \in \mathbb{R}^{dn \times dn}$. In the special case where all stalks are $\mathbb{R}$ and all restriction maps are the identity, the sheaf Laplacian reduces to the standard graph Laplacian.

Neural Sheaf Diffusion. Given node features $\mathbf{x}_u \in \mathcal{F}(u)$, stacking them yields a cochain $\mathbf{x} \in C^0(\mathcal{G}, \mathcal{F})$. With f feature channels, this becomes a feature matrix $\mathbf{X} \in \mathbb{R}^{nd \times f}$. *Sheaf diffusion* generalises standard graph diffusion by propagating information through the geometry induced by $\Delta_{\mathcal{F}}$ rather than the ordinary graph Laplacian. In Neural Sheaf Diffusion (NSD) [4], the sheaf structure is learned directly from data via the discrete update: $\mathbf{X}^{(t+1)} = \mathbf{X}^{(t)} - \sigma\Big(\Delta_{\mathcal{F}(t)}\big(\mathbf{I}_{nd} \otimes \mathbf{W}_1^{(t)}\big)\mathbf{X}^{(t)}\mathbf{W}_2^{(t)}\Big)$, where $\mathbf{W}_1^{(t)}$ and $\mathbf{W}_2^{(t)}$ are learnable weight matrices, and the restriction maps defining $\Delta_{\mathcal{F}(t)}$ are predicted by a parametric function $\mathcal{F}_{u \trianglelefteq e \coloneqq (u,v)} = \boldsymbol{\Phi}(\mathbf{x}_u, \mathbf{x}_v) = \mathrm{MLP}(\mathbf{x}_u \| \mathbf{x}_v)$, with $\|$ denoting vector concatenation. This yields a data-dependent sheaf Laplacian and, consequently, a learned diffusion operator.

3 Heterogeneous Sheaf Neural Networks

Heterogeneous Graph Setting. We consider a heterogeneous graph [62] $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \phi, \psi)$, where $\mathcal{V}$ and $\mathcal{E}$ denote the sets of nodes and edges, while $\phi : \mathcal{V} \to \Phi$ and $\psi : \mathcal{E} \to \Psi$ are node and edge type mapping functions respectively. For an edge $e = (u, v)$, we interchangeably write its type as $\psi(e)$ or $\psi(u, v)$. Each node $u \in \mathcal{V}$ is associated with a feature vector $\mathbf{x}_u$, whose dimensionality may depend on the node type. Concretely, for every type $t \in \Phi$, all nodes of type t, defined as $\{u \in \mathcal{V} \mid \phi(u) = t\}$, share the same input dimension d_0^t, so that $\mathbf{x}_u \in \mathbb{R}^{d_0^{\phi(u)}}$.

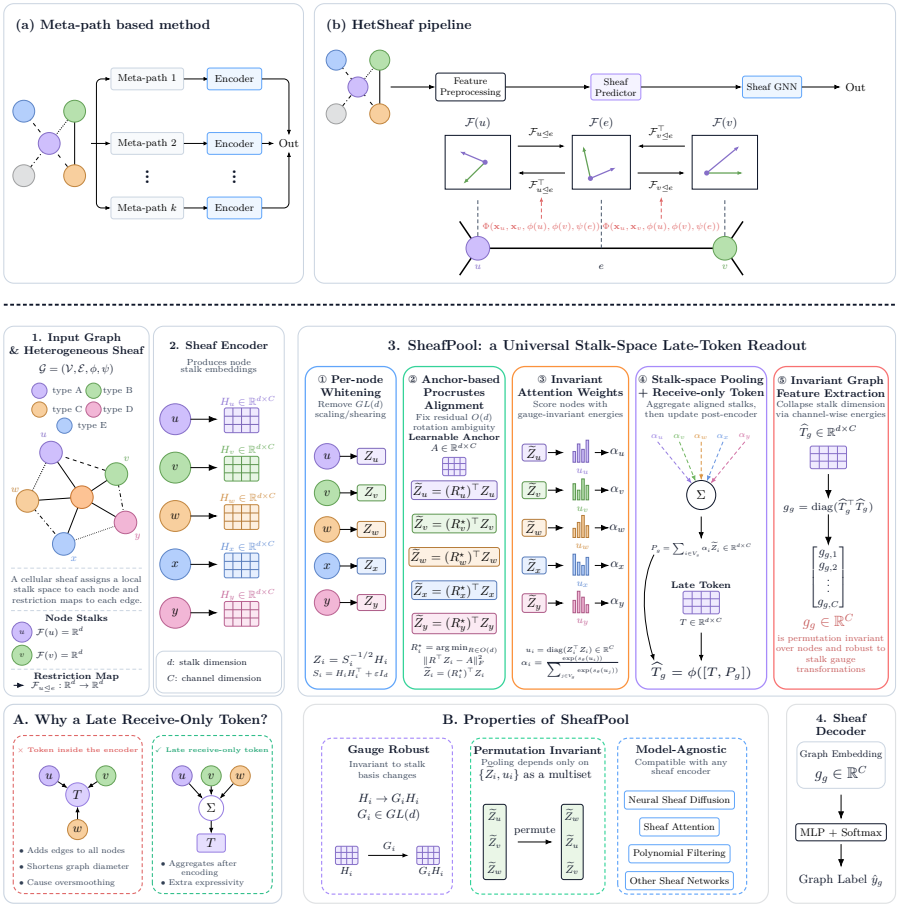


Figure 1: **HetSheaf with SheafPool.** *(Top)*: *(a)* Conventional heterogeneous GNNs model heterogeneity through architecture-level components such as meta-path extractors or relation-specific message passing. *(b)* HetSheaf, instead, captures heterogeneity directly in the data representation by inferring a cellular sheaf from node features and node/edge types, yielding a heterogeneous sheaf that can be processed by any sheaf neural network. *(Bottom)*: *(1-4)* the proposed readout first encodes the heterogeneous sheaf into node stalk embeddings, then whitens them and aligns them to a shared anchor to remove gauge ambiguity. Gauge-invariant attention weights are computed from channel-wise energies and used to pool the aligned stalks in stalk space. The pooled representation updates a *late receive-only token (A)*, introduced only after encoding so that it does not interfere with message passing. Finally, channel-wise energies of the updated token produce a graph embedding that is permutation invariant and robust to local stalk gauge transformations *(B)*, which is then passed to a downstream decoder for graph classification.

3.1 HetSheaf: A Sheaf-based framework for Heterogeneous data

Existing heterogeneous GNNs (Figure 1a) handle heterogeneity at the architectural level often leading to increasingly specialised and parameter-heavy designs, scaling with the number of entity types and making the treatment of heterogeneity tightly coupled to the neural architecture itself. For this reason, we instead take a *representation-centric view* by modifying the input structure to encode specific heterogeneous knowledge while maintaining topological information. Unlike standard graphs, which assume a single shared feature space for all nodes, cellular sheaves naturally capture heterogeneous data through type-specific local vector spaces and restriction maps that preserve local structure while enabling coherent global message passing. Motivated by this perspective, we introduce HetSheaf(Figure 1b), a general framework for heterogeneous sheaf neural networks. Given a heterogeneous graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \phi, \psi)$, it first preprocesses type-dependent node features into a common channel dimension, then infers a cellular sheaf whose restriction maps are conditioned on both features and type information, and finally applies a sheaf neural architecture for downstream prediction. More formally, a heterogeneous sheaf neural network consists of three components:

1. *Type-aware preprocessing*, which projects type-specific input features into a shared channel space. Indeed, as each node type may have a different number of feature channels, we use a linear layer with bias for each node type to project the node features to the same number of input channels. This can be combined with the sheaf projection step of the SheafGNN to create the sheaf cochain $\mathbf{x} \in C^0(\mathcal{G}, \mathcal{F})$. The modified linear layer takes in the original feature channels f_0 and outputs df features as the node stalk projection. We follow [48, 81] to generate the input feature types by using either all of the node feature types (type-0), only the feature of the node target type (type-1) or replacing all node features as one-hot vectors (type-2).
2. *Heterogeneous sheaf inference*, (Figure 2) which predicts restriction maps conditioned on node features and type information.
3. *Sheaf-based representation learning*, which applies a sheaf neural encoder followed by a task-specific decoder or readout. Similar to Lv et al. [48], we perform ℓ_2 normalisation on the output embedding, $\mathbf{o}_u = \frac{\mathbf{h}_u^{(L)}}{\|\mathbf{h}_u^{(L)}\|}$, where $\mathbf{o}_u$ is the final output embedding for a node $u \in \mathcal{V}$ and $\mathbf{h}_u^{(L)}$ is the final hidden representation from the SheafGNN. For link prediction, we adapt JKNet [72] to create the final embedding by concatenating the hidden representations for each layer, i.e.: $\mathbf{o}_u = \Big\|_{l=1}^{L} \frac{\mathbf{h}_u^{(l)}}{\|\mathbf{h}_u^{(l)}\|}$.

Let $\mathcal{P}$ denote the preprocessing module, $\mathbf{\Phi}$ the sheaf predictor, $\mathcal{M}$ a sheaf neural encoder, and $\mathcal{D}$ a downstream decoder. The overall pipeline can be written abstractly as:

$$\mathbf{z}_u = \mathcal{P}_{\phi(u)}(\mathbf{x}_u) \tag{1}$$

$$\mathcal{F}_{u \trianglelefteq e := (u,v)} = \mathbf{\Phi}(\mathbf{z}_u, \mathbf{z}_v, \phi(u), \phi(v), \psi(e)) \tag{2}$$

$$\hat{\mathbf{y}} = \mathcal{D}(\mathcal{M}(\mathcal{G}, \mathcal{F}, \{\mathbf{z}_u\}_{u \in \mathcal{V}})) \tag{3}$$

Since heterogeneity is encoded into the inferred sheaf itself, once the sheaf has been constructed, it can be processed by any sheaf-based neural architecture, including Neural Sheaf Diffusion [4], Sheaf Attention Networks [2], Polynomial Neural Sheaf Diffusion [5] or other sheaf variants, without requiring ad hoc heterogeneous redesign.

3.2 Heterogeneous Sheaf Predictors, Readout, and Model Adaptations

Heterogeneous Sheaf Predictors. A key ingredient of HetSheaf is the explicit inference of restriction maps from heterogeneous information. In standard sheaf learning, restriction maps are predicted from node features alone as $\mathcal{F}_{u \trianglelefteq (u,v)} = \mathbf{\Phi}(\mathbf{x}_u, \mathbf{x}_v)$. Although such a predictor may implicitly recover type-dependent behaviour, it does not explicitly exploit the typed structure available in heterogeneous graphs. We therefore parameterise the restriction maps so that the learned sheaf structure is directly conditioned on node features, node and edge types as described in Eq 1. Following [4], we consider diagonal, orthogonal, and general restriction-map families (see more details in Section B.4.3), and we instantiate the

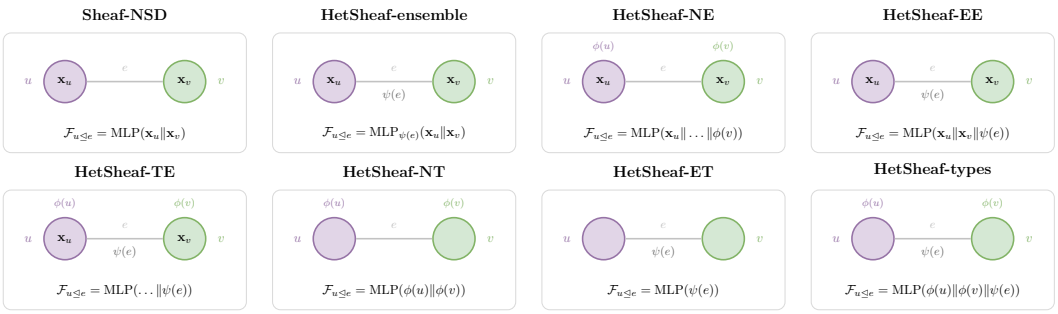


Figure 2: **Heterogeneous Sheaf Predictors.** For an edge $e = (u, v)$, $\phi(u)$ and $\phi(v)$ denote node types, $\psi(e)$ denotes the edge type, x_u, x_v are node features, $\|$ denotes concatenation, and $\text{MLP}_{\psi(e)}$ denotes a separate MLP for each edge type.

heterogeneous sheaf through a *family of heterogeneous sheaf predictors* that vary along two axes. The former concerns *which type signals are used*: from no type information (Sheaf-NSD, where we only concatenate information relative to the node features $\mathbf{x}_u, \mathbf{x}_v$), to node-only (HetSheaf-NE, HetSheaf-NT) or edge-only (HetSheaf-EE, HetSheaf-ET) variants, up to predictors conditioned on both node and edge types (HetSheaf-TE, HetSheaf-types). The latter regards instead *how type information is routed*: most variants use a shared MLP with type indicators as additional inputs, whereas HetSheaf-ensemble allocates a separate predictor to each edge type $\text{MLP}_{\psi(e)}$, increasing expressivity at the cost of additional parameters. Figure 2 summarises the predictor family, while full details and complexity analyses are deferred to Section B.4.4.

Restriction Map Types. A further design choice concerns how the raw output of $\mathbf{\Phi}$ is converted into a restriction map as in Bodnar et al. [4]. Alternatively, one may apply an attention-style post-processing step. For the general case, a row-wise softmax yields row-stochastic restriction maps, and this transformation depends only on the output of $\mathbf{\Phi}$, applying uniformly to all heterogeneous predictor variants:

$$\mathcal{F}_{u \trianglelefteq e} = \mathbf{I}_d - \text{softmax}(\mathbf{\Phi}(\mathbf{x}_u, \mathbf{x}_v)) \tag{4}$$

Task-specific Preprocessing and Readout. Because different node types may have different input dimensionalities, we use a type-specific linear projection with bias to map all node features into a shared channel dimension. These projections are learned jointly with the rest of the model and can be combined with the initial stalk projection of the sheaf encoder to construct the input cochain $\mathbf{x} \in C^0(\mathcal{G}, \mathcal{F})$. Following Lv et al. [48] and Zhou et al. [81], we consider multiple feature initialisation strategies, including using all available node features, target-type-only features, and one-hot type indicators. For *node classification*, we apply ℓ_2 normalisation to the final node representation: $\mathbf{o}_u = \frac{\mathbf{h}_u^{(L)}}{\|\mathbf{h}_u^{(L)}\|}$, where $\mathbf{h}_u^{(L)}$ is the final hidden state of node u. For *link prediction*, we follow a JKNet-style strategy [72] and concatenate the normalised hidden states across layers: $\mathbf{o}_u = \big\|_{l=1}^{L} \frac{\mathbf{h}_u^{(l)}}{\|\mathbf{h}_u^{(l)}\|}$.

Sheaf Models Adaptations. A key advantage of HetSheaf is its model-agnosticity: once a heterogeneous sheaf has been inferred, the resulting typed geometric structure can be processed by a broad class of existing sheaf neural architectures with only minimal modifications. To demonstrate this property, we adapt both *Sheaf Attention Networks* (SheafAN) [2] and *Polynomial Neural Sheaf Diffusion* (PolyNSD) [5] to the heterogeneous setting. For SheafAN, we make the attention mechanism type-aware by augmenting the attention input with node-type and edge-type information. For PolyNSD, instead, we preserve the original polynomial spectral filtering scheme and apply it directly to the heterogeneous sheaf Laplacian $p(L_{\mathcal{F}}) = \sum_{k=0}^{K} c_k L_{\mathcal{F}}^k$, where $L_{\mathcal{F}}$ is now induced by the type-aware restriction maps learned by HetSheaf, allowing to perform higher-order diffusion directly over the heterogeneous sheaf without requiring any ad hoc redesign. Full model-specific derivations and implementation details are deferred to Sections B.4.6 and B.4.7.

4 SheafPool: Universal Stalk-Space Late-token Readout

Graph-level prediction is more delicate in sheaf neural networks than in standard GNNs. While relying on the common latent space assumption, for which we can apply pooling hierarchy, sheaf node embeddings $\mathbf{H}_i \in \mathbb{R}^{d\times C}$ are meaningful only up to node-wise changes of basis. Consequently, a valid graph readout must satisfy two requirements simultaneously: it must be permutation-invariant over nodes, and it must be compatible with stalk-wise gauge reparameterisations $\mathbf{H}_i \mapsto \mathbf{G}_i\mathbf{H}_i, \mathbf{G}_i \in \mathcal{G}$, where $\mathcal{G}$ denotes the gauge family induced by the restriction-map parameterisation (e.g. diagonal, orthogonal, or general invertible maps). Indeed, naively applying flatten-and-pool operations on $\mathrm{vec}(\mathbf{H}_i) \in \mathbb{R}^{dC}$ generally violate this requirement, since they depend on arbitrary local stalk coordinates, in addition to make the sheaf loose their strength (as also shown in Table 1). To address this issue, we introduce SheafPool, a *Universal Stalk-Space Late-token Readout* graph pooling mechanism that first canonicalises node embeddings, then aggregates them directly in stalk space, and only collapses the stalk dimension at the very end. The construction preserves the stalk geometry, while remaining applicable all sheaf families.

Table 1: HetSheaf Pooling ablation over graph classification. Best results per dataset in **bold**.

	MUTAG		PROTEINS		ENZYMES		NCI1	
Pooling	Macro F1	Micro F1	Macro F1	Micro F1	Macro F1	Micro F1	Macro F1	Micro F1
Mean	94.88	95.00	57.43	66.96	19.87	23.33	57.93	57.93
Max	82.58	82.50	67.78	73.21	13.63	15.00	64.06	64.06
Add	74.24	74.30	65.76	66.96	9.98	16.67	57.50	57.50
Attention [43]	82.91	85.00	69.05	76.79	13.69	16.67	65.66	65.66
SheafPool (ours)	**100.00**	**100.00**	**80.56**	**82.14**	**62.35**	**63.33**	**79.46**	**79.46**

Universal Stalk-Space Late-Token Formalisation. Formally, we define a graph-level readout as $\mathcal{R} : \{\mathbf{H}_i\}_{i\in V_g} \longmapsto \mathbf{g}_g \in \mathbb{R}^C$, whose output is permutation-invariant over nodes and robust to local stalk reparameterisations. The readout consists in five stages that can be summarised as:

$$\mathbf{H}_i \xrightarrow{\text{whiten}} \mathbf{Z}_i \xrightarrow{\text{align}} \widetilde{\mathbf{Z}}_i \xrightarrow{\text{attn pool}} \mathbf{P}_g \xrightarrow{\text{late token}} \widehat{\mathbf{T}}_g \xrightarrow{\text{energy}} \mathbf{g}_g \tag{5}$$

(1) Per-node whitening. We first remove node-wise scaling and shearing in the stalk dimension by forming $\mathbf{S}_i = \mathbf{H}_i\mathbf{H}_i^\top + \varepsilon\mathbf{I}_d$, and defining $\mathbf{Z}_i = \mathbf{S}_i^{-1/2}\mathbf{H}_i$. This approximately isotropises the stalk covariance, reducing the remaining ambiguity to an orthogonal transform.

(2) Anchor-based alignment. After whitening, different nodes may still be represented in different orthogonal frames. We therefore align each node to a shared learnable anchor $\mathbf{A} \in \mathbb{R}^{d\times C}$ via $\mathbf{R}_i = \arg\min_{\mathbf{R}\in O(d)} \|\mathbf{R}^\top\mathbf{Z}_i - \mathbf{A}\|_F^2$, and set $\widetilde{\mathbf{Z}}_i = \mathbf{R}_i^\top\mathbf{Z}_i$. This places all node embeddings in a common stalk frame before pooling.

(3) Invariant attention weights. To weight node contributions without using basis-dependent coordinates, we score each node through the channel-wise energies of its whitened representation $\mathbf{u}_i = \mathrm{diag}(\mathbf{Z}_i^\top\mathbf{Z}_i) \in \mathbb{R}^C$. Since $\mathbf{u}_i$ is invariant to residual orthogonal reparameterisations, a learnable scorer s_θ can produce graph-wise attention weights $\alpha_i = \frac{\exp(s_\theta(\mathbf{u}_i))}{\sum_{j\in V_g}\exp(s_\theta(\mathbf{u}_j))}$.

(4) Stalk-space pooling with a late receive-only token. We then aggregate the aligned embeddings directly in stalk space $\mathbf{P}_g = \sum_{i\in V_g}\alpha_i\widetilde{\mathbf{Z}}_i \in \mathbb{R}^{d\times C}$. To increase expressivity without inserting a hub token into the encoder, which would alter message passing and may exacerbate oversmoothing, we use a *late, receive-only token* $\mathbf{T} \in \mathbb{R}^{d\times C}$ updated only after pooling $\widehat{\mathbf{T}}_g = \phi([\mathbf{T}, \mathbf{P}_g])$, where ϕ is a small channel-wise MLP with layer normalisation.

(5) Invariant graph feature extraction. Finally, we collapse the stalk dimension through channel-wise energies: $\mathbf{g}_g = \mathrm{diag}(\widehat{\mathbf{T}}_g^\top\widehat{\mathbf{T}}_g) \in \mathbb{R}^C$. The resulting graph embedding is then fed to a standard MLP decoder for graph classification.

Full derivations, invariance arguments, stability remarks, and complexity discussion are deferred to Section A.

5 Experimental Evaluation

We evaluate HetSheaf on a diverse suite of heterogeneous *node-classification*, *link-prediction*, and *graph-classification* task benchmarks, and complement the main results with ablations and architectural analyses. We study the impact of *heterogeneous sheaf predictors*, different *restriction-map families*, *attention-based* sheaf constructions, and the proposed *universal stalk-space late-token readout* for graph-level prediction. We also assess the *model-agnosticity* of the framework by instantiating heterogeneous sheaves over multiple sheaf architectures [2, 4, 5], and we analyse the associated computational overhead and parameter efficiency. Altogether, these experiments test four claims: (i) encoding heterogeneity directly in the inferred sheaf yields strong performance across node-, edge-, and graph-level tasks; (ii) explicitly conditioning restriction maps on typed information improves over type-agnostic sheaf learning and remains competitive with specialised heterogeneous GNN baselines; (iii) HetSheaf is *model-agnostic*, enabling multiple sheaf architectures to operate on the same heterogeneous sheaf representation without ad hoc redesign; and (iv) these gains are achieved with favourable parameter efficiency, while preserving typed geometric structure through a gauge-aware graph-level readout.

Datasets, Baselines and Evaluation Protocol. We rely on a broad collection of heterogeneous benchmarks [19, 20, 38, 40, 44, 48–50, 68, 70, 74, 81] spanning *node classification*, *link prediction*, and *graph classification*. The suite covers citation graphs (e.g., DBLP, ACM), knowledge graphs (e.g., IMDB), user-item interaction networks (e.g., LastFM, MovieLens, Amazon, YouTube), link prediction graphs (e.g., PubMed), biological graphs (e.g., PROTEINS, ENZYMES), chemical compounds (e.g., MUTAG, NCI1), biomedical networks (e.g., CTD-DDA, DrugBank, NDFRT-DDA, String-PPI), and networking tasks (e.g., AS773, RouteNet Traffic). We compare against a representative set of homogeneous, heterogeneous, and sheaf-based baselines, namely GCN [42], GAT [66], GIN [71], GraphSAGE [33], R-GCN [59], HAN [68], HGT [40], and Neural Sheaf Diffusion [4]. Unless otherwise stated, we follow the standard protocols associated with each benchmark, using the HGB evaluation setup [48]. For node and graph classification, we report both *Macro-F1* and *Micro-F1*, while for link prediction we report *AUPR* and *AUROC*. Negative edges for link prediction are generated following prior work on heterogeneous relation prediction [9, 25, 79]. Model selection is performed through large-scale hyperparameter sweeps, and we report the best validation-selected configuration for each model. Detailed dataset descriptions, preprocessing choices, additional baseline comparisons from the literature, hyperparameter search spaces, hardware details, and other implementation notes are deferred to Sections B.1, B.3 and B.4.

5.1 Experimental Results

Table 2 reports the main results across tasks, where HetSheaf emerges as the most consistently strong method. It achieves the best performance on all three node-classification benchmarks, ranks first on the two largest recommendation-style link-prediction datasets, and remains highly competitive on the remaining relation-prediction tasks. On graph classification, where the challenge is no longer only to learn typed local interactions but also to compress them into a stable graph-level representation, HetSheaf again performs strongly. This pattern is particularly noteworthy because the three task families stress very different aspects of the model: local typed discrimination, relation-aware compatibility, and global aggregation under heterogeneous geometry. The supplementary results reinforce this picture. Section B.2.2 and B.2.3 extend the study to biomedical networks and networking tasks, where HetSheaf and Sheaf-NSD consistently occupy the top ranks, indicating that the benefits of sheaf-based modelling are not limited to the benchmark datasets in the main text. Section B.2 then dissects the source of these gains through a detailed analysis of heterogeneous sheaf predictors and restriction-map families, including extensions of HetSheaf to additional sheaf architectures such as PolyNSD and SheafAN. Further sensitivity analyses over the stalk dimension and the number of layers are reported in Sections B.2.7 and B.2.8, while Section B.3 situates HetSheaf within a broader landscape of heterogeneous GNNs from the recent literature. Finally, Sections A and B.4 provides a complementary systems-level view, including runtime and parameter-count comparisons, showing that the empirical gains are obtained with modest computational overhead. Taken together, the results support three conclusions. First, explicitly encoding heterogeneity in the inferred sheaf yields systematic

Table 2: *Performance on heterogeneous node classification, graph classification, and link prediction benchmarks.* Results for the heterogeneous sheaf learners and the baselines are reported. *(a)* shows the highest macro and micro F1 scores achieved across the hyperparameter sweep for each model on heterogeneous node and graph classification benchmarks. *(b)* shows the highest binary AUPR and AUROC scores achieved across the hyperparameter sweep for each model on heterogeneous link prediction benchmarks. The top three models are coloured by **First**, **Second**, and **Third**.

(a) Heterogeneous node and graph classification benchmarks.

	Node Classification						Graph Classification							
	ACM		DBLP		IMDB		MUTAG		PROTEINS		ENZYMES		NCI1	
	Macro F1	Micro F1	Macro F1	Micro F1	Macro F1	Micro F1	Macro F1	Micro F1	Macro F1	Micro F1	Macro F1	Micro F1	Macro F1	Micro F1
Homogeneous														
GCN	92.27	92.21	90.97	91.69	58.27	60.92	89.58	90.00	76.37	80.36	12.74	28.33	64.17	64.96
GAT	92.19	92.07	92.34	92.78	60.25	**64.03**	89.58	90.00	75.78	78.57	14.17	16.67	64.17	64.96
GIN	88.73	88.81	**94.84**	**95.21**	60.22	63.91	**94.88**	**95.00**	70.61	72.32	20.18	28.33	64.17	64.96
SAGE	92.20	92.12	90.13	90.74	**60.64**	63.77	89.58	90.00	74.66	78.57	8.53	18.33	63.67	65.45
Heterogeneous														
R-GCN	**93.30**	**93.25**	**94.29**	**94.68**	59.68	62.37	84.96	85.00	75.78	78.57	10.37	16.67	64.84	65.69
HAN	**93.65**	**93.58**	94.01	94.44	56.82	61.68	82.91	85.00	**79.02**	**83.04**	**46.14**	**45.00**	**77.98**	**78.10**
HGT	92.35	92.26	92.83	93.35	56.54	60.22	84.65	85.00	**77.64**	**81.25**	**48.27**	**48.33**	**79.29**	**79.32**
Sheaf-NSD	92.05	91.97	92.96	93.42	**61.96**	**65.11**	**100.00**	**100.00**	73.60	75.89	33.65	35.00	72.30	72.30
HetSheaf (ours)	**94.31**	**94.24**	**94.97**	**95.39**	**64.54**	**66.65**	**100.00**	**100.00**	**80.56**	**82.14**	**62.35**	**63.33**	**79.46**	**79.46**

(b) Heterogeneous link prediction benchmarks.

	LastFM		MovieLens		PubMed		Amazon		YouTube	
	AUPR	AUROC	AUPR	AUROC	AUPR	AUROC	AUPR	AUROC	AUPR	AUROC
Homogeneous										
GCN	97.06	96.96	**99.63**	**99.60**	96.05	92.93	92.40	89.66	89.30	83.26
GAT	63.06	51.14	97.64	97.93	96.07	93.27	88.62	86.10	66.76	58.40
GIN	**97.99**	**97.75**	99.61	**99.58**	96.14	92.87	92.10	89.62	88.64	82.97
SAGE	95.11	92.72	99.23	99.11	94.50	89.79	90.79	87.95	84.54	80.20
Heterogeneous										
R-GCN	**97.89**	**97.93**	99.39	99.37	95.03	93.15	93.37	91.42	**91.68**	**88.33**
HAN	88.26	83.32	63.58	52.21	95.78	93.62	**95.24**	**93.64**	**93.46**	**90.10**
HGT	86.02	74.55	77.67	73.99	**97.58**	**96.48**	**96.15**	**95.66**	**94.95**	**92.51**
Sheaf-NSD	97.16	96.58	**99.66**	99.57	**96.41**	**93.65**	92.63	90.04	89.32	83.36
HetSheaf (ours)	**98.51**	**98.33**	**99.67**	**99.62**	**96.36**	**93.98**	**93.76**	**91.71**	91.27	87.33

gains over both type-agnostic sheaf models and strong heterogeneous GNN baselines. Second, these improvements are not confined to node-level prediction, but extend to graph-level reasoning, where they provide empirical support for the proposed universal stalk-space late-token readout. Third, the advantage of HetSheaf is robust across markedly different dataset regimes.

6 Conclusion

In this work, we presented HetSheaf, a sheaf-agnostic framework for heterogeneous sheaf neural networks. Rather than handling heterogeneity through increasingly specialised architectures, HetSheaf models it directly at the representation level through cellular sheaves, enabling a principled treatment of type-specific local feature spaces and interactions. We introduced a family of heterogeneous sheaf predictors that explicitly condition restriction maps on node and edge type information, yielding sheaf structures tailored to heterogeneous relational data. To support graph-level learning, we further proposed SheafPool, a universal stalk-space late-token readout that is permutation-invariant and robust to local stalk gauge transformations, thereby making graph classification in sheaf networks both well defined and practically effective for the first time. Across a broad set of benchmarks, HetSheaf delivers strong, often state-of-the-art, predictive performance while remaining substantially more parameter-efficient than competing heterogeneous GNN baselines. Because the framework is compatible with existing sheaf architectures, it provides a general foundation for future progress in heterogeneous sheaf learning, and its underlying principles extend naturally beyond graphs to hypergraphs and more general topological domains.

Author Contributions

• **Luke Braithwaite:** Conceptualization, Formal analysis, Investigation, Methodology, Software, Visualization, Writing - original draft.

• **Alessio Borgi:** Conceptualization, Formal analysis, Investigation, Methodology, Software, Visualization, Writing - original draft.

• **Gabriele Onorato:** Conceptualization, Formal analysis, Investigation, Methodology, Software, Visualization, Writing - original draft.

• **Kristjan Tarantelli:** Conceptualization, Investigation, Methodology, Software.

• **Iulia Duta:** Conceptualization, Supervision, Validation, Writing - review & editing.

• **Francesco Restuccia:** Project administration, Supervision, Validation, Writing - review & editing.

• **Fabrizio Silvestri:** Project administration, Supervision, Validation, Writing - review & editing.

• **Pietro Liò:** Project administration, Supervision, Validation, Writing - review & editing.

Acknowledgments

This work was supported by the U.S. National Science Foundation Award CNS-2312875 and OAC-2530896, by the Office of Naval Research under grant N00014-23-1-2221 and by the Air Force Office of Scientific Research under grant FA9550-23-1-0261 and by DARPA under cooperative agreement D25AC00374-00.

References

[1] Uri Alon and Eran Yahav. "On the Bottleneck of Graph Neural Networks and its Practical Implications". In: *International Conference on Learning Representations*. 2021.

[2] Federico Barbero et al. "Sheaf Attention Networks". In: *NeurIPS 2022 Workshop on Symmetry and Geometry in Neural Representations*. Nov. 2022.

[3] Federico Barbero et al. "Sheaf Neural Networks with Connection Laplacians". In: *Proceedings of Topological, Algebraic, and Geometric Learning Workshops 2022*. PMLR, Nov. 2022, pp. 28–36.

[4] Cristian Bodnar et al. "Neural Sheaf Diffusion: A Topological Perspective on Heterophily and Oversmoothing in GNNs". In: *NeurIPS 2022*. May 2022.

[5] Alessio Borgi, Fabrizio Silvestri, and Pietro Liò. *Polynomial Neural Sheaf Diffusion: A Spectral Filtering Approach on Cellular Sheaves*. 2026. arXiv: 2512.00242 [cs.LG].

[6] Joan Bruna et al. "Spectral networks and locally connected networks on graphs". In: *arXiv preprint arXiv:1312.6203* (2013).

[7] Chen Cai and Yusu Wang. "A Note on Over-Smoothing for Graph Neural Networks". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 108. Proceedings of Machine Learning Research. Vienna, Austria: PMLR, 2020, pp. 1–9.

[8] Ferran Hernandez Caralt et al. "Joint Diffusion Processes as an Inductive Bias in Sheaf Neural Networks". In: *ICML 2024 Workshop on Geometry-grounded Representation Learning and Generative Modeling*. 2024.

[9] Yukuo Cen et al. "Representation Learning for Attributed Multiplex Heterogeneous Network". In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. KDD '19. New York, NY, USA: Association for Computing Machinery, July 2019, pp. 1358–1368. ISBN: 978-1-4503-6201-6.

[10] Guanzi Chen et al. *Preventing Over-Smoothing for Hypergraph Neural Networks*. arXiv:2203.17159. Nov. 2022. arXiv: 2203.17159 [cs].

[11] Gabriele Corso et al. "Principal Neighbourhood Aggregation for Graph Nets". In: *Advances in Neural Information Processing Systems*. 2020.

[12] Justin Michael Curry. "Sheaves, Cosheaves and Applications". PhD thesis. University of Pennsylvania, 2014.

[13] Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. "Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering". In: *Advances in Neural Information Processing Systems*. Vol. 29. Curran Associates, Inc., 2016.

[14] Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. "Convolutional neural networks on graphs with fast localized spectral filtering". In: *Advances in neural information processing systems* 29 (2016).

[15] Yuxiao Dong, Nitesh V. Chawla, and Ananthram Swami. "metapath2vec: Scalable Representation Learning for Heterogeneous Networks". In: *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '17. New York, NY, USA: Association for Computing Machinery, Aug. 2017, pp. 135–144. ISBN: 978-1-4503-4887-4.

[16] Alexey Dosovitskiy et al. "An image is worth 16x16 words: Transformers for image recognition at scale". In: *arXiv preprint arXiv:2010.11929* (2020).

[17] Iulia Duta et al. "Sheaf Hypergraph Networks". In: *Thirty-seventh Conference on Neural Information Processing Systems*. NeurIPS 2023, Nov. 2023.

[18] Wenqi Fan et al. "Graph Neural Networks for Social Recommendation". In: *The World Wide Web Conference*. San Francisco CA USA: ACM, May 2019, pp. 417–426. ISBN: 978-1-4503-6674-8.

[19] Miquel Ferriol-Galmés et al. "RouteNet-Erlang: A Graph Neural Network for Network Performance Evaluation". In: *IEEE INFOCOM 2022 - IEEE Conference on Computer Communications*. London, United Kingdom: IEEE Press, 2022, pp. 2018–2027. DOI: 10.1109/INFOCOM48880.2022.9796944.

[20] Miquel Ferriol-Galmés et al. "RouteNet-Fermi: Network Modeling With Graph Neural Networks". In: *IEEE/ACM Trans. Netw.* 31.6 (May 2023), pp. 3080–3095. ISSN: 1063-6692. DOI: 10.1109/TNET.2023.3269983.

[21] Matthias Fey and Jan E. Lenssen. "Fast Graph Representation Learning with PyTorch Geometric". In: *ICLR Workshop on Representation Learning on Graphs and Manifolds*. 2019.

[22] Matthias Fey et al. "PyG 2.0: Scalable Learning on Real World Graphs". In: *Temporal Graph Learning Workshop @ KDD*. 2025.

[23] Stefano Fiorini et al. "Sheaves Reloaded: A Direction Awakening". In: *The Fourteenth International Conference on Learning Representations*. 2026.

[24] Alex Fout et al. "Protein Interface Prediction using Graph Convolutional Networks". In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., 2017.

[25] Xinyu Fu et al. "MAGNN: Metapath Aggregated Graph Neural Network for Heterogeneous Graph Embedding". In: *Proceedings of The Web Conference 2020*. Apr. 2020, pp. 2331–2341.

[26] Justin Gilmer et al. "Neural message passing for Quantum chemistry". In: *Proceedings of the 34th International Conference on Machine Learning - Volume 70*. ICML'17. Sydney, NSW, Australia: JMLR.org, 2017, pp. 1263–1272.

[27] Justin Gilmer et al. "Neural message passing for quantum chemistry". In: *International conference on machine learning*. Pmlr. 2017, pp. 1263–1272.

[28] Christoph Goller and Andreas Kuchler. "Learning task-dependent distributed representations by backpropagation through structure". In: *Proceedings of international conference on neural networks (ICNN'96)*. Vol. 1. IEEE. 1996, pp. 347–352.

[29] Marco Gori, Gabriele Monfardini, and Franco Scarselli. "A new model for learning in graph domains". In: *Proceedings. 2005 IEEE international joint conference on neural networks, 2005*. Vol. 2. IEEE. 2005, pp. 729–734.

[30] Daniele Grattarola et al. "Understanding Pooling in Graph Neural Networks". In: *IEEE Transactions on Neural Networks and Learning Systems* 35.2 (2024), pp. 2708–2718. DOI: 10.1109/TNNLS.2022.3190922.

[31] Shengnan Guo et al. “Attention Based Spatial-Temporal Graph Convolutional Networks for Traffic Flow Forecasting”. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 33.01 (July 2019), pp. 922–929. ISSN: 2374-3468.

[32] Mustafa Hajij et al. “Copresheaf Topological Neural Networks: A Generalized Deep Learning Framework”. In: *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. 2025.

[33] Will Hamilton, Zhitao Ying, and Jure Leskovec. “Inductive Representation Learning on Large Graphs”. In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., 2017.

[34] William L. Hamilton. *Graph Representation Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Cham: Springer International Publishing, 2020. ISBN: 978-3-031-00460-5.

[35] Jakob Hansen and Thomas Gebhart. “Sheaf Neural Networks”. In: *NeurIPS Workshop on TDA & Beyond*. 2020.

[36] Jakob Hansen and Robert Ghrist. “Learning Sheaf Laplacians from Smooth Signals”. In: *ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2019, pp. 5446–5450. DOI: 10.1109/ICASSP.2019.8683709.

[37] Yu He, Cristian Bodnar, and Pietro Lio. “Sheaf-based Positional Encodings for Graph Neural Networks”. In: *NeurIPS 2023 Workshop on Symmetry and Geometry in Neural Representations*. Nov. 2023.

[38] Junwei Hu et al. “Heterogeneous graph neural networks for link prediction in biomedical networks”. In: *Bioinformatics Advances* 5.1 (Aug. 2025), vbaf187. ISSN: 2635-0041. DOI: 10.1093/bioadv/vbaf187. eprint: https://academic.oup.com/bioinformaticsadvances/article-pdf/5/1/vbaf187/63961115/vbaf187.pdf.

[39] Ziniu Hu et al. “GPT-GNN: Generative Pre-Training of Graph Neural Networks”. In: *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. KDD ’20. New York, NY, USA: Association for Computing Machinery, Aug. 2020, pp. 1857–1867. ISBN: 978-1-4503-7998-4.

[40] Ziniu Hu et al. “Heterogeneous Graph Transformer”. In: *Proceedings of The Web Conference 2020*. WWW ’20. Taipei, Taiwan: Association for Computing Machinery, 2020, pp. 2704–2710. ISBN: 9781450370233. DOI: 10.1145/3366423.3380027.

[41] Chenqing Hua, Guillaume Rabusseau, and Jian Tang. “High-order pooling for graph neural networks with tensor decomposition”. In: *NeurIPS 2022*. NIPS ’22. Red Hook, NY, USA: Curran Associates Inc., 2022, pp. 6021–6033. ISBN: 978-1-71387-108-8.

[42] Thomas N. Kipf and Max Welling. “Semi-Supervised Classification with Graph Convolutional Networks”. In: *ICLR 2016*. Nov. 2016.

[43] Junhyun Lee, Inyeop Lee, and Jaewoo Kang. “Self-attention graph pooling”. In: *International conference on machine learning*. pmlr. 2019, pp. 3734–3743.

[44] Jure Leskovec, Jon Kleinberg, and Christos Faloutsos. “Graphs over time: densification laws, shrinking diameters and possible explanations”. In: *Proceedings of the Eleventh ACM SIGKDD International Conference on Knowledge Discovery in Data Mining*. KDD ’05. Chicago, Illinois, USA: Association for Computing Machinery, 2005, pp. 177–187. ISBN: 159593135X. DOI: 10.1145/1081870.1081893.

[45] Chang Li and Dan Goldwasser. “Encoding Social Information with Graph Convolutional Networks forPolitical Perspective Detection in News Media”. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Ed. by Anna Korhonen, David Traum, and Lluís Màrquez. Florence, Italy: Association for Computational Linguistics, July 2019, pp. 2594–2604.

[46] Yujia Li et al. “Gated Graph Sequence Neural Networks”. In: *Proceedings of ICLR’16*. 2016.

[47] Weihong Lin et al. “Heterogeneous Graph Neural Network with Adaptive Relation Reconstruction”. In: *Neural Networks* 187 (2025), p. 107313. ISSN: 0893-6080. DOI: https://doi.org/10.1016/j.neunet.2025.107313.

[48] Qingsong Lv et al. "Are we really making much progress? Revisiting, benchmarking and refining heterogeneous graph neural networks". In: *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. KDD '21. New York, NY, USA: Association for Computing Machinery, Aug. 2021, pp. 1150–1160. ISBN: 978-1-4503-8332-5.

[49] Julian McAuley and Jure Leskovec. "Hidden Factors and Hidden Topics: Understanding Rating Dimensions with Review Text". In: *Proceedings of the 7th ACM Conference on Recommender Systems*. RecSys '13. ACM, 2013, pp. 165–172.

[50] Christopher Morris et al. "TUDataset: A collection of benchmark datasets for learning with graphs". In: *ICML 2020 Workshop on Graph Representation Learning and Beyond (GRL+ 2020)*. 2020. arXiv: 2007.08663.

[51] Emanuele Mule et al. "Directional Sheaf Hypergraph Networks: Unifying Learning on Directed and Undirected Hypergraphs". In: *The Fourteenth International Conference on Learning Representations*. 2026.

[52] Bao Nguyen et al. *Sheaf HyperNetworks for Personalized Federated Learning*. 2024.

[53] Anton Obukhov. *Efficient Householder Transformation in PyTorch*. Version v1.0.1. Version: 1.0.1, DOI: 10.5281/zenodo.5068733. 2021. DOI: 10.5281/zenodo.5068733.

[54] Gabriele Onorato. *Bayesian Optimization for Hyperparameters Tuning in Neural Networks*. 2024. arXiv: 2410.21886 [cs.LG].

[55] Antonio Purificato et al. "Sheaf4Rec: Sheaf Neural Networks for Graph-based Recommender Systems". In: *ACM Trans. Recomm. Syst.* 4.2 (Nov. 2025). DOI: 10.1145/3742898.

[56] Jiezhong Qiu et al. "DeepInf: Social Influence Prediction with Deep Learning". In: *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. KDD '18. New York, NY, USA: Association for Computing Machinery, July 2018, pp. 2110–2119. ISBN: 978-1-4503-5552-0.

[57] André Ribeiro et al. "Cooperative Sheaf Neural Networks". In: *International Conference on Learning Representations*. 2026.

[58] Franco Scarselli et al. "The graph neural network model". In: *IEEE transactions on neural networks* 20.1 (2008), pp. 61–80.

[59] Michael Schlichtkrull et al. "Modeling Relational Data with Graph Convolutional Networks". In: *The Semantic Web*. Ed. by Aldo Gangemi et al. Cham: Springer International Publishing, 2018, pp. 593–607. ISBN: 978-3-319-93417-4.

[60] Allen D. Shepard. "A Cellular Description of the Derived Category of a Stratified Space". PhD thesis. Brown University, 1985.

[61] Jasper Snoek, Hugo Larochelle, and Ryan P Adams. "Practical Bayesian Optimization of Machine Learning Algorithms". In: *Advances in Neural Information Processing Systems*. Ed. by F. Pereira et al. Vol. 25. Curran Associates, Inc., 2012.

[62] Yizhou Sun and Jiawei Han. *Mining Heterogeneous Information Networks: Principles and Methodologies*. Synthesis Lectures on Data Mining and Knowledge Discovery. Cham: Springer International Publishing, 2012. ISBN: 978-3-031-01902-9.

[63] Yizhou Sun et al. "PathSim: meta path-based top-K similarity search in heterogeneous information networks". In: *Proc. VLDB Endow.* 4.11 (Aug. 2011), pp. 992–1003. ISSN: 2150-8097.

[64] Damian Szklarczyk et al. "The STRING database in 2021: customizable protein–protein networks, and functional characterization of user-uploaded gene/measurement sets". In: *Nucleic Acids Research* 49.D1 (2021), pp. D605–D612.

[65] Petar Velickovic et al. "Graph attention networks". In: *stat* 1050.20 (2017), pp. 10–48550.

[66] Petar Veličković et al. "Deep Graph Infomax". In: *International Conference on Learning Representations*. Sept. 2018.

[67] Petar Veličković et al. "Graph Attention Networks". In: *ICLR 2018*. Feb. 2018.

[68] Xiao Wang et al. "Heterogeneous Graph Attention Network". In: *The World Wide Web Conference*. WWW '19. New York, NY, USA: Association for Computing Machinery, May 2019, pp. 2022–2032. ISBN: 978-1-4503-6674-8.

[69] David S. Wishart et al. “DrugBank 5.0: a major update to the DrugBank database for 2018”. In: *Nucleic Acids Research* 46.D1 (2018), pp. D1074–D1082.

[70] Wang Xiaoqi et al. “BioERP: biomedical heterogeneous network-based self-supervised representation learning approach for entity relationship predictions”. In: *Bioinformatics* (2021). DOI: 10.1093/bioinformatics/btab565.

[71] Keyulu Xu et al. “How Powerful are Graph Neural Networks?” In: *International Conference on Learning Representations*. Sept. 2018.

[72] Keyulu Xu et al. “Representation Learning on Graphs with Jumping Knowledge Networks”. In: *Proceedings of the 35th International Conference on Machine Learning*. PMLR, July 2018, pp. 5453–5462.

[73] Bishan Yang et al. “Embedding Entities and Relations for Learning and Inference in Knowledge Bases”. In: *Proceedings of the International Conference on Learning Representations (ICLR) 2015*. 2015.

[74] Carl Yang et al. “Heterogeneous Network Representation Learning: A Unified Framework With Survey and Benchmark”. In: *IEEE Trans. on Knowl. and Data Eng.* 34.10 (Oct. 2022), pp. 4854–4873. ISSN: 1041-4347.

[75] Carl Yang et al. “MultiSage: Empowering GCN with Contextualized Multi-Embeddings on Web-Scale Multipartite Networks”. In: *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. KDD ’20. New York, NY, USA: Association for Computing Machinery, Aug. 2020, pp. 2434–2443. ISBN: 978-1-4503-7998-4.

[76] Gang Yang, Xiaofeng Zhang, and Yueping Li. “Session-Based Recommendation with Graph Neural Networks for Repeat Consumption”. In: *Proceedings of the 2020 9th International Conference on Computing and Pattern Recognition*. ICCPR ’20. New York, NY, USA: Association for Computing Machinery, Jan. 2021, pp. 519–524. ISBN: 978-1-4503-8783-5.

[77] Seongjun Yun et al. “Graph Transformer Networks”. In: *Advances in Neural Information Processing Systems*. Vol. 32. Curran Associates, Inc., 2019.

[78] Olga Zaghen et al. “Sheaf Diffusion Goes Nonlinear: Enhancing GNNs with Adaptive Sheaf Laplacians”. In: *ICML 2024 Workshop on Geometry-grounded Representation Learning and Generative Modeling*. 2024.

[79] Chuxu Zhang et al. “Heterogeneous Graph Neural Network”. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. KDD ’19. New York, NY, USA: Association for Computing Machinery, July 2019, pp. 793–803. ISBN: 978-1-4503-6201-6.

[80] Tianyu Zhao et al. “Space4HGNN: A Novel, Modularized and Reproducible Platform to Evaluate Heterogeneous Graph Neural Network”. In: *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. SIGIR ’22. New York, NY, USA: Association for Computing Machinery, July 2022, pp. 2776–2789. ISBN: 978-1-4503-8732-3.

[81] Ziang Zhou et al. “SlotGAT: Slot-based Message Passing for Heterogeneous Graphs”. In: *Proceedings of the 40th International Conference on Machine Learning*. PMLR, July 2023, pp. 42644–42657.

[82] Guanghui Zhu et al. “HAGNN: Hybrid Aggregation for Heterogeneous Graph Neural Networks”. In: *IEEE Transactions on Neural Networks and Learning Systems* 36.8 (2025), pp. 14536–14550. DOI: 10.1109/TNNLS.2024.3519427.

[83] Shichao Zhu et al. “Relation Structure-Aware Heterogeneous Graph Neural Network”. In: *2019 IEEE International Conference on Data Mining (ICDM)*. Nov. 2019, pp. 1534–1539.

Heterogeneous Sheaf Neural Networks

Supplementary Material

Contents

A Universal Stalk-Space Late-token Readout: Formalization and Properties 18

A.1 Problem Setting and Gauge Quotient Viewpoint 18

A.2 Flatten-and-Pool fails for Sheaf Neural Networks 19

A.3 Design Principle: Canonicalize and Pool 19

A.4 The Proposed Universal Stalk-Space Late-token Readout 20

A.4.1 Whitening Removes the $GL(d)$ Gauge up to an Orthogonal Factor . . 21

A.4.2 Orthogonal Procrustes Alignment 22

A.4.3 Exact Invariance of the Canonicalized Representative 22

A.4.4 Permutation Invariance and Gauge Invariance of the Readout 23

A.5 Classification Token in Vision Transformers Comparison 23

A.6 Injective Universal Stalk-Space Readout on the Canonicalized Descriptor Space 24

A.6.1 Injective Readout on Gauge Orbits 25

A.7 Graph-level Readout Overhead 27

B Additional Experimental, Ablation, and Implementation Details 29

B.1 Experimental Protocol, Hyperparameters, and Datasets 29

B.1.1 Model Hyperparameters 29

B.2 Additional Results and Model Analysis 29

B.2.1 Benchmark Datasets 30

B.2.2 Biomedical Datasets 31

B.2.3 Networking Datasets 32

B.2.4 Sheaf Predictor Ablation Study 33

B.2.5 Effect of Restriction Map Type 37

B.2.6 Attention-based Restriction Map Variants 37

B.2.7 Stalk Dimension Ablation Study 37

B.2.8 Layer Depth Ablation Study 39

B.3 External Comparisons with Additional Heterogeneous GNNs 41

B.4 Implementation Details 43

B.4.1 Hardware and Software Setup 44

B.4.2 Downstream Decoders and Loss Functions 44

B.4.3 Restriction Map Parameterisations 45

B.4.4 Full Description of Heterogeneous Sheaf Predictors 45

B.4.5 HetSheaf Computational Complexity and Overhead 46

B.4.6 Heterogeneous Sheaf Attention Networks 48

B.4.7 Heterogeneous Polynomial Neural Sheaf Diffusion 48

A Universal Stalk-Space Late-token Readout: Formalization and Properties

This appendix provides a formal treatment of our graph-level readout for sheaf neural networks. We proceed in four steps. First, we formalize graph classification in the presence of local stalk gauges and explain why standard flatten-and-pool readouts are not well-defined in this setting. Second, we derive the proposed *Universal Stalk-Space Late-token Readout* and prove its permutation invariance and gauge compatibility under natural identifiability assumptions. Third, we compare the proposed *late, receive-only token* to the *classification token* used in Vision Transformers (ViTs), clarifying why the latter cannot be imported naively into sheaf encoders. Finally, following the multiset perspective used in expressive GNN theory, we state and prove an *injective universal readout theorem* on the canonicalized stalk descriptor space. This theorem formalizes that injective graph-level pooling is possible in principle after gauge canonicalization and our practical late-token energy head should be viewed as a structured, low-overhead instantiation of this broader family.

A.1 Problem Setting and Gauge Quotient Viewpoint

Let $G_g = (V_g, E_g)$ be a graph and let a sheaf neural encoder produce node embeddings:

$$\mathbf{H}_i \in \mathbb{R}^{d\times C} \quad \text{for} \quad i \in V_g \tag{6}$$

where d is the stalk dimension and C is the channel dimension. For graph classification, the goal is to construct a readout that summarizes the graph into a single graph representation:

$$\mathcal{R} : \{\mathbf{H}_i\}_{i\in V_g} \mapsto \mathbf{g}_g \in \mathbb{R}^F \tag{7}$$

In ordinary GNNs, node embeddings live in a common ambient space (shared latent space) $\mathbb{R}^C$, so a graph representation can be obtained by applying any permutation-invariant pooling operator to the multiset $\{\mathbf{h}_i\}_{i\in V_g}$. In a sheaf model, by contrast, the node state at i belongs to the local stalk $\mathcal{F}(i)$, and only becomes a matrix $\mathbf{H}_i \in \mathbb{R}^{d\times C}$ after choosing a basis of $\mathcal{F}(i)$. Since there is in general no canonical basis shared across nodes, the coordinates $\mathbf{H}_i$ are not intrinsic: if the basis of $\mathcal{F}(i)$ is changed by $\mathbf{G}_i \in \mathcal{G}_i \subseteq GL(d)$, then the same underlying stalk signal is represented by:

$$\mathbf{H}_i \mapsto \mathbf{H}'_i := \mathbf{G}_i\mathbf{H}_i \quad \text{with} \quad \mathbf{G}_i \in \mathcal{G} \tag{8}$$

where $\mathcal{G}$ depends on the restriction-map family: a diagonal subgroup of $GL(d)$ in the diagonal case, $O(d)$ or a metric-preserving subgroup in the orthogonal/bundle case, and $GL(d)$ in the general case. We refer to this action as a *local gauge action* on the stalk.

Definition 1 (Gauge Orbit)**.** *For a node embedding $\mathbf{H} \in \mathbb{R}^{d\times C}$, define its gauge orbit as:*

$$[\mathbf{H}]_{\mathcal{G}} := \{\mathbf{GH} : \mathbf{G} \in \mathcal{G}\} \tag{9}$$

A graph-level readout is gauge-invariant *if it depends only on the multiset of node-wise gauge orbits.*

Thus, a principled sheaf readout must satisfy two requirements simultaneously:

1. **Permutation invariance over nodes:** reordering the nodes of a graph must not change the output.
2. **Gauge compatibility / invariance:** equivalent local coordinate realizations of the same stalk signal must produce the same graph descriptor.

Definition 2 (Gauge-compatible graph readout)**.** *Let $\mathcal{G} := \prod_{i\in V_g} \mathcal{G}_i$ be the product of the admissible node-wise gauge groups, where $\mathcal{G}_i$ is determined by the restriction-map family (e.g. diagonal, orthogonal, or general invertible transforms). A graph-level readout*

$$\mathcal{R} : \{\mathbf{H}_i\}_{i\in V_g} \mapsto \mathbf{g}_g \tag{10}$$

is said to be gauge-compatible *if*

$$\mathcal{R}(\{\mathbf{G}_i\mathbf{H}_i\}_{i\in V_g}) = \mathcal{R}(\{\mathbf{H}_i\}_{i\in V_g}) \qquad \forall (\mathbf{G}_i)_{i\in V_g} \in \mathcal{G}, \tag{11}$$

and permutation-invariant *if its value is unchanged under any reordering of the node index set V_g.*

A.2 Flatten-and-Pool fails for Sheaf Neural Networks

In sheaf neural networks, since each node representation $\mathbf{H}_i \in \mathbb{R}^{d\times C}$ lives in a *local stalk space*, two tensors that represent the same underlying local signal may differ by a transformation of the form $\mathbf{H}_i \mapsto \mathbf{G}_i\mathbf{H}_i$, with $\mathbf{G}_i$ taken from the relevant gauge group. Any pooling rule that acts directly on the raw coordinates of $\mathbf{H}_i$ therefore risks depending on arbitrary local frame choices rather than on the intrinsic sheaf signal itself. This issue is particularly important after flattening. Indeed, vectorizing $\mathbf{H}_i$ identifies the stalk and channel axes with a single ambient Euclidean coordinate system, thereby implicitly assuming that all nodes are already expressed in a common stalk basis. But this assumption is precisely what fails in the sheaf setting: different nodes may carry equivalent information in different local coordinates. As a result, applying a standard graph readout to $\mathrm{vec}(\mathbf{H}_i) \in \mathbb{R}^{dC}$ mixes together quantities that are not canonically comparable across nodes.

From a *geometric viewpoint*, flatten-and-pool collapses the quotient structure induced by the local gauge action. A correct sheaf readout should should first construct canonical representatives of these classes before aggregation and naive flattening does not do that. Therefore, even if the encoder has learned meaningful stalk-structured node states, a basis-dependent readout may destroy this structure at the final stage and produce graph embeddings that vary under purely representational changes.

Proposition 1 (Flatten-and-pool is not gauge-invariant)**.** *Let* $\mathrm{vec}(\cdot)$ *denote vectorization. Consider any readout that pools flattened node embeddings through a coordinate-wise operator such as sum, mean, max, or standard attention on* $\mathrm{vec}(\mathbf{H}_i) \in \mathbb{R}^{d\times C}$*. Then, this readout is not invariant under node-wise gauge transformations* $\mathbf{H}_i \mapsto \mathbf{G}_i\mathbf{H}_i$*.*

Proof. Using the standard Kronecker identity, we have that:

$$\mathrm{vec}(\mathbf{GH}) = (\mathbf{I}_C \otimes \mathbf{G})\,\mathrm{vec}(\mathbf{H}) \tag{12}$$

Hence, under a node-wise gauge transform we obtain:

$$\mathrm{vec}(\mathbf{H}_i') = (\mathbf{I}_C \otimes \mathbf{G}_i)\,\mathrm{vec}(\mathbf{H}_i) \tag{13}$$

If we consider, for instance, sum pooling, we get:

$$\mathbf{g}_{\mathrm{sum}} := \sum_{i\in V_g} \mathrm{vec}(\mathbf{H}_i) \tag{14}$$

After the reparameterization, we then have:

$$\mathbf{g}'_{\mathrm{sum}} = \sum_{i\in V_g} (\mathbf{I}_C \otimes \mathbf{G}_i)\,\mathrm{vec}(\mathbf{H}_i) \tag{15}$$

which is generically different from $\mathbf{g}_{\mathrm{sum}}$, unless all $\mathbf{G}_i = \mathbf{I}_d$ or a non-generic cancellation occurs. Mean pooling is just a rescaled version of sum pooling and fails for the same reason. Max pooling is not equivariant under general invertible linear maps, and thus coordinate-wise maxima are also basis-dependent. Likewise, attention on flattened features changes because both attention logits and aggregated values depend on the chosen coordinates unless the scoring and value maps are themselves gauge-invariant. Therefore, direct flatten-and-pool is not a principled sheaf readout. □

The proposition shows that one must either pool purely gauge-invariant node descriptors or first canonicalize local stalk frames before pooling in stalk space. The former is robust but collapses the stalk structure too early, while the latter is the route we adopt.

A.3 Design Principle: Canonicalize and Pool

A valid readout for sheaf graph classification should satisfy the following desiderata: it must support graph-level prediction, it must remain meaningful for nontrivial stalk dimension

$d > 1$, it must remain valid across diagonal, orthogonal, and general restriction-map families and it should preserve stalk-structured information as long as possible before the final graph compression.

We have seen that the natural alternatives are indeed insufficient. First, standard pooling on flattened embeddings is computationally simple but fails Proposition 1. Second, direct stalk-space pooling with energy-based weights is valid only if the residual gauge is orthogonal, and therefore does not extend to the general $GL(d)$ case. Third, invariant-only pooling, for example pooling per-node channel energies, is universal but collapses the stalk dimension before cross-node interactions in stalk space can be exploited.

Our solution is a *two-stage canonicalization followed by stalk-space aggregation*. We first use per-node whitening to remove the non-orthogonal part of the local gauge. We then use anchor-based orthogonal Procrustes alignment to express all nodes in a common stalk frame. Only after this canonicalization, we pool in $\mathbb{R}^{d\times C}$. A late, receive-only token is then used to add graph-level expressive capacity without altering the encoder dynamics. The proposed *Universal Stalk-Space Late-token Readout* has already been described in Section 4, but now we are going to see it again together with mathematical derivations and proofs.

A.4 The Proposed Universal Stalk-Space Late-token Readout

Given node embeddings $\mathbf{H}_i \in \mathbb{R}^{d\times C}$, we define the readout:

$$\mathcal{R} : \{\mathbf{H}_i\}_{i\in V_g} \longmapsto \mathbf{g}_g \in \mathbb{R}^C \tag{16}$$

through the five-stage pipeline:

$$\mathbf{H}_i \xrightarrow{\text{whiten}} \mathbf{Z}_i \xrightarrow{\text{align}} \widetilde{\mathbf{Z}}_i \xrightarrow{\text{attn pool}} \mathbf{P}_g \xrightarrow{\text{late token}} \widehat{\mathbf{T}}_g \xrightarrow{\text{energy}} \mathbf{g}_g \tag{17}$$

Step 1: Per-node whitening. For each node, we define the regularized stalk-side Gram matrix as:

$$\mathbf{S}_i = \mathbf{H}_i\mathbf{H}_i^\top + \varepsilon\mathbf{I}_d \in \mathbb{R}^{d\times d} \tag{18}$$

where the term $\varepsilon\mathbf{I}_d$ acts as a Tikhonov-style regularizer that yields a slightly smoothed, but well-defined, whitening transform. Its purpose is to ensure that Eq. 18 remains positive definite, even when $\mathbf{H}_i\mathbf{H}_i^\top$ is singular or ill-conditioned. In particular, if some directions in stalk space have very small variance, the inverse square root $\mathbf{S}_i^{-1/2}$ may otherwise be unstable or undefined. By shifting all eigenvalues of $\mathbf{H}_i\mathbf{H}_i^\top$ by ε, this term stabilizes the whitening operation, prevents the amplification of near-zero directions, and makes the computation numerically robust. Then, the whitened representation is:

$$\mathbf{Z}_i = \mathbf{S}_i^{-1/2}\mathbf{H}_i \in \mathbb{R}^{d\times C} \tag{19}$$

When $\varepsilon = 0$ and $\mathbf{H}_i\mathbf{H}_i^\top$ is invertible, we have $\mathbf{Z}_i\mathbf{Z}_i^\top = \mathbf{I}_d$. Thus whitening removes anisotropic scaling and shearing in stalk space and reduces the ambiguity to an orthogonal one.

Step 2: Anchor-based Procrustes alignment. After whitening, different nodes may still differ by a residual orthogonal transform. We fix a shared learnable anchor $\mathbf{A} \in \mathbb{R}^{d\times C}$ and solve:

$$\mathbf{R}_i^\star = \arg\min_{\mathbf{R}\in O(d)} \|\mathbf{R}^\top\mathbf{Z}_i - \mathbf{A}\|_F^2 \tag{20}$$

The aligned representation is then done by:

$$\widetilde{\mathbf{Z}}_i = \mathbf{R}_i^{\star\top}\mathbf{Z}_i \tag{21}$$

Step 3: Invariant attention weights. We define a per-node invariant descriptor using channel-wise energies of the whitened features:

$$\mathbf{u}_i = \operatorname{diag}(\mathbf{Z}_i^\top\mathbf{Z}_i) \in \mathbb{R}^C \tag{22}$$

A scorer $s_\theta : \mathbb{R}^C \to \mathbb{R}$ gives graph-wise attention weights:

$$\alpha_i = \frac{\exp(s_\theta(\mathbf{u}_i))}{\sum_{j\in V_g}\exp(s_\theta(\mathbf{u}_j))} \tag{23}$$

Step 4: Stalk-space pooling and a late receive-only token. We pool directly in stalk space:

$$\mathbf{P}_g = \sum_{i \in V_g} \alpha_i \widetilde{\mathbf{Z}}_i \in \mathbb{R}^{d \times C} \tag{24}$$

A learnable late token $\mathbf{T} \in \mathbb{R}^{d \times C}$ is then updated only *after* encoder message passing:

$$\widehat{\mathbf{T}}_g = \phi([\mathbf{T}, \mathbf{P}_g]) \in \mathbb{R}^{d \times C} \tag{25}$$

where ϕ is a small channel-wise MLP with layer normalization.

Step 5: Final invariant graph descriptor. We finally collapse the stalk dimension by channel-wise energies:

$$\mathbf{g}_g = \mathrm{diag}(\widehat{\mathbf{T}}_g^\top \widehat{\mathbf{T}}_g) \in \mathbb{R}^C, \qquad [\mathbf{g}_g]_c = \sum_{a=1}^{d} (\widehat{\mathbf{T}}_g)_{a,c}^2 \tag{26}$$

A.4.1 Whitening Removes the $GL(d)$ Gauge up to an Orthogonal Factor

We first state the exact whitening equivariance result in the ideal full-row-rank setting, and then discuss the practically relevant regularized case.

Theorem 1 (Exact whitening canonicalizes up to $O(d)$ in the full-rank case)**.** *Fix a node and let $\mathbf{H} \in \mathbb{R}^{d \times C}$. Assume that $\mathbf{H}$ has full row rank, so that:*

$$\mathbf{S} := \mathbf{H}\mathbf{H}^\top \succ 0 \tag{27}$$

Let's define:

$$\mathbf{Z} = \mathbf{S}^{-1/2}\mathbf{H} \tag{28}$$

Let $\mathbf{H}' = \mathbf{G}\mathbf{H}$ with $\mathbf{G} \in GL(d)$, and define:

$$\mathbf{S}' = \mathbf{H}'\mathbf{H}'^\top = \mathbf{G}\mathbf{S}\mathbf{G}^\top, \qquad \mathbf{Z}' = (\mathbf{S}')^{-1/2}\mathbf{H}' \tag{29}$$

Then there exists $\mathbf{Q} \in O(d)$ such that:

$$\mathbf{Z}' = \mathbf{Q}\mathbf{Z} \tag{30}$$

Proof. Let $\mathbf{M} := \mathbf{G}\mathbf{S}^{1/2}$. Then, we have that:

$$\mathbf{S}' = \mathbf{G}\mathbf{S}\mathbf{G}^\top = \mathbf{M}\mathbf{M}^\top \tag{31}$$

By the polar decomposition, there exist $\mathbf{Q} \in O(d)$ and $\mathbf{P} \succ 0$ such that:

$$\mathbf{M} = \mathbf{Q}\mathbf{P} \tag{32}$$

Hence:

$$\mathbf{S}' = \mathbf{Q}\mathbf{P}^2\mathbf{Q}^\top, \qquad (\mathbf{S}')^{-1/2} = \mathbf{Q}\mathbf{P}^{-1}\mathbf{Q}^\top \tag{33}$$

Therefore, we get:

$$\begin{aligned} \mathbf{Z}' &= (\mathbf{S}')^{-1/2}\mathbf{H}' = (\mathbf{S}')^{-1/2}\mathbf{G}\mathbf{H} = (\mathbf{S}')^{-1/2}\mathbf{G}\mathbf{S}^{1/2}\mathbf{S}^{-1/2}\mathbf{H} \\ &= (\mathbf{S}')^{-1/2}\mathbf{M}\mathbf{Z} = \mathbf{Q}\mathbf{P}^{-1}\mathbf{Q}^\top\mathbf{Q}\mathbf{P}\mathbf{Z} = \mathbf{Q}\mathbf{Z} \end{aligned} \tag{34}$$

Thus whitening removes the non-orthogonal part of the $GL(d)$ gauge, leaving only a residual orthogonal ambiguity. □

Remark 1 (On the full-rank assumption)**.** *The matrix $\mathbf{H}\mathbf{H}^\top$ is always positive semidefinite, but it is positive definite only when $\mathbf{H}$ has full row rank. This need not hold automatically in learned sheaf embeddings, since channel mixing and feature collapse may produce rank-deficient or ill-conditioned stalk-side Gram matrices.*

Remark 2 (Regularized case used in practice)**.** *In practice, we use the regularized matrix:*

$$\mathbf{S}_\varepsilon := \mathbf{H}\mathbf{H}^\top + \varepsilon\mathbf{I}_d, \qquad \varepsilon > 0 \tag{35}$$

which is always symmetric positive definite, regardless of the rank of $\mathbf{H}$*. The corresponding whitening transform:*

$$\mathbf{Z}_\varepsilon := \mathbf{S}_\varepsilon^{-1/2}\mathbf{H} \tag{36}$$

is therefore always well defined and numerically stable. However, because:

$$\mathbf{H}'\mathbf{H}'^\top + \varepsilon \mathbf{I}_d \neq \mathbf{G}(\mathbf{H}\mathbf{H}^\top + \varepsilon \mathbf{I}_d)\mathbf{G}^\top \tag{37}$$

in general, the exact identity $\mathbf{Z}'_\varepsilon = \mathbf{Q}\mathbf{Z}_\varepsilon$ *no longer holds. The regularized whitening should therefore be viewed as a stable approximation to the ideal canonicalization result of Theorem 1.*

A.4.2 Orthogonal Procrustes Alignment

We next derive the alignment step.

Proposition 2 (Closed-form Procrustes solution)**.** *Given* $\mathbf{Z} \in \mathbb{R}^{d\times C}$ *and* $\mathbf{A} \in \mathbb{R}^{d\times C}$*, consider:*

$$\mathbf{R}^\star = \arg\min_{\mathbf{R}\in O(d)} \|\mathbf{R}^\top\mathbf{Z} - \mathbf{A}\|_F^2 \tag{38}$$

Let $\mathbf{C} := \mathbf{Z}\mathbf{A}^\top$ *and let its singular value decomposition be* $\mathbf{C} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$*. Then:*

$$\mathbf{R}^\star = \mathbf{U}\mathbf{V}^\top \tag{39}$$

is an optimizer. If one restricts to $\mathbf{R} \in SO(d)$*, then an optimizer is:*

$$\mathbf{R}^\star = \mathbf{U}\mathbf{D}\mathbf{V}^\top \tag{40}$$

where $\mathbf{D}$ *is diagonal with last entry* $\det(\mathbf{U}\mathbf{V}^\top)$ *and all other diagonal entries equal to* 1*.*

Proof. Expanding the objective here, we get:

$$\|\mathbf{R}^\top\mathbf{Z} - \mathbf{A}\|_F^2 = \operatorname{tr}(\mathbf{Z}\mathbf{Z}^\top) + \operatorname{tr}(\mathbf{A}\mathbf{A}^\top) - 2\operatorname{tr}(\mathbf{R}^\top\mathbf{Z}\mathbf{A}^\top) \tag{41}$$

The first two terms do not depend on $\mathbf{R}$, so minimizing is equivalent to maximizing:

$$\operatorname{tr}(\mathbf{R}^\top\mathbf{C}), \qquad \mathbf{C} = \mathbf{Z}\mathbf{A}^\top \tag{42}$$

Let's write $\mathbf{C} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$. Then, we get:

$$\operatorname{tr}(\mathbf{R}^\top\mathbf{C}) = \operatorname{tr}(\mathbf{V}^\top\mathbf{R}^\top\mathbf{U}\mathbf{\Sigma}) = \operatorname{tr}(\mathbf{M}\mathbf{\Sigma}) \tag{43}$$

where $\mathbf{M} := \mathbf{V}^\top\mathbf{R}^\top\mathbf{U} \in O(d)$. Since $\mathbf{\Sigma}$ is diagonal with nonnegative entries, the maximum is attained when $\mathbf{M} = \mathbf{I}_d$, which gives $\mathbf{R}^\star = \mathbf{U}\mathbf{V}^\top$. The proper-rotation case is obtained by the usual determinant correction. □

A.4.3 Exact Invariance of the Canonicalized Representative

The key point is that whitening and alignment together can produce an exact canonical representative in the idealized regime.

Theorem 2 (Exact gauge invariance of the aligned representative)**.** *Assume* $\varepsilon = 0$*,* $\mathbf{H}_i\mathbf{H}_i^\top \succ 0$ *for every node* i*, and that for every node* i*, the cross-covariance* $\mathbf{C}_i := \mathbf{Z}_i\mathbf{A}^\top$ *has full rank and a simple singular spectrum, so that the orthogonal Procrustes solution is unique. Let* $\mathbf{H}'_i = \mathbf{G}_i\mathbf{H}_i$ *with* $\mathbf{G}_i \in GL(d)$*. Let* $\widetilde{\mathbf{Z}}_i$ *and* $\widetilde{\mathbf{Z}}'_i$ *denote the aligned representatives obtained from* $\mathbf{H}_i$ *and* $\mathbf{H}'_i$ *using the same anchor* $\mathbf{A}$*. Then:*

$$\widetilde{\mathbf{Z}}'_i = \widetilde{\mathbf{Z}}_i \qquad \text{for all } i \tag{44}$$

Proof. By Theorem 1, there exists $\mathbf{Q}_i \in O(d)$ such that:

$$\mathbf{Z}'_i = \mathbf{Q}_i\mathbf{Z}_i \tag{45}$$

Now let:

$$\mathbf{C}_i = \mathbf{Z}_i\mathbf{A}^\top, \qquad \mathbf{C}'_i = \mathbf{Z}'_i\mathbf{A}^\top = \mathbf{Q}_i\mathbf{C}_i \tag{46}$$

Suppose $\mathbf{C}_i = \mathbf{U}_i \mathbf{\Sigma}_i \mathbf{V}_i^\top$ is the SVD of $\mathbf{C}_i$. Then an SVD of $\mathbf{C}'_i$ is:

$$\mathbf{C}'_i = (\mathbf{Q}_i \mathbf{U}_i) \mathbf{\Sigma}_i \mathbf{V}_i^\top \tag{47}$$

By Proposition 2, the unique Procrustes solutions are:

$$\mathbf{R}_i^\star = \mathbf{U}_i \mathbf{V}_i^\top, \qquad \mathbf{R}_i'^\star = (\mathbf{Q}_i \mathbf{U}_i) \mathbf{V}_i^\top = \mathbf{Q}_i \mathbf{R}_i^\star \tag{48}$$

Therefore, we get:

$$\widetilde{\mathbf{Z}}'_i = (\mathbf{R}_i'^\star)^\top \mathbf{Z}'_i = (\mathbf{Q}_i \mathbf{R}_i^\star)^\top \mathbf{Q}_i \mathbf{Z}_i = \mathbf{R}_i^{\star\top} \mathbf{Z}_i = \widetilde{\mathbf{Z}}_i \tag{49}$$

Hence the aligned representative is invariant to the original $GL(d)$ gauge action. □

Corollary 1 (Invariance of the energy descriptor)**.** *Under the assumptions of Theorem 2, the descriptor* $\mathbf{u}_i = \mathrm{diag}(\mathbf{Z}_i^\top \mathbf{Z}_i)$ *is gauge-invariant.*

Proof. By Theorem 1, $\mathbf{Z}'_i = \mathbf{Q}_i \mathbf{Z}_i$ with $\mathbf{Q}_i \in O(d)$, hence:

$$\mathbf{Z}_i'^\top \mathbf{Z}'_i = \mathbf{Z}_i^\top \mathbf{Q}_i^\top \mathbf{Q}_i \mathbf{Z}_i = \mathbf{Z}_i^\top \mathbf{Z}_i \tag{50}$$

Taking diagonals proves the claim. □

A.4.4 Permutation Invariance and Gauge Invariance of the Readout

Theorem 3 (Permutation invariance and gauge compatibility of the readout)**.** *Assume the setting of Theorem 2. Then the readout can be written as:*

$$\mathbf{g}_g = \mathrm{diag}\left(\phi\left[\mathbf{T}, \sum_{i \in V_g} \alpha_i \widetilde{\mathbf{Z}}_i \right]^\top \phi\left[\mathbf{T}, \sum_{i \in V_g} \alpha_i \widetilde{\mathbf{Z}}_i \right] \right) \tag{51}$$

with:

$$\alpha_i = \frac{\exp(s_\theta(\mathbf{u}_i))}{\sum_{j \in V_g} \exp(s_\theta(\mathbf{u}_j))} \tag{52}$$

This is permutation-invariant over nodes and gauge-invariant under node-wise $GL(d)$ reparameterizations.

Proof. Permutation invariance follows because the logits $s_\theta(\mathbf{u}_i)$ are computed independently at each node and the normalization is a symmetric sum over nodes. Therefore the multiset $\{(\alpha_i, \widetilde{\mathbf{Z}}_i)\}_{i \in V_g}$ is merely reindexed under a node permutation, and:

$$\sum_{i \in V_g} \alpha_i \widetilde{\mathbf{Z}}_i \tag{53}$$

is unchanged. Since ϕ, concatenation with $\mathbf{T}$, and the final energy map are deterministic functions of this pooled tensor, the output is permutation-invariant. Gauge invariance follows from Theorem 2 and Corollary 1: under node-wise $GL(d)$ reparameterization, $\widetilde{\mathbf{Z}}_i$ and $\mathbf{u}_i$ are unchanged. Therefore the attention weights α_i are unchanged, so $\mathbf{P}_g$, $\widehat{\mathbf{T}}_g$, and finally $\mathbf{g}_g$ are all unchanged. □

A.5 Classification Token in Vision Transformers Comparison

In order to better understand the role of the late token, we can make a comparison with *classification token* (`[CLS]`) in *Vision Transformers*[16], even if the two mechanisms play fundamentally different roles.

The reason for this difference is structural: while in a ViT, all patch tokens already live in a single global embedding space, so a global token may interact with them directly throughout the encoder, in sheaf neural networks, node states live in local stalk spaces and are only defined up to node-wise basis changes. Hence a token cannot be inserted meaningfully into the encoder without first resolving or otherwise respecting the stalk gauge. Moreover, in

graph message passing, inserting a token node connected to all nodes changes the propagation graph itself. This effectively introduces global shortcuts, reduces the effective graph diameter, and may accelerate oversmoothing by forcing repeated mixing through a global hub. Our token is therefore intentionally *late* and *receive-only*.

Proposition 3 (The late token does not alter encoder propagation)**.** *Let $\mathcal{E}_\theta$ denote the sheaf encoder, so that:*

$$\{\mathbf{H}_i\}_{i\in V_g} = \mathcal{E}_\theta(G_g, X_g) \tag{54}$$

Let $\mathcal{R}_\omega$ denote the late-token readout. Then the full model factors as:

$$\widehat{y}_g = \mathcal{D}_\eta \circ \mathcal{R}_\omega \circ \mathcal{E}_\theta(G_g, X_g) \tag{55}$$

and the encoder mapping $\mathcal{E}_\theta$ is independent of the late token parameters.

Proof. By construction, the late token is introduced only after the encoder has already produced $\{\mathbf{H}_i\}_{i\in V_g}$. It is therefore not part of the message passing graph and does not enter any propagation, diffusion, or attention update inside $\mathcal{E}_\theta$. Consequently the propagation operator of the encoder is unchanged. □

Remark 3. *In this sense, our token is closer to a* post-encoder set summarization token *than to a standard transformer class token. Its job is not to mediate message passing, but to add expressive readout capacity after gauge-compatible aggregation has already taken place.*

A.6 Injective Universal Stalk-Space Readout on the Canonicalized Descriptor Space

As concerns the *injectivity property*, the correct place to state an injectivity theorem is *after canonicalization*, i.e. on the space of gauge-compatible node descriptors. This follows the same multiset philosophy used in expressive GNN theory: if one can map multisets of node descriptors injectively, then graph-level pooling can be made discriminative.

Canonical descriptor space. For each node, define the canonical descriptor as:

$$\xi_i := \big(\mathrm{vec}(\widetilde{\mathbf{Z}}_i), \mathbf{u}_i\big) \tag{56}$$

This descriptor contains both the aligned stalk tensor and the invariant energy information. After exact canonicalization, ξ_i is gauge-invariant.

Assumption 1 (Bounded graphs and countable descriptor universe)**.** *There exists a countable set $\mathfrak{X}$ such that all canonical node descriptors satisfy $\xi_i \in \mathfrak{X}$, and there exists $B \in \mathbb{N}$ such that every graph satisfies $|V_g| \le B$.*

This assumption mirrors the standard bounded-multiset/countable-universe assumption used in expressivity theory for injective multiset aggregation. It is exact when descriptors are discrete, quantized, or drawn from a countable universe; in practice, with continuous features, it should be interpreted as a theorem about the idealized discrete case and as guidance for approximate learnable implementations on compact domains.

Theorem 4 (Existence of an injective gauge-aware multiset sum)**.** *Under the previous assumptions, we know that there exists a function:*

$$\psi : \mathfrak{X} \to \mathbb{R} \tag{57}$$

such that the multiset map:

$$\mathcal{S}(\{\xi_i\}_{i\in V_g}) := \sum_{i\in V_g} \psi(\xi_i) \tag{58}$$

is injective on multisets of canonical descriptors of size at most B.

Proof. Because $\mathfrak{X}$ is countable, there exists an injective indexing map $Z : \mathfrak{X} \to \mathbb{N}$. Let's choose any integer $M > B$. We can define then:

$$\psi(\xi) := M^{-Z(\xi)} \tag{59}$$

We claim that:

$$\mathcal{S}(X) = \sum_{\xi \in X} \psi(\xi) \tag{60}$$

is injective on multisets $X \subset \mathfrak{X}$ of cardinality at most B. Indeed, write a multiset X through its multiplicity function $m_X : \mathfrak{X} \to \{0, 1, \ldots, B\}$. Then, we get:

$$\mathcal{S}(X) = \sum_{\xi \in \mathfrak{X}} m_X(\xi)\, M^{-Z(\xi)} \tag{61}$$

This is a base-M expansion whose "digits" are exactly the multiplicities $m_X(\xi)$, each strictly smaller than M. Such an expansion is unique. Therefore if $\mathcal{S}(X) = \mathcal{S}(Y)$, then all digits coincide, i.e. $m_X(\xi) = m_Y(\xi)$ for every $\xi \in \mathfrak{X}$. Hence $X = Y$ as multisets, proving injectivity. □

Corollary 2 (Universal representation of bounded multiset functions)**.** *Under the same assumptions, for any function $g : \{$bounded multisets over $\mathfrak{X}\} \to \mathcal{Y}$, there exists a function ρ such that:*

$$g(X) = \rho\left(\sum_{\xi \in X} \psi(\xi)\right) \tag{62}$$

where ψ is the injective map from Theorem 4.

Proof. By Theorem 4, the quantity $\sum_{\xi \in X} \psi(\xi)$ uniquely identifies the multiset X. Therefore we may define also:

$$\rho\left(\sum_{\xi \in X} \psi(\xi)\right) := g(X) \tag{63}$$

This is well-defined because the sum is injective. □

The previous theorem/corollary are the exact analogues, on the canonicalized sheaf descriptor space, of the classical injective-sum multiset theorems used to analyze expressive GNNs.

A.6.1 Injective Readout on Gauge Orbits

We now strengthen the previous result by making explicit the quotient-space structure induced by the local gauge action and by stating precisely the condition under which the canonicalization map is not only gauge-invariant, but also *orbit-separating*. This allows us to prove injectivity at the level of bounded multisets of node-wise gauge orbits, rather than only at the level of bounded multisets of canonical descriptors.

Definition 3 (Orbit multiset)**.** *For a graph g, we can define the multiset of node-wise gauge orbits as:*

$$\Omega_g := \big\{[\mathbf{H}_i]_{\mathcal{G}} : i \in V_g\big\} \tag{64}$$

where $[\mathbf{H}_i]_{\mathcal{G}}$ denotes the orbit of $\mathbf{H}_i$ under the local gauge group $\mathcal{G}$.

Assumption 2 (Orbit-separating canonicalization)**.** *Let*

$$\mathcal{C}(\mathbf{H}) := \xi = \big(\mathrm{vec}(\widetilde{\mathbf{Z}}), \mathbf{u}\big) \tag{65}$$

denote the canonical descriptor produced by whitening and alignment. We assume that, on the domain of interest, $\mathcal{C}$ is orbit-separating*, i.e.*

$$\mathcal{C}(\mathbf{H}) = \mathcal{C}(\mathbf{H}') \quad \Longrightarrow \quad [\mathbf{H}]_{\mathcal{G}} = [\mathbf{H}']_{\mathcal{G}}. \tag{66}$$

Equivalently, the induced map on the quotient space

$$\overline{\mathcal{C}} : \mathbb{R}^{d \times C}/\mathcal{G} \to \mathfrak{X}, \qquad \overline{\mathcal{C}}\big([\mathbf{H}]_{\mathcal{G}}\big) := \mathcal{C}(\mathbf{H}), \tag{67}$$

is injective, where $\mathfrak{X}$ is the canonical descriptor space.

Theorem 5 (Injective readout on bounded multisets of gauge orbits)**.** *Assume Theorem 2, Assumption 1, and Assumption 3. Let:*

$$\mathcal{C}(\mathbf{H}_i) := \xi_i = \big(\mathrm{vec}(\widetilde{\mathbf{Z}}_i), \mathbf{u}_i\big) \tag{68}$$

be the canonical descriptor map. Then there exists a graph readout of the form:

$$\mathcal{R}_{\mathrm{inj}}(\{\mathbf{H}_i\}_{i\in V_g}) := \rho\left(\sum_{i\in V_g} \psi(\mathcal{C}(\mathbf{H}_i))\right) \tag{69}$$

such that:

1. $\mathcal{R}_{\mathrm{inj}}$ *is permutation-invariant over nodes;*
2. $\mathcal{R}_{\mathrm{inj}}$ *is gauge-invariant under node-wise* $GL(d)$ *actions;*
3. $\mathcal{R}_{\mathrm{inj}}$ *is injective on bounded multisets of node-wise gauge orbits, i.e.:*

$$\Omega_g \neq \Omega_{g'} \quad \Longrightarrow \quad \mathcal{R}_{\mathrm{inj}}(\{\mathbf{H}_i\}_{i\in V_g}) \neq \mathcal{R}_{\mathrm{inj}}(\{\mathbf{H}'_j\}_{j\in V_{g'}}) \tag{70}$$

In particular, one may choose ρ *to be the identity on the image of the multiset sum.*

Assumption 3 (Orbit-separating canonicalization)**.** *Let*

$$\mathcal{C}(\mathbf{H}) := \xi = \big(\mathrm{vec}(\widetilde{\mathbf{Z}}), \mathbf{u}\big) \tag{71}$$

denote the canonical descriptor produced by whitening and alignment. We assume that, on the domain of interest, $\mathcal{C}$ *is* orbit-separating*, i.e.*

$$\mathcal{C}(\mathbf{H}) = \mathcal{C}(\mathbf{H}') \quad \Longrightarrow \quad [\mathbf{H}]_{\mathcal{G}} = [\mathbf{H}']_{\mathcal{G}}. \tag{72}$$

Equivalently, the induced map on the quotient space

$$\overline{\mathcal{C}} : \mathbb{R}^{d\times C}/\mathcal{G} \to \mathfrak{X}, \qquad \overline{\mathcal{C}}\big([\mathbf{H}]_{\mathcal{G}}\big) := \mathcal{C}(\mathbf{H}), \tag{73}$$

is injective, where $\mathfrak{X}$ *is the canonical descriptor space.*

Theorem 6 (Injective readout on bounded multisets of gauge orbits)**.** *Assume Theorem 2, Assumption 1, and Assumption 3. Let:*

$$\mathcal{C}(\mathbf{H}_i) := \xi_i = \big(\mathrm{vec}(\widetilde{\mathbf{Z}}_i), \mathbf{u}_i\big) \tag{74}$$

be the canonical descriptor map. Then there exists a graph readout of the form:

$$\mathcal{R}_{\mathrm{inj}}(\{\mathbf{H}_i\}_{i\in V_g}) := \rho\left(\sum_{i\in V_g} \psi(\mathcal{C}(\mathbf{H}_i))\right) \tag{75}$$

such that:

1. $\mathcal{R}_{\mathrm{inj}}$ *is permutation-invariant over nodes;*
2. $\mathcal{R}_{\mathrm{inj}}$ *is gauge-invariant under node-wise* $GL(d)$ *actions;*
3. $\mathcal{R}_{\mathrm{inj}}$ *is injective on bounded multisets of node-wise gauge orbits, i.e.:*

$$\Omega_g \neq \Omega_{g'} \quad \Longrightarrow \quad \mathcal{R}_{\mathrm{inj}}(\{\mathbf{H}_i\}_{i\in V_g}) \neq \mathcal{R}_{\mathrm{inj}}(\{\mathbf{H}'_j\}_{j\in V_{g'}}) \tag{76}$$

In particular, one may choose ρ *to be the identity on the image of the multiset sum.*

Proof. We proceed in four steps.

Step 1: The canonical descriptor induces a well-defined map on gauge orbits.

By Theorem 2, if two node embeddings belong to the same gauge orbit, i.e. $\mathbf{H}' = \mathbf{GH}$

for some $\mathbf{G} \in \mathcal{G}$, then their aligned representatives and invariant descriptors coincide. Therefore, we have $\mathcal{C}(\mathbf{H}') = \mathcal{C}(\mathbf{H})$. Hence $\mathcal{C}$ is constant on gauge orbits, and so it induces a well-defined quotient map:

$$\overline{\mathcal{C}} : \mathbb{R}^{d\times C}/\mathcal{G} \to \mathfrak{X}, \qquad \overline{\mathcal{C}}\big([\mathbf{H}]_{\mathcal{G}}\big) := \mathcal{C}(\mathbf{H}) \tag{77}$$

Step 2: The quotient map is injective on the domain of interest. By Assumption 3, the induced map $\overline{\mathcal{C}}$ is injective. Therefore two distinct gauge orbits are mapped to two distinct canonical descriptors. In particular, if $[\mathbf{H}]_{\mathcal{G}} \neq [\mathbf{H}']_{\mathcal{G}}$, then, we have:

$$\overline{\mathcal{C}}\big([\mathbf{H}]_{\mathcal{G}}\big) \neq \overline{\mathcal{C}}\big([\mathbf{H}']_{\mathcal{G}}\big) \tag{78}$$

Thus, on the domain of interest, the canonical descriptor does not merely forget the gauge but it provides a faithful representative of the gauge orbit.

Step 3: Canonical orbit multisets are injectively encoded by a multiset sum. Let $\Xi_g := \{\xi_i : i \in V_g\} = \{\mathcal{C}(\mathbf{H}_i) : i \in V_g\}$ be the multiset of canonical descriptors associated with graph g. Since $\mathfrak{X}$ is countable and graph sizes are bounded by Assumption 1, Theorem 4 guarantees the existence of a function $\psi : \mathfrak{X} \to \mathbb{R}^m$ for some m such that the multiset sum $\mathcal{S}(\Xi_g) := \sum_{\xi\in\Xi_g} \psi(\xi)$ is injective on bounded multisets of descriptors. Equivalently, we have that:

$$\mathcal{S}(\Xi_g) = \mathcal{S}(\Xi_{g'}) \quad \Longrightarrow \quad \Xi_g = \Xi_{g'} \tag{79}$$

Step 4: Injectivity transfers from descriptor multisets to orbit multisets. Now suppose that two graphs g and g' satisfy $\mathcal{R}_{\text{inj}}(\{\mathbf{H}_i\}_{i\in V_g}) = \mathcal{R}_{\text{inj}}(\{\mathbf{H}'_j\}_{j\in V_{g'}})$. Choose ρ to be injective on the image of the multiset sum and, in particular, one may simply take ρ to be the identity. Then:

$$\sum_{i\in V_g} \psi(\mathcal{C}(\mathbf{H}_i)) = \sum_{j\in V_{g'}} \psi(\mathcal{C}(\mathbf{H}'_j)) \tag{80}$$

By injectivity of the multiset sum from Step 3, this implies that $\Xi_g = \Xi_{g'}$. Since $\overline{\mathcal{C}}$ is injective by Step 2, equality of descriptor multisets implies equality of the corresponding multisets of gauge orbits, i.e., $\Omega_g = \Omega_{g'}$. Taking the contrapositive, if $\Omega_g \neq \Omega_{g'}$, then:

$$\mathcal{R}_{\text{inj}}(\{\mathbf{H}_i\}_{i\in V_g}) \neq \mathcal{R}_{\text{inj}}(\{\mathbf{H}'_j\}_{j\in V_{g'}}) \tag{81}$$

This proves injectivity on bounded multisets of node-wise gauge orbits.

Finally, permutation invariance follows immediately from the commutativity of summation: $\sum_{i\in V_g} \psi(\mathcal{C}(\mathbf{H}_i))$ depends only on the multiset $\{\mathcal{C}(\mathbf{H}_i)\}_{i\in V_g}$ and not on any ordering of the nodes. Gauge invariance follows from Step 1, since $\mathcal{C}$ is constant on each gauge orbit. Therefore the readout is permutation-invariant, gauge-invariant, and injective on bounded multisets of node-wise gauge orbits. □

Remark 4 (Practical Interpretation)**.** *Theorem 6 shows that, once the local gauge has been removed through an orbit-separating canonicalization, graph-level readout reduces to injective multiset encoding over canonical node descriptors. This is the precise sheaf analogue of injective multiset readout results in expressive graph representation learning: the only additional subtlety is that injectivity must now be established on* gauge orbits*, not on raw node coordinates.*

A.7 Graph-level Readout Overhead

For graph classification, the sheaf diffusion backbone is followed by a graph-level readout (pooling) step that aggregates stalk-space node representations into a single graph embedding. We consider five readout strategies: three parameter-free reductions (Mean, Max, Add), an attention-weighted pool, and a late-interaction token pool. Table 3 reports the parameter complexity of each readout using the same notation as table 25: n nodes, stalk dimension d, feature channels f, $c = df$, output classes K, and attention heads h.

Table 3: **Parameter complexity of graph-level readout strategies.** Notation follows table 25: stalk dimension d, channels f, $c = df$, output classes K, attention heads h.

Readout	**Parameters**	**Computational complexity**
MEAN / MAX / ADD	$K(c+1)$	$\mathcal{O}(Kc + nc)$
Attention	$c(c+h+1) + K(c+1)$	$\mathcal{O}(c^2 + nc)$
Late token	$2df + 4f^2 + 6f + 1 + K(f+1)$	$\mathcal{O}(f^2 + nf)$

The parameter-free reductions (MEAN, MAX, ADD) introduce only the final linear classifier mapping from $\mathbb{R}^c$ to $\mathbb{R}^K$, contributing $\mathcal{O}(Kc)$ parameters. The runtime cost is dominated by the $\mathcal{O}(nc)$ global summation over the node set. Because K is typically small (equal to the number of classes), this overhead is negligible relative to the diffusion layers. The attention readout adds a gating network that computes per-node importance scores before aggregation; its dominant parameter cost is $\mathcal{O}(c^2)$, arising from the query and key projections, and therefore scales quadratically with c rather than with any graph-level quantity. The late-token readout, which appends a learnable [GRAPH] token and processes it through a cross-attention layer, exhibits a similar $\mathcal{O}(f^2)$ scaling in the feature dimension.

Crucially, in all five cases the readout cost depends exclusively on the model hyperparameters (d, f, K, h) and is independent of the number of edges m, up to the $\mathcal{O}(n)$ summation inherent in any global pooling. Because the sheaf diffusion layers themselves already scale as $\mathcal{O}(mc)$ or $\mathcal{O}(md^2c)$ per layer (table 25), and $m \gg 1$ for all benchmarks considered in this work, the readout overhead is dominated by the diffusion cost, making the choice of pooling strategy effectively free from a computational standpoint.

B Additional Experimental, Ablation, and Implementation Details

This appendix consolidates all supplementary material related to the empirical study and implementation of HetSheaf. We begin by describing the experimental protocol, hyperparameter search space, and datasets used throughout the paper. We then present a detailed model analysis, including ablations over heterogeneous sheaf predictors, restriction-map families, attention-based variants, and architectural choices such as depth and stalk dimension. Next, we provide broader comparisons with recent heterogeneous GNNs reported in the literature, in order to situate HetSheaf within the wider benchmark landscape. Finally, we collect the main implementation details, including hardware, decoders, restriction-map parameterisations, the full family of heterogeneous sheaf predictors, computational overhead, graph-level readout costs, and the heterogeneous extensions of Sheaf Attention Networks and Polynomial Neural Sheaf Diffusion. Together, these experiments complement the main paper by clarifying both the empirical behaviour and the practical design choices behind the model.

B.1 Experimental Protocol, Hyperparameters, and Datasets

We gather here the information necessary to reproduce the empirical setup used in the main paper. We first describe the hyperparameter search procedure shared across the evaluated models, and then summarise both the benchmark datasets and the task-specific datasets used in our study.

B.1.1 Model Hyperparameters

Table 4 reports the search space used to optimise all models. For each method, we adopt the same approach usually followed by the main Sheaf Neural Networks frameworks [4], [2], [5] and we run large-scale Bayesian hyperparameter sweeps [54, 61] and select the best configuration according to the validation metric appropriate to the task. This shared protocol allows a fair comparison between homogeneous baselines, heterogeneous GNNs, and sheaf-based models under a common tuning budget.

Table 4: **Hyperparameter search space.**

Hyperparameter	Search space
Sheaf type	$\{\text{general}, O(d), \text{diagonal}, \text{SheafAN}\}$
Sheaf learner	$\{\text{NSD}, \text{TE}, \text{EE}, \text{NE}, \text{types}, \text{NT}, \text{ET}\}$
Attention Learner†	$\{\text{Yes}, \text{No}\}$
d	$\{2, 3, 4, 5\}$
# Layers	$\{2, 3, 4, 5, 6, 7, 8\}$
input_dropout	$[0, 0.9]$
dropout	$[0, 0.9]$
initial_dropout	$[0, 0.9]$
Learning rate	$\{1, 5\} \times \{10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}\}$
Weight decay	$\{1, 5\} \times \{10^{-5}, 10^{-4}, 10^{-3}\}$

† Not supported for *diagonal* sheaf type.

B.2 Additional Results and Model Analysis

This section complements the headline results from the main paper with a more detailed analysis of where the gains of HetSheaf come from. We study in particular the effect of the heterogeneous sheaf predictor, the restriction-map family, the use of attention-based restriction-map constructions, and architectural choices such as depth and stalk dimension. The goal is not only to identify the strongest configuration, but also to understand which modelling components matter most for different tasks and datasets.

Table 5: **Statistics of benchmark datasets.**

Node classification	# Nodes	# Node types	# Edges	# Edge types	Target	# Classes
DBLP	26,128	4	239,566	6	author	4
IMDB	21,420	4	86,642	6	movie	5
ACM	10,942	4	547,806	7	paper	3
Link prediction					Target	
LastFM	20,612	3	141,521	3	user-artist	
MovieLens	10,352	2	201,672	2	user-movie	
PubMed	63,109	4	244,986	10	disease-disease	
YouTube	2,000	1	1,177,130	5	user-user	
Amazon	10,099	1	148,659	2	product-product	
Graph classification	# Graphs	# Node types	# Edges	# Edge types	Target	
MUTAG	188	7	7,442	4	mutagenicity	
PROTEINS	1,113	3	162,088	1	enzyme/non-enzyme	
ENZYMES	600	3	19,580	1	enzyme class	
NCI1	4,110	37	49,205	1	anti-cancer	

B.2.1 Benchmark Datasets

Each of the datasets used for node classification is publicly available as part of the Heterogeneous Graph Benchmark (HGB)[2] first introduced by Lv et al. [48]. The link prediction datasets (LastFM, MovieLens, PubMed and Amazon) are also drawn from HGB. On the other hand, graph classification datasets are drawn from the TU Datasets benchmark [50], a widely used collection of graph-level prediction benchmarks spanning biological and chemical domains. We provide brief descriptions of each dataset.

- **DBLP**[3] is a citation graph based on the DBLP computer science bibliographic database. It has four node types: authors, papers, venues and terms. The six edge types include paper-author, paper-venue, paper-term and author-venue. The target is to predict the class label of authors based on their research area: databases, data mining, AI or information retrieval.
- **IMDB**[4] is an information graph containing movie data. The four node types are movies, directors, actors and keywords. The six edge types include movie-director, movie-actor and movie-keyword. Movies are classified based on their genre: action, comedy, drama, romance and thriller. Each movie can have multiple genres, i.e., a multi-label node classification task.
- **ACM** [68] is a citation graph containing node types of authors, papers, terms and subjects. The edge types include paper-cite-paper, paper-author, paper-subject and paper-term. The target is to predict each paper's category: databases, wireless communication, or data mining.
- **LastFM**[5] is a knowledge graph generated from an online music website. The three node types are user, artist, and tags with edge types of user-artist, user-user and artist-tag. The link prediction task aims to predict edges between users and artists.
- **MovieLens**[6] is a heterogeneous rating graph from the MovieLens website and consists of two node types: movie and user. The task is to predict whether or not a user rates a movie.
- **PubMed**[7] is a biomedical knowledge graph from the HGB benchmark [48], constructed from the PubMed literature database. It contains four node types (gene, disease, chemical and species) connected by ten edge types encoding biological relations such as gene-disease associations, chemical-chemical interactions and species-disease links. The link prediction task targets disease-disease edges.

[2] `https://www.biendata.xyz/hgb/`
[3] `https://web.cs.ucla.edu/~yzsun/data/`
[4] `https://www.kaggle.com/datasets/karrrimba/movie-metadatacsv`
[5] `https://grouplens.org/datasets/hetrec-2011/`
[6] `https://movielens.org/`
[7] `https://pubmed.ncbi.nlm.nih.gov`

- **Amazon** is a single-node-type product graph from the HGB benchmark [48] containing 10,099 items connected by co-viewing and co-purchasing relations (two edge types). The link prediction task targets product-product edges.
- **YouTube** is a single-node-type social network from the HGB benchmark [48] containing 2,000 users connected by five types of interactions. The link prediction task targets user-user edges.
- **MUTAG** [50] is a collection of 188 mutagenic aromatic and heteroaromatic nitro compounds, with 7 atom types and 4 bond types. The task is to predict whether a compound has a mutagenic effect on a bacterium.
- **PROTEINS** [50] is a dataset of 1,113 proteins represented as graphs, where nodes are secondary structure elements with 3 node types and edges connect amino acids that are neighbours in the sequence or in 3D space. The task is to classify proteins as enzymes or non-enzymes.
- **ENZYMES** [50] is a dataset of 600 protein tertiary structures obtained from the BRENDA enzyme database, with 3 node types representing secondary structure elements. The task is to classify each protein into one of 6 enzyme commission top-level classes.
- **NCI1** [50] is a dataset of 4,110 chemical compounds screened for their ability to suppress or inhibit the growth of a panel of human tumour cell lines, with 37 atom types. The task is to predict anti-cancer activity.

B.2.2 Biomedical Datasets

The following datasets are used exclusively for biomedical link prediction [38, 70] experiments and are provided in the appendix for completeness.

Table 6: **Statistics of biomedical link prediction datasets.**

Biomedical	# Nodes	# Node types	# Edges	# Edge types	Target
CTD-DDA	12,765	2	43,492	1	drug-disease
DrugBank-DDI	2,191	1	242,027	1	drug-drug
NDFRT-DDA	13,545	2	4,709	1	drug-disease
String-PPI	15,131	1	359,776	1	protein-protein

- **CTD-DDA**[8] is a heterogeneous graph derived from the Comparative Toxicogenomics Database, containing curated associations between chemical compounds and diseases. The link prediction task aims to predict drug-disease associations.
- **DrugBank**[8] is a comprehensive heterogeneous graph constructed from the DrugBank database [69], containing drugs, targets, enzymes, and carriers. The link prediction task aims to predict drug-target interactions.
- **NDFRT-DDA**[8] is a heterogeneous graph derived from the National Drug File Reference Terminology, containing drug-disease associations curated from clinical data. The task is to predict drug-disease associations.
- **String-PPI**[8] is a protein-protein interaction network derived from the STRING database [64], containing functional associations between proteins across multiple organisms. The task is to predict protein-protein interactions.

Table 7 reports link prediction results on four biomedical networks. HetSheaf ranks first on CTD-DDA (99.53 AUPR, 99.31 AUROC), DrugBank (94.92 AUPR), and NDFRT-DDA (98.23 AUPR, 97.67 AUROC), while Sheaf-NSD leads on String-PPI (96.20 AUPR, 94.40 AUROC). The two sheaf models consistently occupy the top two positions across all datasets, with R-GCN as the strongest non-sheaf competitor. HAN and HGT fail on CTD-DDA and NDFRT-DDA (AUROC ≈ 52, near random), whereas the sheaf models remain robust. The slight advantage of Sheaf-NSD over HetSheaf on String-PPI is consistent with this dataset having a single node type, which limits the benefit of heterogeneous type conditioning.

[8] `https://github.com/Zaiwen/Link_Prediction_in_Biomedical_Network`

Table 7: **Performance on heterogeneous biomedical networks link prediction benchmarks.** Results for the heterogeneous sheaf learners and baselines are shown. We report the highest binary AUROC and AUPR scores achieved across the hyperparameter sweep for each model. The top three models are coloured by **First**, **Second** and **Third**.

	CTD-DDA		DrugBank		NDFRT-DDA		String-PPI	
	AUPR	AUROC	AUPR	AUROC	AUPR	AUROC	AUPR	AUROC
Homogeneous								
GCN	**99.36**	**99.09**	93.46	91.57	97.80	96.51	95.11	92.62
GAT	89.55	89.92	88.73	87.46	94.01	92.88	94.77	92.26
GIN	99.34	99.07	93.71	92.43	97.83	96.44	95.27	92.77
SAGE	93.78	93.82	94.20	92.28	94.50	92.87	**95.94**	93.78
Heterogeneous								
R-GCN	99.03	98.93	**94.77**	**94.13**	**98.18**	**97.32**	95.79	**94.15**
HAN	63.34	51.76	88.68	85.54	63.89	52.82	93.39	90.08
HGT	63.32	51.72	89.95	87.38	63.97	52.99	93.07	90.27
Sheaf-NSD	**99.47**	**99.23**	**94.90**	**93.90**	**98.13**	**97.02**	**96.20**	**94.40**
HetSheaf (ours)	**99.53**	**99.31**	**94.92**	**93.93**	**98.23**	**97.67**	**96.05**	**94.24**

B.2.3 Networking Datasets

The following datasets are used exclusively for networking link prediction and node classification experiments and are provided in the appendix for completeness.

Table 8: **Statistics of networking datasets.**

Networking	# Nodes	# Node types	# Edges	# Edge types	Task	Target
AS-733	6,474	1	13,895	3	LP	customer-provider
RouteNet Traffic Models	∼674	2	∼1,200	2	NC	link congestion

- **AS-733**[9] is a heterogeneous graph of the Internet's Autonomous System (AS) topology, derived from BGP routing tables collected by the University of Oregon Route Views Project [44]. Nodes represent Autonomous Systems and edges represent BGP peering relationships, classified into three semantically distinct types: *customer–provider* (paid transit), *peer–to–peer* (settlement-free peering), and *provider–customer* (reverse transit). The link prediction task aims to predict missing or future peering agreements between Autonomous Systems. The dataset contains 6,474 nodes and 13,895 edges from the January 2, 2000 snapshot.
- **RouteNet Traffic Models**[10] is a bipartite heterogeneous graph for network performance modelling, generated using the BNNetSimulator [19, 20]. The graph consists of two node types: *link* nodes (physical network connections with bandwidth capacity features) and *path* nodes (logical end-to-end routes with traffic and delay features), connected by *traverses* and *belongs_to* edges encoding the routing table. The node classification task predicts which links become congested (utilization exceeding a threshold of 0.50, yielding a 54%/46% class balance). The dataset comprises five traffic generation patterns: Constant Bit Rate (CBR), which produces deterministic fixed-rate traffic; ON-OFF, which alternates between active and idle periods to model bursty applications; Autocorrelated Exponentials, where successive inter-arrival times exhibit temporal correlations capturing long-range dependence; and Modulated Exponentials, whose rate parameter varies over time to model changing activity levels. We evaluate on the *All Multiplexed* configuration, which mixes all four traffic types simultaneously across different source-destination pairs, creating the most realistic scenario in which the model must handle qualitatively different congestion dynamics within a single graph. Training uses NSFNET (14 nodes) and GEANT2 (24 nodes) topologies.

Table 9 reports results on two networking benchmarks. On AS-733, HetSheaf achieves the highest AUPR (97.94) and AUROC (96.81), followed closely by Sheaf-NSD (97.84 / 96.65).

[9] `https://snap.stanford.edu/data/as-733.html`
[10] `https://github.com/BNN-UPC/NetworkModelingDatasets`, v5

Table 9: **Performance on networking benchmarks.** Results for the heterogeneous sheaf learners and baselines are shown. We report the highest binary AUROC and AUPR for AS-733 (link prediction) and the highest Macro F1 and Micro F1 for RouteNet Traffic Models (node classification), achieved across the hyperparameter sweep for each model. The top three models are coloured by **First**, **Second** and **Third**.

	AS-733 (LP)		RouteNet Traffic (NC)	
	AUPR	AUROC	Macro F1	Micro F1
Homogeneous				
GCN	97.25	96.18	**80.57**	**81.25**
GAT	93.43	91.20	33.33	50.00
GIN	**97.33**	96.23	**87.30**	**87.50**
SAGE	96.71	95.21	33.33	50.00
Heterogeneous				
R-GCN	97.15	**96.59**	**62.50**	**62.50**
HAN	93.80	94.03	33.33	50.00
HGT	96.31	95.77	33.33	50.00
Sheaf-NSD	**97.84**	**96.65**	**87.30**	**87.50**
HetSheaf (ours)	**97.94**	**96.81**	**87.30**	**87.50**

The three heterogeneous edge types (customer–provider, peer–to–peer, provider–customer) encode semantically distinct peering relationships whose directionality and economic asymmetry provide a natural testbed for type-aware restriction maps; even a modest gain over Sheaf-NSD suggests that the heterogeneous predictor captures inter-type structure that a single shared learner averages away. Among baselines, GIN and GCN are competitive on AUPR, while R-GCN attains the third-best AUROC, indicating that explicit relation typing helps on this task even without sheaf structure.

On RouteNet Traffic Models, HetSheaf and GIN both reach the top Macro F1 (87.30) and Micro F1 (87.50), with GCN as the second-best model (80.57 / 81.25). Notably, GAT, SAGE, HAN, and HGT all collapse to majority-class predictions (Macro F1 of 33.33, Micro F1 of 50.00), suggesting that the bipartite link–path topology and the diversity of traffic generation patterns (CBR, ON-OFF, Autocorrelated, Modulated Exponentials) pose a difficult optimisation landscape for attention-based and certain heterogeneous architectures. R-GCN avoids the degenerate solution but lags considerably behind GIN and the sheaf models, confirming that per-relation linear transforms alone are insufficient to capture traffic-dependent congestion dynamics. The fact that HetSheaf matches GIN, the strongest homogeneous baseline, while additionally providing interpretable stalk-space representations, underscores its versatility across graph domains.

B.2.4 Sheaf Predictor Ablation Study

Tables 10–12 report the full ablation grid over all heterogeneous sheaf predictors and restriction-map families for node classification, link prediction, and graph classification, respectively. These results isolate the contribution of explicit type conditioning from the contribution of the geometric family itself.

For node classification (Table 10), heterogeneous predictors consistently improve over the vanilla NSD baseline on at least one dataset, confirming that typed conditioning of the restriction maps is beneficial in practice. On ACM and DBLP, the strongest configurations typically pair typed predictors with polynomial or general restriction-map families, indicating that type conditioning and higher-order diffusion are complementary. On IMDB, however, the best results are obtained by non-polynomial families, suggesting that on semantically noisier heterogeneous graphs a simpler local transport geometry may generalise more robustly. Notably, predictors that use only type indicators without node features remain competitive, indicating that type information alone already carries substantial signal for heterogeneous node classification.

For link prediction (Table 11), the gap between predictors is generally smaller, consistent with the fact that several datasets are already close to saturation under AUROC/AUPR. Even so, typed predictors remain consistently competitive with or stronger than NSD. General

Table 10: **Ablation study of the impact of different restriction maps and sheaf predictor types on node classification performance.** The highest performing restriction map type for each sheaf learner is highlighted, and the highest performing sheaf learner is shown in **bold**.

Sheaf Predictor		ACM		DBLP		IMDB	
		Macro F1	Micro F1	Macro F1	Micro F1	Macro F1	Micro F1
NSD	Diag	91.51	91.41	91.32	91.97	61.96	65.33
	Bundle	90.40	90.46	92.12	92.64	61.23	64.68
	General	89.81	89.75	89.33	89.96	61.77	64.85
	Diag-Poly	92.05	91.97	91.38	91.87	60.56	63.24
	Bundle-Poly	89.07	88.95	88.95	89.58	59.47	62.62
	Gen-Poly	86.74	86.64	91.33	91.76	59.62	62.62
	SheafAN	87.64	87.58	91.08	91.73	54.38	58.79
TE	Diag	91.99	91.88	94.03	94.51	63.55	65.40
	Bundle	91.17	91.12	91.71	92.22	62.94	66.39
	General	91.63	91.50	93.13	93.66	61.62	64.58
	Diag-Poly	**94.22**	**94.15**	94.34	94.79	61.52	64.71
	Bundle-Poly	92.07	91.97	93.04	93.56	60.26	63.29
	Gen-Poly	88.57	88.39	94.08	94.47	60.59	63.71
	SheafAN	88.20	88.20	92.39	92.96	56.06	58.83
Ensemble	Diag	92.58	92.45	93.38	93.84	63.22	65.83
	Bundle	90.29	90.23	93.11	93.56	62.62	65.47
	General	91.39	91.31	93.93	94.40	61.45	64.48
	Diag-Poly	93.36	93.25	94.26	94.65	60.34	63.73
	Bundle-Poly	87.63	87.54	90.08	90.70	61.45	63.47
	Gen-Poly	90.97	90.89	94.16	94.65	62.50	65.27
	SheafAN	86.48	86.54	81.94	83.63	55.79	60.21
NE	Diag	91.90	91.78	93.10	93.56	**63.65**	**66.56**
	Bundle	91.15	91.08	92.94	93.45	61.28	65.00
	General	91.74	91.69	93.68	94.08	62.26	65.06
	Diag-Poly	92.55	92.49	93.40	93.84	61.30	64.11
	Bundle-Poly	89.60	89.52	91.82	92.36	59.73	63.77
	Gen-Poly	91.29	91.17	94.34	94.75	60.22	62.97
	SheafAN	88.76	88.76	91.06	91.76	54.90	58.52
EE	Diag	92.50	92.40	93.50	93.91	62.86	66.49
	Bundle	91.46	91.50	92.52	92.99	62.16	64.39
	General	91.90	91.78	93.77	94.23	61.86	64.56
	Diag-Poly	93.76	93.67	94.35	94.79	60.57	63.46
	Bundle-Poly	84.38	84.37	85.66	86.55	56.44	61.57
	Gen-Poly	88.31	88.15	94.11	94.58	60.94	64.33
	SheafAN	88.14	88.05	90.62	91.23	55.56	59.23
NT	Diag	93.72	93.63	93.53	94.01	62.48	65.22
	Bundle	90.99	90.84	92.29	92.82	62.16	63.86
	General	91.66	91.55	93.50	93.98	60.95	63.99
	Diag-Poly	92.32	92.26	94.34	94.75	60.03	63.72
	Bundle-Poly	88.76	88.67	93.27	93.80	60.39	63.60
	Gen-Poly	93.58	93.48	94.06	94.47	61.18	64.20
	SheafAN	88.95	88.95	81.45	82.25	55.25	58.33
ET	Diag	92.95	92.87	94.11	94.51	61.41	64.57
	Bundle	89.90	89.80	90.35	90.85	60.23	63.42
	General	92.08	91.97	93.95	94.37	62.99	65.78
	Diag-Poly	93.07	92.97	**94.91**	**95.28**	60.71	63.80
	Bundle-Poly	89.62	89.47	86.73	87.61	60.35	63.52
	Gen-Poly	91.11	91.08	94.78	95.18	59.69	63.32
	SheafAN	88.39	88.39	81.76	82.78	55.64	58.78

and diagonal restriction maps perform particularly well on recommendation-style datasets, while the ensemble predictor occasionally benefits from edge-type-specific routing when the relation semantics are rich enough to justify separate predictors.

Table 11: **Restriction map and sheaf predictor ablation study for heterogeneous link prediction.** The highest performing restriction map type for each sheaf learner is highlighted, and the highest performing sheaf learner is shown in **bold**.

Sheaf Predictor		LastFM		MovieLens		Amazon		YouTube	
		AUPR	AUROC	AUPR	AUROC	AUPR	AUROC	AUPR	AUROC
NSD	Diag	98.06	97.73	99.68	99.60	92.59	90.03	89.06	82.85
	Bundle	97.69	97.35	99.66	99.60	92.58	89.98	89.30	83.19
	General	97.98	97.68	99.68	**99.62**	92.48	89.95	88.98	82.73
	Diag-Poly	97.34	96.56	99.65	99.56	92.60	89.79	89.45	83.32
	Bundle-Poly	97.49	97.04	99.67	99.58	92.69	90.00	89.37	83.23
	Gen-Poly	97.62	97.22	99.63	99.57	92.25	89.37	–	–
	SheafAN	93.05	91.52	99.48	99.37	91.30	87.95	82.45	76.24
TE	Diag	98.39	98.24	**99.69**	99.60	93.85	91.59	89.46	83.38
	Bundle	98.50	98.20	99.65	99.58	93.51	91.22	89.01	82.78
	General	98.45	98.30	99.67	99.60	93.71	91.55	88.87	82.58
	Diag-Poly	98.40	98.02	99.66	99.57	93.33	90.71	89.39	83.36
	Bundle-Poly	98.35	98.01	99.68	99.60	93.27	90.77	89.35	83.24
	Gen-Poly	98.30	98.07	99.65	99.59	93.49	91.28	89.21	82.95
	SheafAN	97.71	97.26	99.51	99.42	91.40	87.62	86.44	80.23
Ensemble	Diag	98.47	98.16	99.68	99.60	93.87	91.52	90.53	85.61
	Bundle	97.98	97.72	99.66	99.61	93.45	91.10	88.79	82.45
	General	**98.54**	**98.33**	99.67	99.60	93.76	91.58	90.65	85.95
	Diag-Poly	98.08	97.51	99.61	99.56	93.46	90.88	89.49	83.46
	Bundle-Poly	97.49	96.95	99.67	99.59	93.40	90.95	89.39	83.30
	Gen-Poly	98.39	98.07	99.66	99.61	93.55	91.23	–	–
	SheafAN	96.43	95.39	99.48	99.41	91.99	88.52	87.10	80.61
NE	Diag	98.43	98.24	**99.69**	99.61	92.59	90.04	89.27	83.36
	Bundle	98.42	98.16	99.67	99.60	92.49	89.88	89.33	83.23
	General	98.46	98.25	99.67	99.61	92.64	90.08	89.25	83.10
	Diag-Poly	98.42	98.05	99.66	99.57	92.56	89.79	89.41	83.28
	Bundle-Poly	98.22	97.87	99.67	99.58	92.53	89.90	89.38	83.21
	Gen-Poly	98.30	98.02	99.66	99.59	92.48	89.50	–	–
	SheafAN	97.81	97.56	99.47	99.36	91.46	88.21	82.68	76.62
EE	Diag	98.41	98.16	99.68	99.60	**93.89**	91.65	89.95	84.06
	Bundle	97.62	97.28	99.66	99.61	93.62	91.27	88.86	82.65
	General	98.51	98.13	99.67	99.61	93.71	91.51	**91.05**	**86.76**
	Diag-Poly	97.87	97.34	99.62	99.53	93.29	90.53	89.87	84.02
	Bundle-Poly	97.74	97.19	99.67	99.58	93.12	90.63	89.40	83.31
	Gen-Poly	98.17	97.77	99.66	99.59	93.60	91.30	–	–
	SheafAN	86.12	79.22	99.46	99.40	91.27	87.48	86.11	79.63
NT	Diag	98.47	98.25	99.68	99.61	92.59	90.13	89.31	83.26
	Bundle	98.45	98.24	99.66	99.61	92.37	89.75	89.23	83.17
	General	98.41	98.29	99.67	99.61	92.51	89.95	89.34	83.24
	Diag-Poly	98.28	98.08	99.66	99.57	92.61	89.82	89.41	83.26
	Bundle-Poly	98.22	97.94	99.68	99.59	92.60	89.89	89.40	83.29
	Gen-Poly	98.32	98.08	99.66	99.58	92.50	89.98	–	–
	SheafAN	97.74	97.31	99.50	99.41	91.26	87.53	82.54	76.39
ET	Diag	98.43	98.15	**99.69**	99.61	**93.89**	91.56	90.74	86.03
	Bundle	97.56	97.18	99.67	99.61	92.31	89.85	89.26	83.11
	General	98.43	98.16	99.67	99.60	93.81	**91.71**	90.62	85.95
	Diag-Poly	97.97	97.40	99.63	99.55	93.18	90.49	90.62	85.82
	Bundle-Poly	97.69	97.20	99.65	99.58	92.71	89.95	89.28	83.11
	Gen-Poly	98.03	97.72	99.67	99.58	93.77	91.58	–	–
	SheafAN	95.87	95.66	99.56	99.49	90.89	86.58	86.29	79.95

For graph classification (Table 12), predictor choice has a much stronger effect on performance, especially on PROTEINS, ENZYMES, and NCI1. Here, type-only predictors such as NT and ET often become surprisingly effective. This is consistent with the inductive nature of graph classification: when the model must generalise across unseen graphs, structural type information can transfer more robustly than instance-specific node features. Overall, the graph-classification ablations support the view that typed restriction-map inference is particularly valuable when the model must compress heterogeneous local geometry into a stable graph-level representation.

Table 12: **Ablation study of the impact of different restriction maps and sheaf predictor types on graph classification performance.** The highest performing restriction map type for each sheaf learner is highlighted, and the highest performing sheaf learner is shown in **bold**.

Sheaf Predictor		MUTAG		PROTEINS		ENZYMES		NCI1	
		Macro F1	Micro F1	Macro F1	Micro F1	Macro F1	Micro F1	Macro F1	Micro F1
NSD	Diag	75.00	75.00	62.20	62.50	18.82	28.33	55.11	55.11
	Bundle	68.75	70.00	73.60	75.89	9.56	23.33	38.69	38.69
	General	76.19	80.00	26.67	33.04	8.75	23.33	–	–
	Diag-Poly	96.00	96.00	51.77	51.79	8.38	21.67	45.73	45.73
	Bundle-Poly	94.88	95.00	71.56	76.79	26.67	30.00	63.24	63.24
	Gen-Poly	89.01	90.00	66.21	71.43	5.71	20.00	72.00	72.00
	SheafAN	89.90	90.00	24.32	32.14	9.48	20.00	62.08	62.08
TE	Diag	75.00	75.00	49.98	50.00	12.70	21.67	61.85	61.85
	Bundle	69.70	70.00	64.54	65.18	12.66	21.67	62.08	62.08
	General	37.50	60.00	27.22	33.93	7.31	18.33	62.08	62.08
	Diag-Poly	95.65	95.65	33.60	35.71	13.20	23.33	61.36	61.36
	Bundle-Poly	**100.00**	**100.00**	62.78	71.43	18.54	23.33	68.62	68.62
	Gen-Poly	100.00	100.00	55.36	55.36	30.75	33.33	72.54	72.54
	SheafAN	84.00	85.00	24.32	32.14	16.98	21.67	69.70	69.70
Ensemble	Diag	78.57	78.57	73.60	75.89	11.85	20.00	29.03	29.03
	Bundle	69.70	70.00	58.28	58.93	15.92	25.00	21.43	21.43
	General	67.03	70.00	53.20	53.57	5.56	20.00	64.42	64.42
	Diag-Poly	95.65	95.65	41.27	67.86	8.20	21.67	62.08	62.08
	Bundle-Poly	100.00	100.00	69.22	71.43	24.52	25.00	51.18	51.18
	Gen-Poly	100.00	100.00	58.92	58.93	38.05	41.67	54.98	54.98
	SheafAN	37.50	60.00	24.32	32.14	15.22	23.33	62.08	62.08
NE	Diag	75.00	75.00	68.93	72.32	16.10	23.33	9.95	9.95
	Bundle	78.02	80.00	71.96	74.11	6.38	21.67	23.11	23.11
	General	64.29	70.00	28.90	33.04	9.96	21.67	54.15	54.15
	Diag-Poly	100.00	100.00	25.79	33.04	16.42	28.33	60.20	60.20
	Bundle-Poly	100.00	100.00	72.97	74.11	19.31	26.67	69.52	69.52
	Gen-Poly	100.00	100.00	63.39	66.96	13.66	20.00	41.41	41.41
	SheafAN	89.01	90.00	75.47	80.36	35.53	38.33	24.63	24.63
EE	Diag	76.19	80.00	63.87	64.29	20.03	28.33	46.99	46.99
	Bundle	37.50	60.00	25.79	33.04	5.56	20.00	18.35	18.35
	General	79.80	80.00	38.00	40.18	6.02	21.67	59.94	59.94
	Diag-Poly	96.00	96.00	38.24	41.07	13.62	21.67	52.10	52.10
	Bundle-Poly	94.88	95.00	54.29	60.71	32.82	31.67	63.89	63.89
	Gen-Poly	94.88	95.00	27.22	33.93	10.47	20.00	66.09	66.09
	SheafAN	94.88	95.00	66.16	68.75	17.13	20.00	62.08	62.08
NT	Diag	80.00	80.00	73.60	75.89	**52.96**	**53.33**	**78.24**	**78.24**
	Bundle	79.80	80.00	69.84	70.54	50.80	50.00	62.64	62.64
	General	74.94	75.00	63.25	63.39	49.92	51.67	70.85	70.85
	Diag-Poly	94.88	95.00	72.51	74.11	22.67	26.67	23.01	23.01
	Bundle-Poly	94.88	95.00	**78.84**	80.36	50.01	50.00	74.74	74.74
	Gen-Poly	89.90	90.00	78.66	**81.25**	48.13	48.33	72.86	72.86
	SheafAN	89.90	90.00	76.95	79.46	46.58	48.33	75.06	75.06
ET	Diag	76.19	80.00	48.65	67.86	12.73	26.67	57.60	57.60
	Bundle	74.42	75.00	54.37	54.46	10.39	25.00	44.92	44.92
	General	89.90	90.00	24.32	32.14	8.80	21.67	59.20	59.20
	Diag-Poly	94.88	95.00	61.35	68.75	9.15	21.67	62.08	62.08
	Bundle-Poly	94.88	95.00	65.06	74.11	15.57	25.00	44.88	44.88
	Gen-Poly	100.00	100.00	24.32	32.14	17.86	23.33	72.87	72.87
	SheafAN	58.33	60.00	61.23	61.61	17.86	20.00	62.08	62.08

B.2.5 Effect of Restriction Map Type

The predictor ablations also reveal a second important axis: the choice of restriction-map family. Across tasks, no single family dominates universally. Polynomial families are often strongest on node classification and on several graph classification benchmarks, where longer-range typed interactions are beneficial. By contrast, non-polynomial diagonal or bundle families can be preferable on more semantically heterogeneous or noisier datasets such as IMDB and ENZYMES. This task dependence is itself informative: it suggests that the optimal local transport geometry depends on whether the problem rewards smooth long-range propagation, local typed alignment, or robust inductive generalisation.

For node classification, Diag-Poly and Gen.-Poly are often strongest on ACM and DBLP, whereas non-polynomial families dominate on IMDB. For link prediction, the restriction-map family tends to have a smaller effect, with diagonal maps performing consistently well and general maps providing only modest gains in some settings. For graph classification, polynomial families are essential on some datasets but can also become unstable on others, especially when small graph size and structural diversity interact poorly with higher-order spectral filtering. This reinforces the importance of treating the restriction-map family as a genuine modelling choice rather than as a fixed default.

B.2.6 Attention-based Restriction Map Variants

Tables 13–15 repeat the main ablation study using attention-based restriction-map constructions. These experiments test whether replacing the standard predictor output by a normalised attention-style map provides a useful inductive bias.

For node classification (Table 13), attention-based variants broaden the set of competitive configurations and occasionally outperform their non-attention counterparts, especially when paired with bundle or general maps. For link prediction (Table 14), attention predictors are competitive but do not provide a uniform improvement, suggesting that the advantage of additional normalisation depends strongly on the relation structure of the dataset. For graph classification (Table 15), the picture is mixed: attention expands the space of viable configurations, but does not yield a systematic gain over the standard construction. Overall, attention-based restriction maps are a useful modelling option, but not a universal replacement.

B.2.7 Stalk Dimension Ablation Study

Tables 16–18 report the effect of varying the stalk dimension $d \in \{2, 3, 4, 5\}$ across all restriction map families, tasks, and datasets. Each cell reports Macro-F1 / Micro-F1 for classification tasks and AUPR / AUROC for link prediction. Several patterns emerge.

First, no single stalk dimension is universally optimal: the best d varies by dataset and family, suggesting that d should be treated as a hyperparameter tuned per task. For instance, Gen.-P peaks at $d=2$ on both ACM (94.3/94.2) and DBLP (95.0/95.4), but at $d=3$ on PROTEINS (80.6/82.1) and $d=2$ on NCI1 (79.5/79.5).

Second, polynomial variants (Diag-P, Bundle-P, Gen.-P) consistently match or outperform their non-polynomial counterparts on node and graph classification. Gen.-P achieves the global best Macro-F1 on ACM (94.3), DBLP (95.0), PROTEINS (80.6), and NCI1 (79.5). On MUTAG, both Bundle-P and Gen.-P reach perfect 100.0/100.0 across all stalk dimensions, indicating that the polynomial spectral filtering combined with orthogonal or general restriction maps provides sufficient expressivity to fully separate the two classes. However, this advantage does not extend uniformly to all datasets: on ENZYMES, Diag-P performs poorly (peaking at only 25.6/30.0), well below the non-polynomial Diag (53.0/53.3) and Bundle (62.3/63.3 at $d=4$).

Third, SheafAN lags behind the Laplacian-based families on node classification, particularly on IMDB where it peaks at 56.4/59.3 Macro/Micro-F1 versus 64.5/67.2 for Bundle. The gap is smaller but consistent on ACM and DBLP. However, SheafAN is more competitive on link prediction, where it achieves strong AUPR/AUROC on LastFM (97.9/97.6 at $d=5$) and near-saturated performance on MovieLens. This suggests that the attention-based adjacency

Table 13: **Ablation study of the impact of different restriction maps and sheaf attention predictor types on node classification performance.** The highest performing restriction map type for each sheaf learner is highlighted, and the highest performing sheaf learner is shown in **bold**.

Sheaf Predictor		ACM		DBLP		IMDB	
		Macro F1	Micro F1	Macro F1	Micro F1	Macro F1	Micro F1
Att. NSD	Bundle	89.49	89.38	92.10	92.57	61.88	64.49
	General	90.35	90.32	90.94	91.58	61.71	64.51
	Bundle-Poly	88.24	88.20	90.51	91.13	61.29	64.04
	Gen-Poly	90.93	90.89	92.96	93.42	59.88	63.40
	SheafAN	88.67	88.62	87.77	88.77	55.64	59.05
Att. TE	Bundle	90.22	90.18	93.21	93.70	63.88	66.34
	General	91.76	91.64	93.13	93.52	62.16	65.55
	Bundle-Poly	88.11	87.91	94.56	95.00	61.00	64.28
	Gen-Poly	90.18	90.09	92.79	93.27	62.39	65.81
	SheafAN	88.01	87.96	85.70	86.65	55.25	59.35
Att. Ens.	Bundle	**92.57**	**92.54**	93.46	93.94	**64.54**	**67.18**
	General	90.75	90.65	93.96	94.40	62.38	65.37
	Bundle-Poly	89.28	89.14	93.28	93.77	59.15	64.31
	Gen-Poly	88.65	88.62	93.91	94.33	58.87	62.83
	SheafAN	89.72	89.71	81.28	81.94	56.23	59.23
Att. NE	Bundle	90.76	90.70	93.66	94.12	63.03	66.47
	General	91.09	90.98	93.11	93.59	60.32	63.51
	Bundle-Poly	89.67	89.52	92.10	92.71	60.97	63.72
	Gen-Poly	88.98	88.95	94.61	95.04	58.30	61.27
	SheafAN	87.15	87.02	81.32	81.94	55.25	58.70
Att. EE	Bundle	91.79	91.69	93.32	93.77	63.50	66.44
	General	90.59	90.51	93.95	94.44	60.97	64.70
	Bundle-Poly	86.21	86.26	92.90	93.59	60.00	62.22
	Gen-Poly	91.01	90.93	92.70	93.24	60.53	63.10
	SheafAN	85.59	85.41	91.13	91.87	54.78	58.32
Att. NT	Bundle	90.78	90.70	93.73	94.23	63.00	66.37
	General	90.95	90.84	93.52	93.98	60.77	63.84
	Bundle-Poly	89.72	89.61	92.89	93.35	60.01	62.54
	Gen-Poly	91.52	91.41	94.17	94.58	59.34	63.35
	SheafAN	88.16	88.15	80.82	81.44	55.76	58.38
Att. ET	Bundle	91.76	91.69	93.81	94.26	62.57	65.97
	General	91.14	91.08	94.67	95.07	62.02	64.87
	Bundle-Poly	90.30	90.23	**94.67**	**95.11**	58.11	61.80
	Gen-Poly	88.28	88.20	93.85	94.30	62.29	65.11
	SheafAN	85.49	85.41	83.86	84.61	56.36	59.18

aggregation of SheafAN is better suited for pairwise scoring tasks than for node-level classification.

Fourth, link prediction performance is largely insensitive to d. AUPR and AUROC are near-saturated across all configurations (e.g., 98.3–98.5 AUPR on LastFM, 99.5–99.7 on MovieLens), and vary by less than 0.3 percentage points within each family. This contrasts sharply with node and graph classification, where performance swings of 5–20 points across stalk dimensions are common (e.g., General on MUTAG ranges from 74.9 at $d=5$ to 94.9 at $d=3$).

Finally, the non-polynomial families (Diag, Bundle, General) tend to be more stable across d than their polynomial counterparts. The polynomial families occasionally exhibit sharp performance drops, such as Diag-P on ENZYMES falling from 25.6 at $d=3$ to 14.0 at $d=4$, or Gen.-P on ACM degrading from 94.3 at $d=2$ to 90.6 at $d=5$. This instability may stem from the higher-order Chebyshev coefficients amplifying noise when the graph structure does not support long-range spectral patterns, and reinforces the recommendation to treat both d and the restriction map family as hyperparameters.

Table 14: **Ablation study of the impact of different restriction maps and sheaf attention predictor types on link prediction performance.** The highest performing restriction map type for each sheaf learner is highlighted, and the highest performing sheaf learner is shown in **bold**.

Sheaf Predictor		LastFM		MovieLens		Amazon		YouTube	
		AUPR	AUROC	AUPR	AUROC	AUPR	AUROC	AUPR	AUROC
Att. NSD	Bundle	97.98	97.71	99.66	99.60	92.53	90.00	89.32	83.36
	General	97.85	97.52	99.66	99.60	92.46	89.79	89.22	83.09
	Bundle-Poly	97.56	97.20	99.66	99.58	92.63	90.04	89.31	83.10
	Gen-Poly	97.86	97.37	99.66	99.59	92.54	89.81	–	–
	SheafAN	94.25	92.73	99.47	99.35	92.03	89.22	82.70	76.66
Att. TE	Bundle	98.46	98.20	99.65	99.59	93.85	91.59	89.17	82.99
	General	98.33	98.24	99.66	**99.61**	93.87	91.59	89.24	83.10
	Bundle-Poly	98.16	97.94	**99.68**	99.60	93.54	91.12	89.36	83.25
	Gen-Poly	97.10	96.71	99.65	99.57	92.78	90.07	–	–
	SheafAN	97.74	97.50	99.47	99.35	92.14	89.04	86.56	80.22
Att. Ens.	Bundle	98.43	98.13	99.66	99.60	93.82	91.61	**90.73**	**85.89**
	General	98.42	98.21	99.66	**99.61**	93.67	91.48	89.26	83.09
	Bundle-Poly	98.28	98.06	99.66	99.59	93.61	91.15	89.43	83.36
	Gen-Poly	98.16	97.83	99.64	99.57	**100.00**	**100.00**	64.60	64.60
	SheafAN	96.78	96.11	99.48	99.36	92.23	89.05	86.72	80.14
Att. NE	Bundle	98.45	98.28	99.65	99.59	92.67	90.02	89.31	83.20
	General	98.43	98.25	99.66	**99.61**	92.54	90.04	89.18	83.00
	Bundle-Poly	98.25	97.90	99.66	99.59	92.60	89.92	89.33	83.18
	Gen-Poly	98.29	97.93	99.67	99.59	92.58	89.90	–	–
	SheafAN	97.62	97.22	99.50	99.38	91.74	88.70	83.05	76.96
Att. EE	Bundle	98.38	98.17	99.65	99.60	93.78	91.54	89.28	83.17
	General	98.44	98.14	99.64	99.58	93.87	91.55	89.93	84.48
	Bundle-Poly	98.05	97.61	99.66	99.59	93.54	91.13	89.41	83.26
	Gen-Poly	**100.00**	**100.00**	99.65	99.58	93.61	91.23	–	–
	SheafAN	94.78	92.51	99.52	99.41	91.81	88.42	86.27	80.13
Att. NT	Bundle	98.41	98.24	99.66	99.59	92.55	90.05	89.28	83.16
	General	98.45	98.28	99.65	99.60	92.41	89.94	89.04	82.83
	Bundle-Poly	98.36	97.98	99.66	99.58	92.63	89.82	89.39	83.19
	Gen-Poly	98.34	98.06	99.67	99.59	92.56	89.79	–	–
	SheafAN	97.94	97.62	99.49	99.42	91.18	87.58	82.40	76.25
Att. ET	Bundle	98.35	98.13	99.66	99.60	93.75	91.44	90.27	84.88
	General	98.43	98.19	99.66	99.60	93.82	91.55	90.05	84.88
	Bundle-Poly	98.27	97.95	99.67	99.58	93.41	91.09	89.42	83.34
	Gen-Poly	98.22	97.82	99.66	99.58	93.81	91.49	–	–
	SheafAN	93.70	91.14	99.52	99.44	91.92	87.95	86.11	79.88

B.2.8 Layer Depth Ablation Study

Tables 19–21 report the effect of varying the number of message passing layers $L \in \{2, 3, 4, 5, 6, 7, 8\}$ across all restriction map families, tasks, and datasets, alongside the homogeneous and heterogeneous GNN baselines. Each cell reports Macro-F1 / Micro-F1 for classification tasks and AUPR / AUROC for link prediction. Several patterns emerge.

The most striking observation is the *absence of catastrophic oversmoothing* in HetSheaf. On ACM, the best Macro-F1 across all sheaf families varies by at most 3.4 percentage points between the shallowest ($L=2$) and deepest ($L=8$) configuration (Gen.-P: 94.3/94.2 at $L=2$, 92.6/92.5 at $L=8$), and several families (Diag, General) achieve their peak performance at $L \geq 6$. In contrast, the homogeneous baselines suffer dramatic degradation at depth: on ACM, GAT drops from 92.2/92.1 at $L=3$ to 17.0/34.3 at $L=8$, GCN from 92.3/92.2 at $L=2$ to 32.8/40.9 at $L=8$, and GIN from 88.7/88.8 at $L=2$ to 41.3/43.7 at $L=8$. Even the heterogeneous baselines are not immune: HAN collapses from 93.6/93.6 at $L=5$ to 27.2/37.1 at $L=8$ on ACM, and HGT drops from 92.3/92.3 at $L=2$ to 40.1/46.0 at $L=8$. The only baseline that remains stable across depth is R-GCN, whose edge-type-specific parameterisation provides a form of oversmoothing resistance, though at a substantially higher parameter cost (Table 26).

On DBLP, the advantage of depth is more pronounced for sheaf models. Several families (General, Gen.-P) achieve their best results at $L \geq 4$, with Gen.-P peaking at 95.0/95.4

Table 15: **Ablation study of the impact of different restriction maps and sheaf attention predictor types on graph classification performance.** The highest performing restriction map type for each sheaf learner is highlighted, and the highest performing sheaf learner is shown in **bold**.

Sheaf Predictor		MUTAG		PROTEINS		ENZYMES		NCI1	
		Macro F1	Micro F1	Macro F1	Micro F1	Macro F1	Micro F1	Macro F1	Micro F1
Att. NSD	Bundle	64.91	70.00	25.79	33.04	33.65	35.00	55.12	55.12
	General	71.51	75.00	24.32	32.14	5.56	20.00	35.44	35.44
	Bundle-Poly	76.19	80.00	24.32	32.14	12.17	20.00	62.08	62.08
	Gen-Poly	**100.00**	**100.00**	66.75	71.43	14.13	23.33	**72.30**	**72.30**
	SheafAN	94.88	95.00	52.37	52.68	12.35	21.67	48.47	48.47
Att. TE	Bundle	79.80	80.00	54.29	54.46	18.27	26.67	59.88	59.88
	General	41.33	60.00	63.25	63.39	8.01	21.67	58.82	58.82
	Bundle-Poly	59.60	65.00	72.57	75.89	21.29	21.67	62.08	62.08
	Gen-Poly	100.00	100.00	49.42	50.00	16.60	28.33	60.06	60.06
	SheafAN	78.02	80.00	75.24	76.79	17.05	28.33	–	–
Att. Ens.	Bundle	84.00	85.00	43.49	44.64	18.47	30.00	60.38	60.38
	General	73.33	75.00	64.22	66.96	5.56	20.00	63.49	63.49
	Bundle-Poly	73.33	75.00	62.53	63.39	22.13	23.33	19.44	19.44
	Gen-Poly	94.67	95.00	41.70	43.75	16.62	23.33	–	–
	SheafAN	94.88	95.00	28.63	34.82	15.35	20.00	19.18	19.18
Att. NE	Bundle	94.88	95.00	24.32	32.14	36.67	43.33	55.36	55.36
	General	73.33	75.00	25.25	32.14	17.36	30.00	28.09	28.09
	Bundle-Poly	94.88	95.00	49.60	67.86	2.98	8.33	63.02	63.02
	Gen-Poly	100.00	100.00	70.10	74.11	19.45	31.67	5.26	5.26
	SheafAN	71.51	75.00	63.04	63.39	31.74	35.00	62.08	62.08
Att. EE	Bundle	78.02	80.00	33.97	38.39	16.35	26.67	62.30	62.30
	General	94.88	95.00	24.32	32.14	19.35	25.00	60.44	60.44
	Bundle-Poly	94.88	95.00	63.25	63.39	21.27	23.33	19.18	19.18
	Gen-Poly	100.00	100.00	41.27	43.75	23.58	26.67	–	–
	SheafAN	37.50	60.00	70.05	71.43	18.49	26.67	11.76	11.76
Att. NT	Bundle	79.17	80.00	78.84	80.36	51.23	53.33	67.98	67.98
	General	79.17	80.00	46.27	46.43	13.17	26.67	24.45	24.45
	Bundle-Poly	84.65	85.00	**79.69**	**81.25**	**56.71**	**56.67**	–	–
	Gen-Poly	89.58	90.00	75.94	80.36	41.23	43.33	70.06	70.06
	SheafAN	84.96	85.00	76.95	79.46	46.52	50.00	64.30	64.30
Att. ET	Bundle	84.65	85.00	24.32	32.14	7.78	21.67	22.81	22.81
	General	79.17	80.00	38.92	41.96	8.67	23.33	47.85	47.85
	Bundle-Poly	89.90	90.00	24.32	32.14	24.06	25.00	64.74	64.74
	Gen-Poly	100.00	100.00	65.17	66.07	21.86	23.33	69.89	69.89
	SheafAN	94.88	95.00	24.32	32.14	13.36	20.00	62.08	62.08

Table 16: Node classification: Macro-F1 / Micro-F1 (%) by restriction map family and stalk dimension d. Cyan = best in family, **bold** = global best per dataset.

	ACM				DBLP				IMDB			
Family	$d=2$	$d=3$	$d=4$	$d=5$	$d=2$	$d=3$	$d=4$	$d=5$	$d=2$	$d=3$	$d=4$	$d=5$
Diag	93.7/93.6	93.0/92.9	92.6/92.5	93.0/92.9	94.0/94.5	94.1/94.5	94.1/94.5	93.5/94.0	62.4/65.4	63.6/66.6	62.8/65.9	63.2/65.8
Bundle	92.6/92.5	91.2/91.1	91.0/90.8	92.1/92.1	93.8/94.3	93.7/94.2	93.2/93.7	93.4/93.9	64.4/**67.2**	**64.5**/66.6	63.9/66.5	64.2/66.4
General	91.8/91.7	91.7/91.7	91.9/91.8	92.1/92.0	93.8/94.3	94.0/94.4	94.7/95.1	94.0/94.4	63.5/65.6	62.5/65.5	62.4/65.4	63.0/65.8
Diag-P	93.3/93.2	93.1/93.0	94.2/94.1	93.5/93.4	94.9/95.3	94.7/95.1	94.9/95.3	94.6/95.0	61.5/64.7	61.6/64.6	61.1/64.1	60.2/63.6
Bundle-P	90.3/90.2	92.1/92.0	89.3/89.1	90.3/90.2	94.7/95.1	93.3/93.8	94.6/95.0	93.6/94.0	61.5/64.3	61.8/64.6	61.0/64.3	61.9/65.2
Gen.-P	**94.3/94.2**	93.6/93.5	93.1/93.1	90.6/90.5	**95.0/95.4**	94.4/94.8	94.6/95.0	94.6/95.0	62.4/65.8	62.5/64.9	62.3/65.3	60.9/64.6
SheafAN	88.9/89.0	89.7/89.7	88.8/88.8	90.9/90.9	90.9/91.6	91.1/91.9	92.4/93.0	87.3/88.3	56.1/60.2	56.2/59.2	55.9/59.2	56.4/59.3

Table 17: Graph classification: Macro-F1 / Micro-F1 (%) by restriction map family and stalk dimension d. Cyan = best in family, **bold** = global best per dataset.

	MUTAG				PROTEINS				ENZYMES				NCI1			
Family	$d=2$	$d=3$	$d=4$	$d=5$	$d=2$	$d=3$	$d=4$	$d=5$	$d=2$	$d=3$	$d=4$	$d=5$	$d=2$	$d=3$	$d=4$	$d=5$
Diag	80.0/80.0	80.0/80.0	83.3/83.3	76.9/76.9	74.6/75.9	67.3/67.9	73.6/75.9	61.5/67.9	53.0/53.3	46.1/46.7	34.2/35.0	39.1/40.0	72.1/72.1	68.5/68.5	73.7/73.7	61.8/61.8
Bundle	78.0/80.0	84.7/85.0	94.9/95.0	84.7/85.0	74.8/76.8	75.8/77.7	78.8/80.4	54.4/54.5	50.8/50.0	51.2/53.3	**62.3/63.3**	51.0/51.7	62.6/62.6	57.6/57.6	68.0/68.0	60.4/60.4
General	89.0/90.0	94.9/95.0	89.9/90.0	74.9/75.0	71.3/72.3	64.2/74.1	78.7/81.2	64.2/67.0	49.9/51.7	47.7/50.0	50.7/51.7	46.9/46.7	63.2/63.2	64.4/64.4	67.5/67.5	62.1/62.1
Diag-P	95.7/95.7	95.7/95.7	96.0/96.0	**100.0/100.0**	51.8/67.9	78.9/**82.1**	71.6/74.1	63.2/74.1	22.7/28.3	25.6/30.0	14.0/21.7	14.9/23.3	60.2/60.2	62.1/62.1	62.1/62.1	61.4/61.4
Bundle-P	**100.0/100.0**	**100.0/100.0**	**100.0/100.0**	**100.0/100.0**	75.6/77.7	78.8/80.4	78.4/80.4	79.8/**82.1**	52.6/53.3	46.5/48.3	51.9/51.7	56.7/56.7	69.2/69.2	54.4/54.4	74.7/74.7	76.3/76.3
Gen.-P	**100.0/100.0**	**100.0/100.0**	**100.0/100.0**	**100.0/100.0**	75.9/80.4	**80.6/82.1**	78.7/81.2	77.5/79.5	34.4/36.7	41.6/43.3	41.7/43.3	48.1/48.3	**79.5/79.5**	72.9/72.9	71.5/71.5	73.0/73.0
SheafAN	89.9/90.0	94.9/95.0	94.9/95.0	94.9/95.0	75.8/78.6	76.9/79.5	77.2/79.5	76.7/80.4	46.6/48.3	49.3/51.7	41.6/41.7	37.3/40.0	72.7/72.7	71.9/71.9	73.6/73.6	75.1/75.1

Table 18: Link prediction: (AUPR / AUROC, %). by restriction map family and stalk dimension d. Cyan = best in family, **bold** = global best per dataset.

	LastFM				MovieLens				Amazon				YouTube			
Family	$d=2$	$d=3$	$d=4$	$d=5$	$d=2$	$d=3$	$d=4$	$d=5$	$d=2$	$d=3$	$d=4$	$d=5$	$d=2$	$d=3$	$d=4$	$d=5$
Diag	98.4/98.2	98.5/98.2	98.4/98.2	98.5/98.2	**99.7**/99.6	99.7/99.6	99.7/99.6	**99.7**/99.6	93.8/91.5	93.8/91.6	93.8/91.6	93.8/91.5	90.5/85.4	90.7/86.0	90.6/85.7	90.6/85.9
Bundle	98.5/98.2	98.4/98.3	98.5/98.2	98.5/98.2	99.7/**99.6**	99.7/99.6	99.7/99.6	99.7/99.6	93.7/91.3	93.8/91.6	93.7/91.5	93.8/91.6	90.1/84.4	89.3/83.2	90.4/85.4	90.7/85.9
General	98.5/98.3	98.5/98.3	98.5/98.3	98.5/98.3	99.7/99.6	99.7/**99.6**	99.7/**99.6**	99.7/**99.6**	93.8/91.6	93.8/91.6	93.8/91.6	93.8/91.7	90.7/86.0	90.1/84.9	**91.3/87.3**	**91.3/87.3**
Diag-P	98.3/97.9	98.4/98.0	98.3/98.0	98.4/98.1	99.6/99.5	99.7/99.6	99.6/99.6	99.7/99.6	93.2/90.6	93.3/90.5	93.5/90.9	93.3/90.8	90.1/85.3	90.5/85.8	90.7/86.5	90.6/85.4
Bundle-P	98.3/98.0	98.3/98.1	98.3/98.1	98.4/98.0	99.7/99.6	99.7/99.6	99.7/99.6	99.7/99.6	93.5/91.1	93.5/91.1	93.6/91.1	93.6/91.2	89.4/83.4	89.4/83.3	89.4/83.3	89.4/83.3
Gen.-P	98.3/98.1	98.3/98.0	98.3/98.1	**98.5/98.3**	99.7/99.6	99.7/99.6	99.7/99.6	99.7/99.6	**93.8/91.7**	93.8/91.5	93.8/91.6	93.8/91.4	89.2/83.0	-/-	-/-	-/-
SheafAN	97.7/97.5	97.7/97.3	97.8/97.2	97.9/97.6	99.5/99.4	99.5/99.4	99.6/99.5	99.5/99.4	91.9/88.8	92.0/88.7	92.2/89.1	92.1/89.2	87.0/80.5	87.1/80.6	87.0/80.5	86.1/79.6

at $L = 4$ and remaining above 93.9/94.3 through $L = 8$. The baselines also benefit from moderate depth on DBLP (GIN peaks at 94.8/95.2 at $L=7$), but none match the best sheaf configurations. On IMDB, where all methods struggle, the sheaf families show a mild but consistent benefit from additional layers: Diag and Bundle both achieve their best results at $L \geq 6$ (Diag: 63.6/66.6 at $L=8$; Bundle: 64.5/66.6 at $L=6$), suggesting that deeper message passing helps capture the complex multi-label structure of this dataset. Notably, the IMDB baselines are extremely brittle: GCN and HAN oscillate between functional performance at odd layer counts and near-collapse at even ones, a pattern that is entirely absent from the sheaf models.

For graph classification, the depth–performance relationship is noisier, reflecting the smaller test sets and inductive evaluation protocol. On ENZYMES, deeper sheaf models consistently outperform shallower ones: Bundle achieves 62.3/63.3 at $L = 8$ (its global best) versus 38.2/43.3 at $L = 2$, and Diag reaches 53.0/53.3 at $L = 8$ versus 31.0/40.0 at $L = 2$. The baselines on ENZYMES are uniformly poor (GCN: 9–13, GAT: 6–14, SAGE: 6–9 Macro-F1 across all depths), and only HAN (46.1/45.0 at $L=6$) and HGT (48.3/48.3 at $L=5$) reach moderate performance. On NCI1, Gen.-P peaks at 79.5/79.5 at $L=7$, and Diag reaches 78.2/78.2 at $L=3$, both substantially above the homogeneous baselines (GCN: 64.2/65.0, GAT: 64.2/65.0 at best). The polynomial families exhibit greater variance across layers than the non-polynomial families: on NCI1, Diag-P ranges from 61.4 to 62.1, whereas Diag ranges from 61.8 to 78.2. This is consistent with the observation from the stalk dimension ablation that polynomial spectral filtering amplifies sensitivity to hyperparameter choices.

For link prediction (Table 21), all sheaf families are effectively invariant to depth. On LastFM, AUPR varies by at most 0.2 percentage points across layers for all non-polynomial families, and AUROC varies by at most 0.3 points. On MovieLens, performance is near-constant across all depths and families (AUPR $\geq$ 99.5 and AUROC $\geq$ 99.4 for every sheaf configuration), substantially outperforming HAN which is essentially non-functional on this dataset (AUPR $\approx$ 63.0 regardless of depth). On Amazon and YouTube, the baselines are more competitive, with HGT achieving the strongest baseline AUPR on both datasets (96.2 and 95.0 respectively), but the sheaf models match or exceed these scores across all layer depths.

Overall, these results demonstrate that the sheaf Laplacian diffusion process provides a principled mechanism for deep message passing without the oversmoothing pathology that limits standard GNNs to shallow architectures. The restriction maps provide per-edge transport operators that preserve discriminative signal across layers, while the polynomial variants add spectral filtering that can exploit deeper receptive fields when the graph structure supports it. In practice, we recommend treating L as a hyperparameter tuned alongside d and the restriction map family, with the expectation that $L \in \{3, \ldots, 6\}$ providing a robust default range.

B.3 External Comparisons with Additional Heterogeneous GNNs

The main paper compares HetSheaf against a controlled set of strong homogeneous, heterogeneous, and sheaf-based baselines trained under a unified codebase and tuning protocol. To provide additional context, this section compares against a broader set of heterogeneous GNNs reported in recent surveys and benchmarks [47, 48, 82]. Since these baselines were not re-run under a single implementation and, in some cases, use different

Table 19: Node classification: Macro-F1 / Micro-F1 (%) by model and number of layers L. Cyan = best in family, **bold** = global best per dataset.

	ACM							DBLP							IMDB						
	$L=2$	$L=3$	$L=4$	$L=5$	$L=6$	$L=7$	$L=8$	$L=2$	$L=3$	$L=4$	$L=5$	$L=6$	$L=7$	$L=8$	$L=2$	$L=3$	$L=4$	$L=5$	$L=6$	$L=7$	$L=8$
Homogeneous baselines																					
GCN	92.3/92.2	92.1/92.0	65.2/67.3	53.6/57.0	38.6/42.3	30.8/39.1	32.8/40.9	82.2/83.0	88.4/89.2	88.4/89.2	89.8/90.6	90.4/91.1	91.0/91.7	87.0/88.2	15.5/25.8	58.3/60.9	15.4/39.4	57.1/60.1	23.3/45.2	58.2/62.1	16.6/28.2
GAT	92.1/92.0	92.2/92.1	83.1/84.1	28.1/38.2	29.2/37.6	28.8/38.3	17.0/34.3	84.4/85.1	90.3/91.0	89.4/89.9	92.3/92.8	90.0/90.7	91.1/91.7	91.4/92.0	24.7/32.9	60.2/64.0	14.1/40.0	59.0/61.3	24.7/32.9	55.7/61.3	23.3/45.2
GIN	88.7/88.8	86.8/86.5	35.3/45.2	49.0/49.4	42.2/43.3	41.6/45.4	41.3/43.7	79.9/80.5	93.6/93.9	93.2/93.7	93.2/93.7	92.9/93.4	94.8/95.2	92.9/93.3	53.1/56.9	60.2/63.9	55.7/60.0	57.3/60.8	52.2/58.5	59.1/62.2	52.1/56.7
SAGE	91.4/91.3	91.8/91.7	92.0/91.9	89.1/89.0	90.6/90.5	92.2/92.1	85.1/85.0	83.6/84.6	88.0/89.0	90.1/90.7	88.4/88.9	89.8/90.5	88.9/89.7	87.7/88.5	58.2/62.4	58.6/61.2	59.5/61.8	60.6/63.8	55.6/57.6	59.2/63.4	58.1/63.6
Heterogeneous baselines																					
R-GCN	92.0/91.9	93.3/93.2	92.9/92.8	92.1/92.0	91.3/91.2	93.0/92.9	92.0/91.9	93.6/94.0	94.3/94.7	93.6/94.1	94.0/94.5	92.8/93.4	93.8/94.2	93.4/93.9	58.8/62.3	59.3/61.5	59.7/62.4	58.3/61.2	59.1/61.5	59.3/61.6	58.4/60.9
HAN	92.2/92.1	91.4/91.3	91.0/90.9	93.6/93.6	92.3/92.3	27.1/37.0	27.2/37.1	82.7/83.3	92.1/92.6	92.8/93.3	90.8/91.4	92.7/93.2	91.8/92.4	94.0/94.4	14.1/40.0	56.8/61.7	17.2/29.5	55.5/58.9	15.5/25.8	25.5/39.5	14.1/40.0
HGT	92.3/92.3	91.5/91.4	91.8/91.8	29.8/37.0	91.7/91.7	89.0/88.8	40.1/46.0	82.7/83.6	90.6/91.3	92.8/93.3	91.0/91.7	91.2/91.7	91.3/91.9	88.2/89.0	56.5/60.2	41.4/49.4	51.3/53.6	51.5/55.3	39.0/56.1	35.3/41.8	38.8/46.2
HetSheaf (ours)																					
Diag	93.0/92.9	92.6/92.4	93.0/92.9	92.6/92.5	93.7/93.6	92.9/92.9	92.5/92.5	84.5/85.2	94.1/94.5	93.5/94.0	93.5/93.9	93.5/94.0	94.0/94.5	93.8/94.2	59.1/63.0	62.2/65.2	62.2/65.0	62.5/65.4	62.9/65.4	63.2/66.5	63.6/66.6
Bundle	91.5/91.5	91.0/90.9	92.6/92.5	92.1/92.1	90.6/90.5	91.8/91.7	91.8/91.7	84.9/85.8	93.0/93.5	92.6/93.1	93.4/93.9	93.7/94.2	93.3/93.8	93.8/94.3	59.2/63.2	62.6/65.5	62.5/66.0	62.7/65.7	64.5/66.6	63.9/66.5	64.4/**67.2**
General	91.9/91.8	91.8/91.7	91.8/91.7	91.7/91.6	91.1/91.0	91.5/91.4	92.1/92.0	85.0/85.7	93.7/94.1	94.4/94.9	94.7/95.1	94.0/94.4	94.0/94.4	93.9/94.4	58.9/62.7	62.3/64.8	60.9/64.1	63.5/65.8	62.4/65.4	62.5/65.5	62.0/64.9
Diag-P	94.2/94.1	93.8/93.7	93.9/93.8	92.5/92.4	92.6/92.5	92.3/92.3	93.5/93.4	94.5/94.9	94.9/95.3	94.7/95.1	94.6/95.0	94.4/94.8	94.2/94.6	93.9/94.4	61.5/64.7	61.1/64.1	60.5/63.7	60.3/63.5	61.6/64.6	60.6/63.2	61.3/64.1
Bundle-P	89.0/88.9	90.2/90.1	89.3/89.1	92.1/92.0	90.3/90.2	89.6/89.5	90.3/90.2	93.3/93.8	93.6/94.0	94.7/95.1	91.4/92.0	93.4/93.9	78.6/79.5	91.8/92.4	60.4/64.6	61.8/64.2	61.9/65.2	61.3/64.0	59.9/64.3	61.5/64.3	60.0/64.1
Gen.-P	**94.3/94.2**	91.0/90.9	93.6/93.5	91.8/91.7	91.5/91.4	90.9/90.9	92.6/92.5	93.9/94.3	94.7/95.1	**95.0/95.4**	94.6/95.0	94.6/95.0	94.8/95.2	93.9/94.3	60.3/63.3	60.8/64.2	62.4/65.8	**62.5/64.9**	62.3/65.3	62.3/65.1	61.2/63.7
SheafAN	88.7/88.6	88.3/88.4	88.4/88.4	88.8/88.7	88.2/88.1	90.9/90.9	88.2/88.2	86.3/87.0	91.1/91.8	82.7/83.7	90.9/91.6	92.4/93.0	91.1/91.9	90.6/91.2	55.9/59.2	54.6/58.7	56.1/58.9	55.6/59.1	56.2/59.2	56.4/59.3	55.8/60.2

Table 20: Graph classification: Macro-F1 / Micro-F1 (%) by model and number of layers L. Cyan = best in family, **bold** = global best per dataset.

	MUTAG							PROTEINS						
	$L=2$	$L=3$	$L=4$	$L=5$	$L=6$	$L=7$	$L=8$	$L=2$	$L=3$	$L=4$	$L=5$	$L=6$	$L=7$	$L=8$
Homogeneous baselines														
GCN	89.6/90.0	89.6/90.0	89.6/90.0	89.6/90.0	89.6/90.0	89.6/90.0	89.6/90.0	76.4/80.4	71.8/77.7	69.5/71.4	75.8/78.6	74.1/76.8	69.0/70.5	75.8/78.6
GAT	89.6/90.0	89.6/90.0	89.6/90.0	89.6/90.0	89.6/90.0	58.3/60.0	84.0/85.0	56.2/56.2	75.8/78.6	71.8/77.7	48.8/70.5	71.8/77.7	74.6/77.7	52.2/52.7
GIN	89.6/90.0	94.9/95.0	89.9/90.0	85.0/85.0	85.0/85.0	89.6/90.0	84.7/85.0	70.3/71.4	51.3/71.4	64.1/69.6	70.6/72.3	–	66.8/67.9	62.7/68.8
SAGE	60.0/70.0	60.0/70.0	89.6/90.0	89.6/90.0	89.6/90.0	85.0/85.0	89.6/90.0	69.9/78.6	71.8/77.7	64.7/75.9	64.7/75.9	64.7/75.9	71.8/77.7	74.7/78.6
Heterogeneous baselines														
R-GCN	73.3/75.0	84.7/85.0	79.2/80.0	85.0/85.0	79.2/80.0	84.7/85.0	84.7/85.0	74.6/77.7	74.7/78.6	71.8/77.7	74.1/76.8	71.8/77.7	71.8/77.7	75.8/78.6
HAN	71.5/75.0	74.4/75.0	74.4/75.0	64.2/65.0	67.0/70.0	82.9/85.0	73.3/75.0	–	79.0/83.0	76.8/80.4	72.9/75.9	77.1/80.4	75.8/77.7	74.6/75.9
HGT	68.8/70.0	84.7/85.0	74.4/75.0	74.9/75.0	84.7/85.0	79.2/80.0	74.4/75.0	74.4/76.8	71.3/74.1	77.6/81.2	24.3/32.1	61.6/61.6	65.0/65.2	24.2/30.4
HetSheaf (ours)														
Diag	80.0/80.0	80.0/80.0	80.0/80.0	83.3/83.3	80.0/80.0	80.0/80.0	80.0/80.0	73.6/75.9	68.2/68.2	70.7/75.9	62.2/62.5	68.9/72.3	74.6/75.9	67.3/70.5
Bundle	79.8/80.0	84.7/85.0	84.7/85.0	94.9/95.0	84.7/85.0	84.7/85.0	79.2/80.0	78.8/80.4	74.8/75.9	75.5/76.8	50.7/50.9	75.8/77.7	74.7/76.8	69.8/70.5
General	79.8/80.0	76.2/80.0	74.9/75.0	89.9/90.0	73.3/75.0	94.9/95.0	78.0/80.0	33.3/37.5	38.0/40.2	78.7/81.2	64.1/66.1	63.2/63.4	71.3/74.1	64.2/67.0
Diag-P	89.9/90.0	96.0/96.0	95.7/95.7	**100.0/100.0**	94.9/95.0	95.7/95.7	96.0/96.0	40.6/42.9	63.2/74.1	71.6/74.1	41.3/67.9	76.4/80.4	55.4/55.4	78.9/**82.1**
Bundle-P	**100.0/100.0**	94.9/95.0	**100.0/100.0**	**100.0/100.0**	94.9/95.0	**100.0/100.0**	94.9/95.0	74.9/77.7	79.8/**82.1**	76.9/79.5	79.7/81.2	78.8/80.4	78.4/80.4	71.4/78.6
Gen.-P	**100.0/100.0**	**100.0/100.0**	**100.0/100.0**	**100.0/100.0**	**100.0/100.0**	**100.0/100.0**	**100.0/100.0**	77.8/80.4	78.1/80.4	76.7/77.7	73.9/75.9	78.7/81.2	75.8/78.6	**80.6/82.1**
SheafAN	89.0/90.0	79.2/80.0	84.0/85.0	94.9/95.0	89.9/90.0	94.9/95.0	94.9/95.0	73.8/78.6	74.7/78.6	77.2/80.4	76.4/78.6	76.7/78.6	75.6/77.7	75.2/76.8

	ENZYMES							NCI1						
	$L=2$	$L=3$	$L=4$	$L=5$	$L=6$	$L=7$	$L=8$	$L=2$	$L=3$	$L=4$	$L=5$	$L=6$	$L=7$	$L=8$
Homogeneous baselines														
GCN	11.2/25.0	12.7/28.3	9.3/23.3	11.2/15.0	10.9/15.0	11.9/18.3	9.8/11.7	63.2/63.5	64.2/65.0	–	63.2/63.5	64.2/65.0	63.2/63.5	64.2/65.0
GAT	14.2/16.7	5.6/20.0	12.0/18.3	9.7/20.0	5.9/21.7	5.9/21.7	6.9/18.3	64.2/65.0	62.0/62.0	43.1/50.4	35.5/55.0	63.2/63.5	61.3/65.0	35.5/55.0
GIN	9.6/21.7	8.1/11.7	9.1/21.7	13.4/18.3	10.1/18.3	16.3/21.7	20.2/28.3	64.2/65.0	60.9/63.5	64.2/65.0	63.0/64.2	62.0/62.0	63.2/64.7	55.7/57.4
SAGE	5.6/8.3	6.0/21.7	8.3/20.0	5.6/20.0	8.5/18.3	5.6/20.0	5.9/21.7	62.0/62.0	63.6/63.7	62.0/62.0	60.9/61.6	63.7/65.5	62.9/63.0	62.0/62.0
Heterogeneous baselines														
R-GCN	7.9/16.7	9.0/16.7	10.4/16.7	9.7/20.0	8.6/20.0	6.8/10.0	5.9/21.7	64.8/65.7	63.9/64.5	62.0/62.0	64.2/65.0	63.2/63.5	58.6/64.7	63.2/63.5
HAN	26.6/28.3	41.6/41.7	–	31.6/33.3	46.1/45.0	38.3/38.3	19.1/25.0	–	74.7/75.2	76.1/76.2	78.0/78.1	75.4/75.4	76.4/76.4	75.1/75.2
HGT	–	40.1/40.0	36.1/36.7	48.3/48.3	39.4/43.3	9.3/18.3	13.4/28.3	71.6/71.8	67.8/68.1	79.3/79.3	58.7/63.7	65.0/65.0	66.3/67.9	–
HetSheaf (ours)														
Diag	31.0/40.0	46.1/46.7	34.2/35.0	19.3/28.3	45.2/45.0	48.3/50.0	53.0/53.3	72.1/72.1	78.2/78.2	73.7/73.7	74.5/74.5	68.5/68.5	76.8/76.8	61.8/61.8
Bundle	38.2/43.3	50.2/53.3	38.6/41.7	36.6/36.7	51.2/53.3	22.4/30.0	**62.3/63.3**	68.0/68.0	62.1/62.1	59.9/59.9	62.8/62.8	53.2/53.2	60.4/60.4	62.3/62.3
General	31.9/36.7	47.7/50.0	46.9/46.7	43.5/45.0	49.9/51.7	17.8/28.3	50.7/51.7	61.7/61.7	48.7/48.7	64.4/64.4	28.1/28.1	63.2/63.2	67.5/67.5	62.1/62.1
Diag-P	14.4/21.7	10.1/21.7	11.3/23.3	20.6/23.3	14.9/21.7	25.6/30.0	22.7/28.3	–	61.4/61.4	62.1/62.1	49.1/49.1	23.0/23.0	46.6/46.6	62.1/62.1
Bundle-P	46.5/48.3	56.7/56.7	52.6/53.3	51.9/51.7	32.4/35.0	50.0/50.0	34.6/35.0	69.2/69.2	63.9/63.9	76.3/76.3	68.6/68.6	74.2/74.2	73.0/73.0	62.1/62.1
Gen.-P	41.2/45.0	41.7/43.3	48.1/48.3	45.5/46.7	37.1/41.7	38.0/41.7	34.4/36.7	67.2/67.2	72.3/72.3	73.0/73.0	72.9/72.9	69.9/69.9	**79.5/79.5**	62.1/62.1
SheafAN	27.0/30.0	34.9/36.7	46.5/50.0	37.7/41.7	38.6/41.7	49.3/51.7	30.9/35.0	62.1/62.1	68.2/68.2	75.1/75.1	69.7/69.7	66.5/66.5	75.1/75.1	73.6/73.6

Table 21: Link prediction: AUPR / AUROC (%) by model and number of layers L. Cyan = best in family, **bold** = global best per dataset.

	LastFM							MovieLens						
	$L=2$	$L=3$	$L=4$	$L=5$	$L=6$	$L=7$	$L=8$	$L=2$	$L=3$	$L=4$	$L=5$	$L=6$	$L=7$	$L=8$
Homogeneous baselines														
GCN	96.7/96.5	96.5/96.0	96.9/96.8	96.8/96.8	97.1/97.0	97.1/96.9	97.0/97.0	99.5/99.5	99.6/99.6	99.6/99.6	99.6/99.6	99.6/99.6	99.6/99.6	99.6/99.6
GAT	62.7/50.4	62.7/50.4	62.7/50.4	63.1/51.1	62.7/50.4	62.7/50.4	62.9/50.7	97.0/97.4	97.6/97.9	96.8/97.3	96.8/97.3	96.8/97.2	96.8/97.2	96.8/97.2
GIN	97.4/96.3	98.1/97.6	98.0/97.7	97.9/97.7	96.7/96.6	97.0/96.9	96.9/96.5	99.6/99.5	99.6/99.6	99.6/99.6	99.6/99.5	99.6/99.6	99.3/99.3	99.5/99.5
SAGE	92.7/89.2	88.0/86.4	83.5/80.1	95.1/92.7	90.0/86.4	88.5/86.5	88.0/86.6	98.8/98.8	98.8/98.8	99.1/99.0	99.2/99.1	99.1/99.0	99.2/99.0	99.2/99.1
Heterogeneous baselines														
R-GCN	97.8/97.8	97.7/97.7	97.8/97.8	98.0/97.9	97.9/97.9	97.8/97.8	97.8/97.9	99.4/99.4	99.4/99.4	99.4/99.4	99.3/99.3	99.3/99.3	99.2/99.2	99.3/99.3
HAN	75.8/73.4	86.2/83.0	76.7/74.2	88.3/83.3	86.6/81.8	87.5/82.8	87.2/82.7	63.0/51.1	63.0/51.1	63.0/51.1	63.6/52.2	63.1/51.2	63.1/51.3	63.1/51.4
HGT	83.8/71.3	86.1/74.1	85.8/74.5	85.9/74.4	86.0/74.5	86.1/74.5	85.9/74.4	68.9/61.7	70.8/63.6	63.8/52.6	77.7/74.0	67.6/58.9	63.0/51.1	63.0/51.0
HetSheaf (ours)														
Diag	98.4/98.1	98.4/98.2	98.4/98.2	**98.5**/98.2	98.5/98.2	98.4/98.2	98.4/98.2	**99.7**/99.6	**99.7**/99.6	**99.7**/99.6	**99.7**/99.6	**99.7**/99.6	**99.7**/99.6	**99.7**/99.6
Bundle	98.4/98.3	98.5/98.2	98.5/98.2	98.5/98.2	**98.5/98.3**	98.4/98.1	98.4/98.2	99.7/**99.6**	99.7/99.6	99.7/99.6	99.7/99.6	99.7/99.6	99.7/99.6	99.7/99.6
General	98.5/98.2	98.5/**98.3**	**98.5/98.3**	98.5/**98.3**	98.5/**98.3**	98.4/98.2	98.4/98.3	99.7/99.6	99.7/99.6	99.7/99.6	99.7/99.6	99.7/**99.6**	99.7/99.6	99.7/99.6
Diag-P	98.4/98.1	98.4/98.0	98.3/98.0	98.3/98.0	98.4/98.0	98.3/98.1	98.2/97.9	99.6/99.5	99.6/99.6	99.6/99.6	99.7/99.6	99.7/99.6	99.7/99.6	99.7/99.6
Bundle-P	98.3/97.8	98.3/98.0	98.4/98.1	98.3/98.1	98.2/98.0	98.2/97.9	98.2/97.8	99.7/99.6	99.7/99.6	99.7/99.6	99.7/99.6	99.7/99.6	99.7/99.6	99.7/99.6
Gen.-P	98.3/98.0	98.3/98.0	**98.5/98.3**	98.3/98.1	98.2/97.9	98.4/98.1	98.3/98.0	99.7/99.6	99.7/99.6	99.7/99.6	99.7/99.6	99.7/99.6	99.7/99.6	99.7/99.6
SheafAN	97.7/97.2	97.7/97.3	97.6/97.1	97.9/97.6	97.8/97.2	97.7/97.5	97.7/97.3	99.5/99.4	99.5/99.4	99.5/99.4	99.5/99.4	99.5/99.4	99.5/99.5	99.6/99.5

	Amazon							YouTube						
	$L=2$	$L=3$	$L=4$	$L=5$	$L=6$	$L=7$	$L=8$	$L=2$	$L=3$	$L=4$	$L=5$	$L=6$	$L=7$	$L=8$
Homogeneous baselines														
GCN	92.3/89.5	92.1/89.5	92.4/89.6	92.1/89.6	92.3/89.6	92.4/89.7	92.2/89.6	89.3/83.3	89.3/83.2	89.3/83.2	89.3/83.3	89.3/83.3	89.3/83.3	89.3/83.2
GAT	88.6/86.0	88.7/86.0	88.1/85.5	88.6/86.1	80.3/74.0	88.6/86.0	85.4/82.1	62.5/50.0	62.5/50.0	66.8/58.4	62.5/50.0	62.7/50.4	62.5/50.0	62.5/50.0
GIN	92.1/89.6	92.2/89.3	92.3/89.5	92.2/89.5	92.2/89.2	92.2/89.2	–	88.6/83.0	87.6/82.1	85.8/81.3	88.0/82.2	87.0/81.0	87.3/82.1	85.4/79.7
SAGE	89.3/86.2	90.0/87.1	90.8/88.0	90.3/87.3	89.9/87.0	90.4/87.5	90.4/87.5	71.4/66.2	81.4/75.6	84.5/80.2	83.1/78.9	78.5/75.9	79.2/76.1	85.6/79.2
Heterogeneous baselines														
R-GCN	93.4/91.4	92.9/91.0	92.8/91.1	92.8/90.9	87.2/83.3	92.1/90.1	88.6/86.0	91.7/88.3	91.6/88.2	91.7/88.2	91.5/88.2	91.4/88.1	91.4/88.0	91.4/88.1
HAN	94.0/92.3	94.9/93.4	94.3/92.6	91.3/88.7	94.1/92.3	95.2/93.6	94.9/93.1	90.3/87.0	91.1/86.8	93.5/90.1	92.2/88.3	90.0/85.8	90.0/85.2	89.1/84.4
HGT	96.0/95.3	96.1/95.6	95.6/95.3	95.9/95.4	96.1/95.7	96.2/95.6	95.9/95.5	94.2/91.5	94.6/91.8	95.0/92.5	94.5/91.6	94.2/91.6	94.6/91.7	94.1/91.6
HetSheaf (ours)														
Diag	**93.9**/91.6	93.8/91.5	93.8/91.6	93.8/91.5	93.8/91.5	93.9/91.6	**93.9**/91.6	89.5/83.5	90.6/85.4	90.7/86.0	90.4/85.5	90.5/85.7	90.5/85.4	89.4/83.4
Bundle	93.6/91.3	93.8/91.6	93.8/91.5	93.7/91.5	93.8/91.5	93.8/91.6	93.7/91.4	**90.7**/85.9	90.3/84.9	89.3/83.2	90.1/84.4	89.5/83.4	89.2/83.2	89.3/83.4
General	93.9/**91.7**	93.9/91.6	93.8/91.6	93.8/91.6	93.8/91.5	93.8/91.5	93.8/91.6	91.1/86.8	90.7/86.0	**91.3/87.3**	90.4/85.4	**91.3/87.3**	90.6/86.0	90.4/85.6
Diag-P	93.5/90.9	93.3/90.7	93.1/90.5	93.1/90.3	93.2/90.6	93.3/90.5	93.3/90.8	90.6/85.4	90.5/85.2	90.6/85.8	90.7/86.5	89.4/83.3	90.1/85.3	89.4/83.2
Bundle-P	93.5/91.1	93.6/90.9	93.5/91.1	93.5/91.1	93.5/91.1	93.6/91.1	93.6/91.2	89.4/83.3	89.4/83.3	89.4/83.3	89.4/83.4	89.4/83.2	89.4/83.3	89.4/83.3
Gen.-P	93.0/90.3	93.8/91.4	**93.8/91.7**	93.8/91.5	93.6/91.1	93.6/91.3	92.9/90.3	–	–	–	–	–	89.2/83.0	–
SheafAN	92.0/88.7	92.1/88.9	91.7/88.4	91.8/88.6	92.2/89.1	91.9/88.7	92.0/89.2	86.9/80.4	87.0/80.5	86.7/80.5	86.8/80.2	87.0/80.5	86.6/80.2	87.1/80.6

train/validation/test splits, these comparisons should be interpreted as contextual rather than primary.

Among the additional baselines, HAGNN [82] is the strongest competitor on DBLP and IMDB under the HGB protocol. Although presented as a hybrid framework, it still relies on a fused meta-path graph in its first aggregation stage and therefore inherits core limitations of meta-path-based methods: it requires expert-selected meta-paths, restricts intratype aggregation to those manually specified relational templates, and remains bounded by how well the chosen meta-paths approximate the true semantics of the graph. By contrast, HetSheaf dispenses with meta-path engineering and models inter-type interactions directly on the original heterogeneous topology through learned restriction maps. As shown in Table 22, this yields competitive or superior performance without any hand-designed meta-path machinery.

Tables 22 and 23 collect these broader comparisons for node classification and link prediction, respectively.

B.4 Implementation Details

We now collect the implementation details underlying all experiments. We begin with the hardware and software environment, then describe the downstream decoders and loss functions used for the three tasks, followed by the parameterisation of restriction maps and the full family of heterogeneous sheaf predictors. We then report computational complexity and overhead, before detailing the heterogeneous extensions of Sheaf Attention Networks and Polynomial Neural Sheaf Diffusion used in our experiments.

Table 22: Node classification results (Macro-F1 / Micro-F1). The top three models are coloured by **First**, **Second** and **Third**. "OOM" denotes an out-of-memory error. Source: † Lv et al. [48] (HGB split: 24/6/70 %); ‡ Zhu et al. [82] (HGB split); § Lin et al. [47] (20 % train split; results not directly comparable with HGB rows).

	DBLP		IMDB		ACM	
Model	Macro-F1	Micro-F1	Macro-F1	Micro-F1	Macro-F1	Micro-F1
GTN‡	65.21	66.23	59.26	64.07	91.31†	91.20†
HetSANN‡	84.08	84.96	49.25	57.47	90.02†	89.91†
HetGNN‡	92.77	93.23	47.87	50.83	85.91†	86.05†
CKD‡	92.52	92.80	60.30	65.98	–	–
HGNN-AC‡	92.97	93.43	56.63	63.85	–	–
RSHN†	93.34	93.81	59.85	64.22	90.50	90.32
MAGNN‡	93.16	93.65	56.92	65.11	90.88†	90.77†
R-HGNN‡	93.31	93.80	61.39	66.03	–	–
AMHGNN‡	93.71	94.08	63.38	67.29	–	–
SimpleHGN‡	93.81	94.26	63.53	67.42	**93.42**†	**93.35**†
BPHGNN‡	93.89	94.50	64.01	**67.77**	–	–
HINormer‡	**94.57**	**94.94**	**64.65**	**67.83**	–	–
HAGNN‡	**95.06**	**95.40**	**65.57**	**68.62**	–	–
MRGCN§	89.53	90.51	45.62	47.78	87.64	87.52
SSDCM§	89.44	89.94	49.45	59.16	87.72	87.65
MHGCN§	90.90	92.17	50.54	64.20	88.96	89.19
HGATE§	90.25	91.15	41.99	52.10	88.73	88.70
MGDCR§	92.04	92.43	45.03	59.05	89.03	88.87
HMGE§	91.56	92.35	32.93	57.35	90.80	90.66
AMOGCN§	92.02	92.46	50.92	63.60	91.72	91.39
HGNN-AR^{2}§	92.54	93.02	52.51	61.36	**92.42**	**92.36**
Sheaf-NSD	92.96	93.42	61.96	65.11	92.05	91.97
HetSheaf (ours)	**94.97**	**95.39**	**64.54**	66.65	**94.31**	**94.24**

Table 23: Link prediction results (AUROC). The top three models are coloured by **First**, **Second** and **Third**. "OOM" denotes an out-of-memory error. Source: † Lv et al. [48]; ‡ Zhu et al. [82].

Model	**Amazon**	**LastFM**	**PubMed**
MAGNN‡	–	56.81	OOM
HetGNN†	77.74	62.09	73.63
GATNE†	77.39	66.87	63.39
AutoAC‡	–	66.64	82.95
SimpleHGN†	**93.40**	67.16‡	83.39‡
HAGNN‡	–	**67.33**	**84.00**
Sheaf-NSD	**90.04**	**96.58**	**93.65**
HetSheaf (ours)	**91.71**	**98.33**	**93.98**

B.4.1 Hardware and Software Setup

All experiments were conducted on an AWS EC2 instance running Ubuntu 24.04 LTS, equipped with an NVIDIA L4 GPU (CUDA 12.8, cuDNN 9.1). The codebase is implemented in Python 3.13.2 using PyTorch 2.8 and PyTorch Geometric 2.7 [21, 22], with dependency management handled by `uv`[11]. Hyperparameter searches were performed via Weights & Biases[12] sweeps. These choices were kept fixed across models to ensure that empirical differences reflect modelling choices rather than software-stack variation.

B.4.2 Downstream Decoders and Loss Functions

The sheaf encoder produces node-level hidden states that are then consumed by a task-specific decoder. Since node classification, link prediction, and graph classification place different demands on the output space, we use standard downstream decoders tailored to each task while leaving the sheaf backbone unchanged.

Node classification. After the Sheaf GNN processes the graph, the final node embeddings are fed into an MLP to generate the classification logits. For multiclass tasks, we use a

[11] `https://docs.astral.sh/uv/`
[12] `https://wandb.ai`

softmax activation with cross-entropy loss; for multilabel settings, we use a sigmoid activation with binary cross-entropy loss.

Link prediction. For each node pair u, v and relation type r, the model outputs the score

$$P(u \sim_r v) = Decoder(\mathbf{H}_u, \mathbf{H}_v) \tag{82}$$

which gives the probability that u and v are connected by a relation of type r. Since this is a binary prediction problem, the loss is binary cross-entropy.

The decoder can take several forms. A dot-product decoder first computes the inner product and then applies a sigmoid:

$$Decoder(\mathbf{H}_u, \mathbf{H}_v) = \sigma(\langle \mathbf{H}_u, \mathbf{H}_v \rangle). \tag{83}$$

DistMult [73] instead computes the bilinear score

$$Decoder(\mathbf{H}_u, \mathbf{H}_v) = \sigma\big(\mathbf{H}_u^\intercal \mathbf{W}_{\psi(u,v)} \mathbf{H}_v\big), \tag{84}$$

where $\mathbf{W}_{\psi(u,v)} \in \mathbb{R}^{d_L \times d_L}$ is a learnable weight matrix for each edge type. Finally, a concat decoder concatenates the two embeddings, applies a one-layer MLP, and then a sigmoid:

$$Decoder(\mathbf{H}_u, \mathbf{H}_v) = \sigma(\mathrm{MLP}(\mathbf{H}_u \| \mathbf{H}_v)). \tag{85}$$

Following Lv et al. [48], we treat the choice of decoder as a hyperparameter.

B.4.3 Restriction Map Parameterisations

Each restriction map is represented as a $d \times d$ block matrix and parameterised by the matrix-valued function $\Phi(\mathbf{x}_u, \mathbf{x}_v, \phi(u), \phi(v), \psi(e))$. We briefly describe how each family is constructed.

Diagonal restriction maps. The predictor outputs d values, which define the diagonal entries of the restriction map.

General restriction maps. The predictor outputs d^2 values, which are reshaped into a full $d \times d$ matrix.

Orthogonal restriction maps. For orthogonal restriction maps $\mathcal{F}_{\trianglelefteq \cdot} \in \mathrm{O}(d)$, we consider two parameterisations. The first uses Householder products: the MLP outputs $\lfloor \frac{d(d-1)}{2} \rfloor$ parameters, which are converted by the Torch Householder library [53] into a product of Householder reflections covering the full orthogonal group $\mathrm{O}(d)$. The second uses the Cayley transform: the MLP outputs d^2 values forming a matrix $\mathcal{F}'$, from which the restriction map is obtained as

$$\mathcal{F}_{\trianglelefteq \cdot} = (\mathbf{I}_d - \mathbf{S})(\mathbf{I}_d + \mathbf{S})^{-1}, \qquad \mathbf{S} = \tfrac{1}{2}(\mathcal{F}' - \mathcal{F}'^\top). \tag{86}$$

Since the Cayley transform maps skew-symmetric matrices to $\mathrm{SO}(d)$, it can represent rotations but not reflections. In practice, the Householder and Cayley parameterisations provide different geometric biases, and the latter is the default for the attention predictors and SheafAN variant following Barbero et al. [2].

B.4.4 Full Description of Heterogeneous Sheaf Predictors

We next give the complete definition of the heterogeneous predictor family used in the experiments. These variants differ in two ways: first, in *which* typed signals are provided to the predictor (node features, node types, edge types, or combinations thereof), and second, in *how* this information is routed (shared predictor versus edge-type-specific ensemble). This family lets us disentangle the contribution of feature-level conditioning from the contribution of pure structural type information.

Let $\mathcal{G}$ be a heterogeneous graph with node features $\mathbf{x}_u \in \mathbb{R}^f$, one-hot node-type vector $\mathbf{e}_{\phi(u)}$, and one-hot edge-type vector $\mathbf{e}_{\psi(e)}$ for edge $e := (u, v)$. All predictors below are compatible with diagonal, orthogonal, and general restriction-map types.

The predictors are broadly categorised by whether they append type embeddings to node features, discard features in favour of pure type signals, or use specialised routing through edge-type-specific experts.

Table 24: Summary of Heterogeneous Sheaf Predictors and their inputs.

Predictor Name	Acronym Meaning	Inputs Used
HetSheaf-NSD	**N**eural **S**heaf **D**iffusion	$\mathbf{x}_u, \mathbf{x}_v$
HetSheaf-TE	**T**ype **E**mbeddings	$\mathbf{x}, \phi, \psi$
HetSheaf-NE	**N**ode **E**mbeddings	$\mathbf{x}, \phi$
HetSheaf-EE	**E**dge **E**mbeddings	$\mathbf{x}, \psi$
HetSheaf-types	All **types** only	ϕ, ψ
HetSheaf-NT	**N**ode **T**ypes only	ϕ
HetSheaf-ET	**E**dge **T**ypes only	ψ
HetSheaf-ensemble	**Ensemble** of MLPs	$\mathbf{x}$ (via $\mathrm{MLP}_{\psi(e)}$)

Sheaf-NSD.

$$\mathcal{F}_{u \trianglelefteq e} \coloneqq \mathrm{MLP}(\mathbf{x}_u \| \mathbf{x}_v). \tag{87}$$

HetSheaf-TE.

$$\mathcal{F}_{u \trianglelefteq e} \coloneqq \mathrm{MLP}\big(\mathbf{x}_u \| \mathbf{x}_v \| \mathbf{e}_{\phi(u)} \| \mathbf{e}_{\phi(v)} \| \mathbf{e}_{\psi(e)}\big). \tag{88}$$

HetSheaf-NE.

$$\mathcal{F}_{u \trianglelefteq e} \coloneqq \mathrm{MLP}\big(\mathbf{x}_u \| \mathbf{x}_v \| \mathbf{e}_{\phi(u)} \| \mathbf{e}_{\phi(v)}\big). \tag{89}$$

HetSheaf-EE.

$$\mathcal{F}_{u \trianglelefteq e} \coloneqq \mathrm{MLP}\big(\mathbf{x}_u \| \mathbf{x}_v \| \mathbf{e}_{\psi(e)}\big). \tag{90}$$

HetSheaf-types.

$$\mathcal{F}_{u \trianglelefteq e} \coloneqq \mathrm{MLP}\big(\mathbf{e}_{\phi(u)} \| \mathbf{e}_{\phi(v)} \| \mathbf{e}_{\psi(e)}\big). \tag{91}$$

HetSheaf-NT.

$$\mathcal{F}_{u \trianglelefteq e} \coloneqq \mathrm{MLP}\big(\mathbf{e}_{\phi(u)} \| \mathbf{e}_{\phi(v)}\big). \tag{92}$$

HetSheaf-ET.

$$\mathcal{F}_{u \trianglelefteq e} \coloneqq \mathrm{MLP}\big(\mathbf{e}_{\psi(e)}\big). \tag{93}$$

HetSheaf-ensemble.

$$\mathcal{F}_{u \trianglelefteq e} \coloneqq \mathrm{MLP}_{\psi(e)}(\mathbf{x}_u \| \mathbf{x}_v), \tag{94}$$

where $\psi(e)$ selects the edge-type-specific predictor. This yields strict expressivity gains over a single shared MLP at the cost of parameters that scale linearly with the number of edge types.

B.4.5 HetSheaf Computational Complexity and Overhead

We derive the per-layer computational complexity of each HetSheaf predictor following the analysis framework of Bodnar et al. [4]. The aim is twofold: first, to make explicit how type conditioning changes the asymptotic cost relative to vanilla NSD; and second, to quantify the practical overhead of the sheaf-based approach relative to strong non-sheaf baselines.

Consider a heterogeneous graph with n nodes, m edges, s node types, and t edge types. Let d denote the stalk dimension and f the number of feature channels, so that $c = d \times f$. We distinguish between diagonal restriction maps, which require only d parameters per map, and non-diagonal maps, which require d^2 parameters and may incur an additional $\mathcal{O}(d^3)$ orthogonalisation cost in the orthogonal case. Table 25 summarises the resulting per-layer complexities.

Several observations follow. First, the additive cost introduced by type-aware predictors relative to NSD is governed by the number of appended type indicators, and is therefore typically negligible since heterogeneous benchmarks usually satisfy $s, t \ll c$. Second, diagonal maps scale linearly in d per edge, while general or orthogonal maps scale quadratically in d because of the full matrix-valued transport. Third, the ensemble predictor is the only

Table 25: **Per-layer computational complexities of heterogeneous sheaf predictors and baselines.** Notation: n nodes, m edges, s node types, t edge types, stalk dimension d, channels f, $c = df$ (or $c = f$ for non-sheaf models), attention heads h. Diagonal restriction maps are denoted (diag); general or orthogonal maps are denoted (non-diag).

Model	**Computational complexity**
Non-sheaf baselines	
GCN	$\mathcal{O}(nc^2 + mc)$
GAT	$\mathcal{O}(nc^2h + mch)$
HAN	$\mathcal{O}(tnc^2h + tmch)$
HetSheaf predictors (diag)	
NSD	$\mathcal{O}(nc^2 + mdc)$
TE	$\mathcal{O}(nc^2 + md(c + s + t))$
NE	$\mathcal{O}(nc^2 + md(c + s))$
EE	$\mathcal{O}(nc^2 + md(c + t))$
types	$\mathcal{O}(nc^2 + md(s + t))$
NT	$\mathcal{O}(nc^2 + mds)$
ET	$\mathcal{O}(nc^2 + mdt)$
ensemble	$\mathcal{O}(nc^2 + mdct)$
HetSheaf predictors (non-diag)	
NSD	$\mathcal{O}(n(c^2 + d^3) + md^2(c + d))$
TE	$\mathcal{O}(n(c^2 + d^3) + md^2(c + s + t + d))$
NE	$\mathcal{O}(n(c^2 + d^3) + md^2(c + s + d))$
EE	$\mathcal{O}(n(c^2 + d^3) + md^2(c + t + d))$
types	$\mathcal{O}(n(c^2 + d^3) + md^2(s + t))$
NT	$\mathcal{O}(n(c^2 + d^3) + md^2s)$
ET	$\mathcal{O}(n(c^2 + d^3) + md^2t)$
ensemble	$\mathcal{O}(n(c^2 + d^3) + md^2(c + dt))$

variant whose cost scales multiplicatively with the number of edge types, reflecting the use of independent edge-type-specific MLPs. Even so, for the small stalk dimensions and modest type cardinalities used in this work, the practical overhead remains limited.

To complement the asymptotic analysis, Table 26 reports parameter counts and wall-clock costs on the node classification benchmarks. Despite not yet using the most heavily optimised PyG kernels, the sheaf-based models remain substantially smaller than the strongest heterogeneous baselines in terms of parameters, while often matching or surpassing them in predictive performance.

Table 26: **HetSheaf computational overhead on node classification tasks.** The highest performing heterogeneous baseline architectures are highlighted with the highest performing HetSheaf model shown in **bold**.

	ACM		DBLP		IMDB	
	Runtime (s)	# params	Runtime (s)	# params	Runtime (s)	# params
R-GCN	22.1	43M	30.3	87.8M	25.9	72.1M
HAN	28.9	2.2M	18.5	1.9M	15.8	3.4M
HGT	1025.8	7.2M	648.3	6.2M	939.9	10.5M
Sheaf-NSD	47.6	155K	73.3	1.7M	40.5	1.6M
HetSheaf-TE (ours)	44.7	155K	**79.1**	**1.7M**	40.1	1.6M
HetSheaf-ensemble (ours)	55.7	174K	83.2	1.8M	44.9	1.9M
HetSheaf-NE (ours)	47.3	155K	89.6	1.7M	12.0	1.1M
HetSheaf-EE (ours)	**49.6**	**155K**	85.9	1.7M	41.8	1.6M
HetSheaf-NT (ours)	42.0	154K	82.4	1.7M	**38.6**	**1.6M**
HetSheaf-ET (ours)	45.1	154K	80.5	1.7M	39.8	1.6M

B.4.6 Heterogeneous Sheaf Attention Networks

Sheaf Attention Networks (SheafANs) [2] replace Laplacian-based propagation with attention-weighted aggregation over sheaf transports. Since HetSheaf is intended to be model-agnostic, it is important to verify that the inferred heterogeneous sheaf can be consumed not only by NSD-style diffusion layers but also by attention-based sheaf architectures. This subsection therefore describes how SheafAN is extended to the heterogeneous setting.

Their geometric backbone is the sheaf adjacency operator $\hat{A}_{\mathcal{F}} \in \mathbb{R}^{nd \times nd}$, whose off-diagonal blocks are defined as

$$P_{ij} = \mathcal{F}_{i \trianglelefteq e}^{\top} \mathcal{F}_{j \trianglelefteq e} \in \mathbb{R}^{d \times d}, \tag{95}$$

for $e = (i, j)$. In the original homogeneous formulation, the scalar attention coefficients are

$$\Lambda_{ij} = \frac{\exp\big(\mathrm{LeakyReLU}\big(\mathbf{a}^{\top}[\mathbf{W}\mathbf{x}_i \| \mathbf{W}\mathbf{x}_j]\big)\big)}{\sum_{k \in \mathcal{N}_i} \exp(\mathrm{LeakyReLU}(\mathbf{a}^{\top}[\mathbf{W}\mathbf{x}_i \| \mathbf{W}\mathbf{x}_k]))}. \tag{96}$$

To adapt SheafAN to heterogeneous graphs, we augment the attention input with node- and edge-type embeddings:

$$\Lambda_{ij} = \frac{\exp\big(\mathrm{LeakyReLU}\big(\mathbf{a}^{\top}[\mathbf{W}\mathbf{x}_i \| \mathbf{W}\mathbf{x}_j \| \mathbf{e}_{\phi(i)} \| \mathbf{e}_{\phi(j)} \| \mathbf{e}_{\psi((i,j))}]\big)\big)}{\sum_{k \in \mathcal{N}_i} \exp\big(\mathrm{LeakyReLU}\big(\mathbf{a}^{\top}[\mathbf{W}\mathbf{x}_i \| \mathbf{W}\mathbf{x}_k \| \mathbf{e}_{\phi(i)} \| \mathbf{e}_{\phi(k)} \| \mathbf{e}_{\psi((i,k))}]\big)\big)}. \tag{97}$$

At the same time, the transport maps P_{ij} are computed from the heterogeneous restriction maps inferred by HetSheaf. Hence both geometric transport and attention become aware of the typed structure of the graph, while the overall attentive sheaf architecture itself remains unchanged.

B.4.7 Heterogeneous Polynomial Neural Sheaf Diffusion

Polynomial Neural Sheaf Diffusion (PolyNSD) [5] extends first-order sheaf diffusion by replacing repeated local propagation with a learnable spectral polynomial of the sheaf Laplacian. Since PolyNSD is structurally quite different from NSD, its heterogeneous extension provides a strong demonstration of the model-agnosticity of HetSheaf.

For a sheaf Laplacian $L_{\mathcal{F}}$ with spectral decomposition $L_{\mathcal{F}} = U \Lambda U^{\top}$, a polynomial filter $p(L_{\mathcal{F}}) = \sum_{k=0}^{K} c_k L_{\mathcal{F}}^k$ acts diagonally in the sheaf Fourier basis, scaling the i-th mode by $p(\lambda_i)$. PolyNSD uses a Chebyshev parameterisation on a rescaled operator

$$\tilde{L}_{\mathcal{F}} = \frac{2}{\lambda_{\max}} L_{\mathcal{F}} - \mathbf{I}, \tag{98}$$

so that the polynomial remains numerically stable on $[-1, 1]$. The filter is then written as

$$p(\tilde{L}_{\mathcal{F}}) = \sum_{k=0}^{K} \alpha_k T_k(\tilde{L}_{\mathcal{F}}), \qquad \{\alpha_k\} = \mathrm{softmax}(\boldsymbol{\eta}) \tag{99}$$

with the Chebyshev basis evaluated through the usual three-term recurrence.

Our framework extends PolyNSD to heterogeneous graphs without altering this spectral machinery. The only difference is that the Laplacian is now built from a heterogeneous sheaf whose restriction maps are predicted by

$$\mathcal{F}_{u \trianglelefteq e := (u,v)} = \boldsymbol{\Phi}(\mathbf{x}_u, \mathbf{x}_v, \phi(u), \phi(v), \psi(e)) \tag{100}$$

The resulting heterogeneous sheaf Laplacian $L_{\mathcal{F}}$ is then used directly inside the polynomial filter. In this way, the same higher-order spectral diffusion mechanism extends naturally to the heterogeneous setting, again illustrating that once heterogeneity is absorbed into the inferred sheaf, the downstream sheaf architecture itself does not require ad hoc redesign.